PERSONALISED STUDY GUIDE

AIoT ENGINEER

COMPLETE MASTER STUDY GUIDE

EDGE AI | IOT & MQTT | PYTHON | PYTORCH · TFLITE | VIBE CODING | N8N AUTOMATION | LANGCHAIN | JAPANESE N4 | RASPBERRY PI | DSA C++ | AUTONOMOUS VEHICLES | CLAUDE API

163	12	10	40+	13	200+
PAGES	CHAPTERS	PROJECTS	CODE SAMPLES	APPENDICES	TOPICS

— YOUR CAREER ROADMAP —

2025	2026	2029	2033
B.Tech CS-IT Malla Reddy Univ	Remote Internship Edge AI / IoT	Japan AIoT Engr Panasonic / Omron	Vizag Startup Indo-Japan Tech

PERSONALISED FOR

B.Tech CS-IT + EEE Diploma | Malla Reddy University, Hyderabad

DSA C++ · Python · PyTorch · TFLite · MERN · MySQL · Linux · Japanese N5

2025 — 2033

YOUR DECADE. YOUR ROADMAP.

Chapter 1: AI and the Future of Engineering

Artificial Intelligence is transforming every domain of engineering — but not in the way most news headlines suggest. The narrative that "AI will replace all engineers" is both correct and deeply misleading at the same time. Understanding exactly what is happening, and what is not happening, is the most important thing you can do for your career right now.

1.1 What AI Is Actually Replacing

AI is replacing specific tasks, not entire professions. More precisely, it is replacing tasks that are repetitive, pattern-based, and do not require physical interaction with the real world. The jobs that are disappearing first are:

- Data entry and basic report generation — AI can read, extract, and summarise structured data faster than any human
- Generic CRUD application development — AI can write a basic REST API or a React component in seconds
- Basic QA testing — AI can generate test cases and run regression tests automatically
- Junior content writing — AI can produce first drafts of documentation, emails, and reports
- Simple data analysis — AI can compute statistics, plot graphs, and find correlations without a data analyst

Notice what is NOT on that list: physical hardware engineering, sensor integration, real-time embedded systems, anything requiring a physical presence, and domain-specific expertise that took years to build. This is exactly where AIoT engineering lives.

1.2 Why AIoT Engineers Are Safe — And In Demand

AIoT (Artificial Intelligence of Things) sits at the intersection of three domains that AI cannot fully automate: physical hardware, embedded software, and machine learning. Here is why each layer protects you:

Layer 1 — Physical Hardware

Sensors, microcontrollers, Raspberry Pi boards, motors, and industrial machines exist in the physical world. An AI cannot solder a circuit, debug a faulty sensor reading caused by electromagnetic interference, or physically install an edge device in a factory. Your EEE diploma gives you exactly this physical engineering foundation that pure software engineers lack entirely.

Layer 2 — Embedded Software

Writing real-time code that runs on a device with 512MB of RAM, no swap space, and a hard latency requirement of 50ms is not something a generic AI tool handles well. It requires knowledge of Linux internals, memory management, and hardware-specific drivers. This expertise takes years to develop.

Layer 3 — Domain ML

Training a general image classification model is easy. Training a model that detects a specific type of bearing wear in a factory machine, on noisy sensor data, with limited labelled examples, while running on a Raspberry Pi at 15 FPS — that requires deep domain knowledge. AI tools help you write the code, but you must know what to ask them to write.

> 📌 **Note:** The engineer who gets replaced is the one who writes boilerplate code all day without understanding the domain. The engineer who thrives is the one who uses AI to eliminate boilerplate and focus entirely on the hard domain problems only they can solve.

1.3 The AI Augmentation Model — Your Mental Framework

Instead of thinking "AI vs Engineer", adopt the framework of AI Augmentation. Every task you do falls into one of three categories:

Task Type	Example	Your Role	AI Role
Automate fully	Writing boilerplate code	Define what you want	Write the code
AI-assisted	Debugging a sensor driver	Know what is wrong	Suggest fixes
Human only	Hardware integration + testing	Do everything	Cannot help

Your goal over the next 4 years is to push as many tasks as possible into the first two categories, so you spend almost all your time on the third category — the work only you can do.

1.4 The Three AI Skills Every Engineer Needs Now

Regardless of domain, every engineer in 2026 needs exactly three AI-related skills. Without them, you will fall behind even within your own specialisation.

Skill 1: Prompt Engineering

The ability to communicate precisely with an AI model to get exactly the output you need. This is the new "knowing how to use Google" — it seems trivial but there is enormous variation in how effectively different people use AI tools. A well-crafted prompt can get you a complete, working, production-quality solution. A poorly crafted prompt wastes 30 minutes.

Skill 2: AI-Assisted Development (Vibe Coding)

Using AI code editors like Cursor to write code by describing what you want in natural language, reviewing the output critically, and iterating. The critical review part is non-negotiable — if you cannot review AI-generated code, you will ship broken systems. This is why your DSA and Python fundamentals still matter enormously.

Skill 3: AI Agent Building

Creating small autonomous systems that use an LLM as their reasoning engine to complete multi-step tasks. For an AIoT engineer, this means agents that monitor sensor data, detect anomalies, and trigger responses automatically — without human intervention for every decision.

1.5 The Japan AIoT Market — Why Your Timing Is Perfect

Japan is undergoing the largest industrial automation transformation in its history. The country has a severe labour shortage due to an ageing population, and the government has set national targets for factory automation, smart infrastructure, and agricultural robotics. This is creating extraordinary demand for engineers who combine:

- Embedded systems and hardware knowledge (your EEE diploma)
- AI/ML for edge deployment (your PyTorch + TFLite path)
- IoT connectivity and protocols (MQTT, OPC-UA — your Month 3-4 learning)
- Japanese language ability (your JLPT path to N2)

The overlap of all four qualifications in a single engineer is genuinely rare. Japanese companies like Panasonic, Omron, Hitachi, Fanuc, Keyence, and dozens of mid-size manufacturers are actively competing for this profile. Your 2029 target is not ambitious — it is well-timed.

> 💡 **Tip:** When you reach Japan, emphasise your EEE background in every interview. Japanese manufacturing engineers deeply respect hardware knowledge. An Indian engineer who can both write Edge AI code AND understand the machine it is controlling is someone they have not seen before.

Chapter 2: Vibe Coding and Cursor IDE — Complete Guide

Vibe coding is the practice of building software by describing what you want in natural language and letting an AI code editor generate the implementation. The term was coined in early 2025 and describes a workflow that has become the dominant way professional engineers write code in 2026.

2.1 What Is Cursor IDE

Cursor is an AI-first code editor built on top of VS Code. It uses large language models (Claude, GPT-4, and others) to understand your codebase and generate code in context. Unlike GitHub Copilot which only autocompletes, Cursor can:

- Generate entire files from a description
- Refactor existing code while preserving logic
- Debug errors by reading your error message and suggesting fixes
- Answer questions about any part of your codebase
- Apply multi-file edits in one command
- Connect to MCP servers to call external tools

> 📌 **Note:** Cursor is free for 2000 completions per month. The Pro plan ($20/month) gives unlimited completions. Start with the free plan — it is more than enough for learning.

2.2 Installation and Setup

Step 1: Download and Install

Download Cursor from cursor.com. It installs exactly like VS Code. On first launch, it asks you to import your existing VS Code settings — do this. All your themes, extensions, and keybindings carry over.

Step 2: Choose Your AI Model

Cursor supports multiple models. For your use case, use Claude Sonnet 4.6 (best for code generation and reasoning) or GPT-4o (faster, slightly less accurate). Switch between them with Ctrl+/ in the chat panel.

Step 3: Index Your Project

When you open a folder, Cursor indexes the entire codebase. This allows the AI to understand imports, function signatures, and data structures across files. For a Raspberry Pi project, open the entire project folder — not individual files.

Step 4: Configure Cursor Rules

Cursor Rules are instructions that apply to every AI interaction in a project. Create a file called .cursorrules in your project root. For your AIoT projects, use this template:

```
You are an expert AIoT engineer specialising in Edge AI and Raspberry Pi
development.
Always write Python code compatible with Python 3.10+ and Raspberry Pi OS.
Use TensorFlow Lite for all model inference — never full TensorFlow.
Use paho-mqtt for all MQTT communication.
```

```
Add type hints to every function. Add docstrings to every class and function.
When writing sensor code, always include error handling for I2C and SPI failures.
Prefer async patterns for I/O-heavy operations.
```

2.3 The Five Core Cursor Features

Feature 1: Ctrl+K — Inline Edit

Select any block of code and press Ctrl+K to open an inline prompt. Describe what you want to change. Cursor applies the edit directly. This is the fastest way to refactor existing code.

Feature 2: Ctrl+L — Chat Panel

Opens a full chat panel where you can ask questions about your code, request new features, or debug problems. The chat has full context of your entire indexed codebase.

Feature 3: Ctrl+I — Composer (Agent Mode)

The most powerful feature. Describe a complete feature or project and Cursor will create and edit multiple files automatically. It shows you each change before applying it. Use this for building entire modules.

Feature 4: @ Mentions

In any chat message, type @ to reference specific files, folders, or documentation. Example: "Refactor @sensor_reader.py to use async/await based on the pattern in @utils.py". This gives the AI precise context.

Feature 5: Codebase-wide Search

Ask "How does the MQTT connection get established in this project?" and Cursor searches the entire codebase to find the answer. Essential for understanding unfamiliar code.

2.4 Effective Prompt Patterns for AIoT Code

The quality of your prompt directly determines the quality of the output. Here are the prompt patterns that work best for AIoT engineering:

Pattern 1: Role + Task + Constraints

```
Act as an embedded Python developer. Write a sensor reader class for DHT22
on Raspberry Pi GPIO pin 4. Requirements: (1) read every 5 seconds (2) handle
sensor failures gracefully (3) return a dataclass with temperature, humidity,
timestamp (4) log errors to a file (5) compatible with Python 3.10+
```

Pattern 2: Provide Example Input/Output

```
I need a function that converts raw ADC readings to engineering units.
Input example: {"raw": 2048, "sensor": "pressure", "range_min": 0, "range_max": 10}
Output example: {"value": 5.0, "unit": "bar", "timestamp": "2026-09-07T17:30:00"}
Write the function with full type hints and unit tests.
```

Pattern 3: Iterative Refinement

Never try to get the perfect result in one prompt. Start broad, then refine:

- Prompt 1: "Write a basic MQTT publisher for sensor data"
- Prompt 2: "Add reconnection logic with exponential backoff"
- Prompt 3: "Add a message queue so data is not lost during disconnection"
- Prompt 4: "Add unit tests for the queue logic"

> 💡 **Tip:** Treat Cursor like a junior developer who is very fast but needs clear instructions. If the output is wrong, do not rewrite it manually — explain what is wrong and ask Cursor to fix it. This trains you to give better prompts.

2.5 Vibe Coding Workflow for AIoT Projects

Here is the exact workflow to use for every AIoT project you build:

Step 1: Write a PRD (Product Requirements Document)

Before writing any code, create a file called PRD.md in your project folder and describe exactly what you are building. Include: what problem it solves, what hardware it uses, what software components it needs, and what the expected output is. Cursor will use this as context for every code generation request.

Step 2: Generate the Project Structure

```
Open Composer (Ctrl+I) and type:
"Read PRD.md and create the complete project structure with
empty files, a requirements.txt, and a README.md"
```

Step 3: Implement Module by Module

Generate each module separately. Do not try to generate the entire project at once. Start with the sensor reading module, then the data processing module, then the MQTT publisher, then the dashboard.

Step 4: Test Each Module

```
Ask Cursor: "Write unit tests for sensor_reader.py using pytest.
Mock the GPIO pins so tests run on a laptop without a Raspberry Pi."
```

Step 5: Integration and Debugging

When something breaks, paste the full error message into Cursor chat. Never try to debug without sharing the full traceback. Cursor needs the complete error context to suggest accurate fixes.

2.6 AIoT Project 1: Vibe-Coded Sensor Dashboard

This is your first Sunday project. Build this entirely using Cursor prompts.

What to build:

A Python application that simulates 3 IoT sensors (temperature, humidity, vibration), publishes data via MQTT, and displays a real-time web dashboard. Built entirely by prompting Cursor.

Prompt sequence to use:

```
Prompt 1: "Create a Python sensor simulator that generates realistic
temperature (15-45°C), humidity (30-90%), and vibration (0-10g)
readings with Gaussian noise every 2 seconds"
```

```
Prompt 2: "Add MQTT publishing using paho-mqtt. Topic structure:
sensors/{device_id}/{sensor_type}. Payload: JSON with value,
unit, timestamp, quality_flag"
```

```
Prompt 3: "Create a Flask web app that subscribes to all sensor
topics and displays live charts using Chart.js over WebSocket"
```

```
Prompt 4: "Add anomaly detection: if temperature > 40°C or
vibration > 8g, publish an alert to alerts/{device_id}"
```

🔗 **Cursor IDE Download:** cursor.com
🔗 **Tutorial: Cursor Vibe Coding for Beginners (Tech With Tim):** youtube.com/watch?v=8AWEPx5cHWQ
🔗 **ByteGrad: Full Cursor + MCP Workflow:** youtube.com/watch?v=IMseojNtxkk

Chapter 3: Claude API and Prompt Engineering

The Claude API gives you programmatic access to Anthropic's Claude models from any Python script. For an AIoT engineer, this is the most powerful tool available — it lets you add AI reasoning to any part of your system: sensor data analysis, anomaly explanation, automated reporting, and natural language interfaces to your hardware.

3.1 Getting Started with the Claude API

Installation:

```
pip install anthropic python-dotenv
```

API Key Setup:

```
# Create .env file in your project root
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

```
# In Python
from dotenv import load_dotenv
import anthropic
import os

load_dotenv()
client = anthropic.Anthropic()  # reads ANTHROPIC_API_KEY automatically
```

First API Call:

```
message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Explain MQTT in 3 sentences"}]
)
print(message.content[0].text)
```

> 📌 **Note:** Get your free API key at console.anthropic.com. The free tier gives you enough credits for weeks of learning and experimentation.

3.2 System Prompts — The Most Powerful Feature

A system prompt is a set of instructions you give Claude before the conversation begins. It defines Claude's role, constraints, and output format. For AIoT applications, system prompts are how you turn a general AI into a domain-specific expert.

AIoT Sensor Analyst System Prompt:

```
system = """You are an expert AIoT systems engineer specialising in
industrial sensor data analysis. You have deep knowledge of:
```

```
- Sensor physics (temperature, pressure, vibration, current)
- Normal operating ranges for industrial equipment
- Failure modes and their sensor signatures
- Statistical anomaly detection methods

When analysing sensor data:
1. Always state the normal range for the sensor type
2. Identify any anomalies with confidence level (High/Medium/Low)
3. Suggest the most likely physical cause
4. Recommend immediate action if safety-critical
5. Format output as JSON for programmatic processing

Be concise and technical. Use engineering units correctly."""
```

3.3 Complete AIoT Sensor Analysis Application

This is the complete code for an AI-powered sensor analyst — the project from your first Sunday session. Study every line.

```
import anthropic
import json
import csv
import os
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic()

SYSTEM_PROMPT = """You are an AIoT sensor analysis expert.
Analyse sensor readings and detect anomalies.
Always respond with valid JSON only, no other text.
Format: {"status": "normal|warning|critical",
"anomalies": [], "analysis": "", "action": ""}"""

def read_sensor_log(filepath: str, last_n: int = 10) -> list[dict]:
    """Read last N rows from sensor CSV log."""
    with open(filepath, newline="") as f:
        rows = list(csv.DictReader(f))
    return rows[-last_n:]

def analyse_sensors(readings: list[dict]) -> dict:
    """Send readings to Claude for AI analysis."""
```

```
    user_message = f"""Analyse these sensor readings:
    {json.dumps(readings, indent=2)}
    Identify anomalies. Return JSON only."""

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1000,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": user_message}]
    )
    return json.loads(response.content[0].text)

if __name__ == "__main__":
    readings = read_sensor_log("sensor_log.csv", last_n=10)
    result = analyse_sensors(readings)
    print(f"Status: {result['status'].upper()}")
    print(f"Analysis: {result['analysis']}")
    if result["action"]:
        print(f"ACTION REQUIRED: {result['action']}")
```

3.4 Prompt Engineering — Complete Reference

Prompt engineering is the skill of crafting inputs to AI models that reliably produce the output you need. It is a skill with real depth — the difference between a novice and expert prompt engineer is 10x output quality.

Principle 1: Be Specific About Format

Vague: "Analyse the sensor data." — you will get a paragraph of text.

Specific: "Analyse the sensor data. Respond ONLY with a JSON object containing: status (normal/warning/critical), anomalies (array of strings), confidence (0-1 float)."

Principle 2: Provide Context Before the Task

```
# Bad prompt
"Is this temperature reading normal?"

# Good prompt
"Context: Industrial motor bearing sensor. Normal range: 60-80°C.
Critical threshold: >100°C. Is this reading normal? Temperature: 94°C.
Respond with: {normal: bool, risk_level: low/medium/high, reason: str}"
```

Principle 3: Chain of Thought for Complex Reasoning

```
prompt = """Analyse this sensor trend. Think step by step:
Step 1: What is the baseline (first 5 readings)?
Step 2: What is the current value?
```

```
Step 3: What is the rate of change?
Step 4: Based on the rate of change, when will it reach critical threshold?
Data: {data}
After your step-by-step analysis, give a final JSON summary."""
```

Principle 4: Few-Shot Examples

```
prompt = """Classify sensor alerts. Examples:
Input: temp=45°C, normal_max=40°C, rate=+0.5°C/min
Output: {"severity": "warning", "time_to_critical": "10min"}

Input: temp=38°C, normal_max=40°C, rate=-0.1°C/min
Output: {"severity": "normal", "time_to_critical": null}

Now classify this:
Input: temp={temp}°C, normal_max={max}°C, rate={rate}°C/min"""
```

3.5 Multi-Turn Conversations — Memory for AIoT Agents

By default, each Claude API call has no memory of previous calls. For a persistent AIoT monitoring agent, you need to maintain conversation history manually.

```
class AIoTAgent:
    def __init__(self, system_prompt: str):
        self.client = anthropic.Anthropic()
        self.system = system_prompt
        self.history = []  # maintains conversation memory

    def analyse(self, sensor_data: dict) -> str:
        user_msg = f"New reading: {json.dumps(sensor_data)}"
        self.history.append({"role": "user", "content": user_msg})

        response = self.client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=500,
            system=self.system,
            messages=self.history
        )

        reply = response.content[0].text
        self.history.append({"role": "assistant", "content": reply})
        # Trim history to last 20 messages to control token costs
        if len(self.history) > 20:
            self.history = self.history[-20:]
```

```
        return reply
```

🔗 **Claude API Python Guide (Real Python):** realpython.com/claude-api-python/
🔗 **Anthropic Documentation:** docs.anthropic.com
🔗 **Prompt Engineering Guide (Anthropic):** docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview

Chapter 4: n8n AI Automation for AIoT Workflows

n8n is an open-source workflow automation platform that lets you connect applications, APIs, and AI models using a visual node-based editor. For AIoT engineers, it is the most powerful automation tool available because it supports MQTT, HTTP webhooks, databases, and AI models in the same workflow — without writing a web server from scratch.

4.1 Why n8n Over Zapier or Make

Feature	n8n (Self-hosted)	Zapier	Make
Cost	Free (self-host)	$20-50/month	$10-20/month
MQTT support	Yes (native node)	No	No
AI/LLM nodes	Yes (LangChain built-in)	Limited	Limited
Custom code	Full JavaScript/Python	Restricted	Restricted
Data privacy	Your own server	Cloud only	Cloud only
API calls/month	Unlimited	750-50k	1k-10k

> 📌 **Note:** For AIoT workflows, n8n self-hosted on a Raspberry Pi 4 or a $5/month VPS is the best choice. You get unlimited automation runs, full MQTT support, and complete data privacy — critical if your sensor data contains proprietary industrial information.

4.2 n8n Installation on Raspberry Pi

```
# Install Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo bash -
sudo apt install -y nodejs
```

```
# Install n8n globally
sudo npm install -g n8n
```

```
# Start n8n
n8n start
```

```
# Access at: http://your-pi-ip:5678
```

```
# Auto-start with systemd
sudo nano /etc/systemd/system/n8n.service
# Add: [Unit] Description=n8n
# [Service] ExecStart=/usr/bin/n8n start
# [Install] WantedBy=multi-user.target
sudo systemctl enable n8n && sudo systemctl start n8n
```

4.3 AIoT Workflow 1: Sensor Anomaly Alert Pipeline

This workflow monitors your MQTT broker for sensor data, runs it through Claude for analysis, and sends a WhatsApp or email alert if an anomaly is detected.

Nodes in this workflow:

- Node 1: MQTT Trigger — subscribes to sensors/# topic
- Node 2: Code Node — parse JSON payload, extract values
- Node 3: IF Node — check if value exceeds threshold
- Node 4: Claude AI Node — analyse the anomaly context
- Node 5: Gmail Node — send alert email with AI analysis
- Node 6: MongoDB Node — log event to database

Claude AI Node configuration:

```
Model: claude-sonnet-4-6
System Prompt: "You are an AIoT expert. Analyse this sensor anomaly
and provide a 2-sentence technical explanation and recommended action."
User Message: "Sensor: {{$json.sensor_type}}, Value: {{$json.value}},
Normal Range: {{$json.normal_range}}, Location: {{$json.location}}"
```

4.4 AIoT Workflow 2: Daily Sensor Report Generation

Every morning at 8 AM, this workflow queries your MongoDB Atlas database for yesterday's sensor data, asks Claude to generate a summary report, and emails it to your team.

Nodes:

- Node 1: Schedule Trigger — 8:00 AM daily
- Node 2: MongoDB Node — aggregate yesterday's readings
- Node 3: Code Node — format data as readable text
- Node 4: Claude AI Node — generate narrative report
- Node 5: Gmail Node — send HTML formatted report

4.5 AIoT Workflow 3: WhatsApp Chatbot for Your Pi

This is the most impressive demo project — a WhatsApp bot that lets you query your Raspberry Pi sensor data using natural language, from your phone.

Example conversation:

You: "What is the current temperature in Zone 3?"

Bot: "Zone 3 temperature is currently 27.4°C, which is within normal range (20-30°C). The trend over the last hour is stable (+0.2°C)."

How it works:

- WhatsApp message → n8n Webhook → Claude AI (interprets the question) → MongoDB query → Claude AI (formats the answer) → WhatsApp reply

🔗 **n8n Documentation:** docs.n8n.io
🔗 **n8n AI Nodes Guide:** docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-langchain/
🔗 **n8n GitHub:** github.com/n8n-io/n8n

Chapter 5: LangChain and AI Agents

LangChain is a Python framework for building applications powered by large language models. It provides abstractions for memory, tool use, chains of reasoning, and retrieval-augmented generation (RAG). For AIoT, LangChain allows you to build agents that reason over your sensor data, call your APIs, and take autonomous actions.

5.1 Core LangChain Concepts

LLMs and Chat Models

LangChain wraps different LLM providers (Claude, GPT, Gemini) behind a unified interface. This means you can switch from Claude to GPT with one line change — a significant advantage when cost or capability requirements change.

Chains

A chain is a sequence of operations. Input → LLM → Output. Or Input → Prompt Template → LLM → Output Parser → Structured Data. Chains are the building blocks of every LangChain application.

Memory

LangChain provides built-in memory modules that maintain conversation history automatically. For an AIoT monitoring agent, this means the agent remembers that "the temperature was rising last hour" without you manually tracking state.

Tools and Agents

An agent is an LLM that can decide which tools to call based on the user's request. You give it a set of tools (functions it can call), and it decides the sequence of tool calls needed to complete a task. For AIoT, tools include: read_sensor_data(), query_database(), send_alert(), control_actuator().

5.2 Installation

```
pip install langchain langchain-anthropic langchain-community
pip install chromadb  # for vector database
pip install pymongo   # for MongoDB tool
```

5.3 Your First LangChain AIoT Agent

```
from langchain_anthropic import ChatAnthropic
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.tools import tool
from langchain.prompts import ChatPromptTemplate
from pymongo import MongoClient
import json
```

```
# Connect to your MongoDB Atlas
mongo = MongoClient("your-atlas-connection-string")
```

```
db = mongo["aiot_database"]
```

```
# Define tools the agent can use
@tool
def get_latest_readings(sensor_location: str) -> str:
    """Get the latest 5 sensor readings for a location."""
    readings = list(db.sensors.find(
        {"location": sensor_location},
        sort=[("timestamp", -1)], limit=5
    ))
    return json.dumps(readings, default=str)
```

```
@tool
def get_anomaly_count(hours: int = 24) -> str:
    """Get count of anomalies in the last N hours."""
    from datetime import datetime, timedelta
    since = datetime.utcnow() - timedelta(hours=hours)
    count = db.alerts.count_documents({"timestamp": {"$gte": since}})
    return f"{count} anomalies detected in last {hours} hours"
```

```
# Create the agent
llm = ChatAnthropic(model="claude-sonnet-4-6")
tools = [get_latest_readings, get_anomaly_count]
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an AIoT monitoring expert. Use tools to answer."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])
```

```
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

```
# Natural language query
result = executor.invoke({"input": "How many anomalies in Zone 3 today?"})
print(result["output"])
```

5.4 RAG — Make AI Read Your Sensor Logs

Retrieval-Augmented Generation (RAG) allows an AI model to answer questions based on your own documents and data — not just its training data. For AIoT, this means you can ask "What caused the last 3 shutdowns of Machine 7?" and get an answer derived from your actual maintenance logs and sensor history.

How RAG works:

- Step 1: Your documents (sensor logs, maintenance records) are split into chunks
- Step 2: Each chunk is converted to a vector embedding (numerical representation)
- Step 3: Embeddings are stored in a vector database (ChromaDB, Pinecone)
- Step 4: User asks a question → question is also embedded
- Step 5: Most similar document chunks are retrieved
- Step 6: Retrieved chunks + question are sent to Claude for a grounded answer

```
from langchain_anthropic import ChatAnthropic
from langchain_community.vectorstores import Chroma
from langchain_anthropic import AnthropicEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains import RetrievalQA
```

```
# Load your maintenance logs
with open("maintenance_logs.txt") as f:
    raw_text = f.read()
```

```
# Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.create_documents([raw_text])
```

```
# Create vector store
embeddings = AnthropicEmbeddings()
vectorstore = Chroma.from_documents(chunks, embeddings)
```

```
# Build QA chain
llm = ChatAnthropic(model="claude-sonnet-4-6")
qa = RetrievalQA.from_chain_type(llm=llm, retriever=vectorstore.as_retriever())
```

```
# Ask questions about your data
answer = qa.invoke("What caused the temperature spike on July 15th?")
print(answer["result"])
```

🔗 **LangChain Documentation:** python.langchain.com/docs/introduction
🔗 **LangChain + Anthropic Guide:** python.langchain.com/docs/integrations/chat/anthropic
🔗 **LangChain Crash Course (Greg Kamradt):** youtube.com/watch?v=_v_fgW2SkkQ

Chapter 6: Edge AI and TensorFlow Lite — Complete Technical Guide

Edge AI refers to running artificial intelligence models directly on hardware devices — Raspberry Pi, ESP32, NVIDIA Jetson, smartphones — rather than sending data to a cloud server for processing. This is the technical foundation of everything you will do as an AIoT engineer.

6.1 Why Edge AI Matters for Your Career

Property	Cloud AI	Edge AI
Latency	100-500ms (network)	5-50ms (local)
Internet required	Yes, always	No (inference offline)
Data privacy	Sent to cloud	Stays on device
Cost at scale	High (per API call)	Low (hardware fixed cost)
Reliability	Depends on connectivity	Works in remote locations
Japan industry use	Limited (security)	Dominant (factory floors)

Japanese manufacturing companies specifically prefer Edge AI because their factory networks are air-gapped (not connected to the internet) for security. This is why being an Edge AI specialist makes you highly attractive in the Japanese industrial market.

6.2 The Edge AI Workflow: Train → Convert → Deploy

Phase 1: Train in PyTorch or Keras

You train your model on a powerful computer (laptop, cloud GPU, Google Colab) using full PyTorch or TensorFlow. This is your familiar Python ML workflow. Model accuracy and architecture are optimised here.

Phase 2: Convert to TFLite

TensorFlow Lite (TFLite) is a compressed, optimised format for running models on embedded devices. The conversion process applies quantisation (reducing weight precision from 32-bit float to 8-bit integer) which reduces model size by 4x and inference speed by 2-3x with minimal accuracy loss.

Phase 3: Deploy on Pi

The TFLite model (.tflite file) is copied to the Raspberry Pi. The TFLite runtime interprets the model and runs inference. No internet connection required. No GPU required for most models.

6.3 PyTorch → ONNX → TFLite Conversion Pipeline

```
import torch
import torch.nn as nn
import tensorflow as tf
import numpy as np
```

```
# Step 1: Define your PyTorch model (example: simple classifier)
class SensorClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(10, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 4)  # 4 classes: normal, warning, fault, critical
        )
    def forward(self, x): return self.network(x)
```

```
model = SensorClassifier()
# model.load_state_dict(torch.load("trained_model.pth"))
model.eval()
```

```
# Step 2: Export to ONNX
dummy_input = torch.randn(1, 10)
torch.onnx.export(model, dummy_input, "model.onnx",
    export_params=True, opset_version=11,
    input_names=["input"], output_names=["output"])
```

```
# Step 3: Convert ONNX → TFLite
import onnx
from onnx_tf.backend import prepare
```

```
onnx_model = onnx.load("model.onnx")
tf_rep = prepare(onnx_model)
tf_rep.export_graph("model_tf")
```

```
# Convert TF SavedModel → TFLite
converter = tf.lite.TFLiteConverter.from_saved_model("model_tf")
converter.optimizations = [tf.lite.Optimize.DEFAULT]  # INT8 quantization
tflite_model = converter.convert()
```

```
with open("model.tflite", "wb") as f:
    f.write(tflite_model)
```

```
print(f"Model size: {len(tflite_model)/1024:.1f} KB")
```

6.4 Running TFLite Inference on Raspberry Pi

```
import tflite_runtime.interpreter as tflite
import numpy as np
```

```
import time

def load_model(model_path: str):
    interpreter = tflite.Interpreter(model_path=model_path)
    interpreter.allocate_tensors()
    return interpreter

def run_inference(interpreter, input_data: np.ndarray) -> np.ndarray:
    input_details = interpreter.get_input_details()
    output_details = interpreter.get_output_details()

    interpreter.set_tensor(input_details[0]["index"], input_data)

    start = time.perf_counter()
    interpreter.invoke()
    latency_ms = (time.perf_counter() - start) * 1000

    output = interpreter.get_tensor(output_details[0]["index"])
    return output, latency_ms

# Load model once at startup
interpreter = load_model("model.tflite")

# Run inference on new sensor reading
sensor_data = np.array([[25.3, 62.1, 1.2, ...]], dtype=np.float32)
prediction, latency = run_inference(interpreter, sensor_data)
class_idx = np.argmax(prediction)
classes = ["normal", "warning", "fault", "critical"]
print(f"Status: {classes[class_idx]} | Latency: {latency:.1f}ms")
```

6.5 Benchmarking Your Edge AI System

Always benchmark your TFLite model before deploying to production. These are the metrics that Japanese industrial clients will ask about.

```
def benchmark(interpreter, n_runs: int = 100) -> dict:
    """Run N inferences and report performance statistics."""
    latencies = []
    input_details = interpreter.get_input_details()
    dummy = np.random.rand(1, 10).astype(np.float32)

    for _ in range(n_runs):
```

```
        interpreter.set_tensor(input_details[0]["index"], dummy)
        t0 = time.perf_counter()
        interpreter.invoke()
        latencies.append((time.perf_counter() - t0) * 1000)
```

```
    return {
        "mean_ms": np.mean(latencies),
        "p95_ms": np.percentile(latencies, 95),
        "p99_ms": np.percentile(latencies, 99),
        "fps": 1000 / np.mean(latencies),
    }
```

🔗 **TFLite Official Docs:** tensorflow.org/lite
🔗 **Edge AI on Raspberry Pi (The AI Guy YouTube):** youtube.com/c/nickleeai
🔗 **TFLite Model Optimization Guide:** tensorflow.org/lite/performance/model_optimization

Chapter 7: Raspberry Pi and AIoT Full Stack Reference

The Raspberry Pi is the most important hardware platform for your AIoT career. It runs a full Linux OS, supports Python natively, has GPIO pins for sensor interfacing, camera support for computer vision, and USB/UART/I2C/SPI for peripheral connectivity. Every AIoT project you build should target the Raspberry Pi first.

7.1 Recommended Raspberry Pi Setup for AIoT

Component	Recommendation	Why
Hardware	Raspberry Pi 4 (4GB RAM)	Enough RAM for TFLite + OpenCV + Python
OS	Raspberry Pi OS Lite (64-bit)	Lightweight, no desktop, more RAM for apps
Storage	32GB microSD (Class 10)	Fast enough for logging, swap space
Camera	Pi Camera Module 3	Native CSI interface, best TFLite performance
Power	5V 3A USB-C official supply	Prevents undervoltage throttling
Cooling	Aluminium heatsink case	Prevents thermal throttling during inference

7.2 Essential Software Stack (install in this order)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv git
sudo apt install -y libatlas-base-dev  # required for NumPy
sudo apt install -y libopencv-dev python3-opencv
sudo apt install -y mosquitto mosquitto-clients  # MQTT broker
sudo apt install -y nodejs npm  # for Node-RED
```

```
# Install Python packages
pip3 install tflite-runtime numpy pandas
pip3 install paho-mqtt pymongo flask
pip3 install picamera2  # Raspberry Pi Camera
```

7.3 GPIO Sensor Integration Reference

Sensor	Protocol	GPIO Pins	Python Library	Use Case
DHT22	Single-wire	Any GPIO	adafruit-circuitpython-dht	Temp + Humidity
BME280	I2C	GPIO2(SDA) +GPIO3(SCL)	adafruit-circuitpython-	Temp+Humidity+Pressure

			bme280	
ADXL345	I2C or SPI	GPIO2/3 or GPIO10/11	adafruit-circuitpython-adxl34x	Vibration/ Acceleration
HC-SR04	Digital	Any 2 GPIO	RPi.GPIO	Distance measurement
ACS712	Analog (via MCP3008)	SPI	gpiozero	Current sensing
MQ-135	Analog (via MCP3008)	SPI	spidev	Air quality

7.4 Complete AIoT Application Architecture

This is the architecture every production AIoT system uses. Build your NIAT project to match this pattern.

Layer 1 — Sensor Layer (ESP32 or Pi GPIO)

```
while True:
    reading = sensor.read()
    client.publish(f"sensors/{DEVICE_ID}/{SENSOR_TYPE}", json.dumps({
        "value": reading.value,
        "unit": reading.unit,
        "timestamp": datetime.utcnow().isoformat(),
        "quality": reading.quality_flag
    }))
    time.sleep(SAMPLE_INTERVAL)
```

Layer 2 — Edge Processing Layer (Raspberry Pi)

```
def on_message(client, userdata, msg):
    data = json.loads(msg.payload)
    # Run TFLite inference
    features = preprocess(data)
    prediction, latency = run_inference(interpreter, features)
    status = classify(prediction)
    # If anomaly detected, trigger alert
    if status in ["warning", "critical"]:
        alert_topic = f"alerts/{msg.topic}"
        client.publish(alert_topic, json.dumps({
            "status": status, "data": data, "latency_ms": latency
        }))
```

Layer 3 — Data Layer (MongoDB Atlas)

```
def log_to_database(data: dict, prediction: str):
    db.sensor_readings.insert_one({
```

```
        **data,
        "ai_classification": prediction,
        "logged_at": datetime.utcnow()
    })
```

Layer 4 — Presentation Layer (Node-RED Dashboard)

Node-RED subscribes to all MQTT topics and visualises the data in real-time gauges, charts, and alert tables. No additional coding required — configure visually.

🔗 **Raspberry Pi Official Documentation:** raspberrypi.com/documentation
🔗 **Jeff Geerling Pi Setup Guide:** youtube.com/watch?v=BpJCAafw2qE
🔗 **The AI Guy — TFLite on Pi Tutorial:** youtube.com/watch?v=aimSGOAUI8Y

Chapter 8: MQTT and IoT Protocols — Deep Dive

MQTT (Message Queuing Telemetry Transport) is the backbone of modern IoT communication. Understanding it deeply — not just the basics — separates junior IoT developers from senior AIoT engineers. Japanese industrial companies run their factory floors on MQTT and OPC-UA, so mastering these protocols is directly tied to your Japan career goal.

8.1 MQTT Core Concepts

Concept	Explanation	Example
Broker	Central server that routes messages	Mosquitto on your Pi
Publisher	Device that sends messages	ESP32 with DHT22 sensor
Subscriber	Device that receives messages	Pi running TFLite analysis
Topic	Message address (like a URL)	sensors/factory1/zone3/temperature
Payload	Message content	{"value": 27.4, "unit": "C"}
QoS 0	Fire and forget — no guarantee	High-frequency sensor readings
QoS 1	At least once — may duplicate	Alert messages
QoS 2	Exactly once — guaranteed	Control commands to actuators
Retain flag	Broker saves last message	Device status (online/offline)
Will message	Sent if device disconnects	{"status": "offline"}

8.2 Production-Grade MQTT Client (Python)

```
import paho.mqtt.client as mqtt
import json, time, logging
from dataclasses import dataclass
```

```
@dataclass
class MQTTConfig:
    host: str = "localhost"
    port: int = 1883
    keepalive: int = 60
    client_id: str = "aiot_edge_001"
    username: str = None
    password: str = None
```

```
class RobustMQTTClient:
    """Production MQTT client with auto-reconnect."""
```

```
    def __init__(self, config: MQTTConfig):
```

```
        self.config = config
        self.client = mqtt.Client(config.client_id)
        self.client.on_connect = self._on_connect
        self.client.on_disconnect = self._on_disconnect
        self.client.on_message = self._on_message
        if config.username:
            self.client.username_pw_set(config.username, config.password)
        # Will message — sent if connection drops unexpectedly
        self.client.will_set(f"status/{config.client_id}",
            json.dumps({"status": "offline", "reason": "unexpected"}),
            qos=1, retain=True)
```

```
    def _on_connect(self, client, userdata, flags, rc):
        if rc == 0:
            logging.info("MQTT connected")
            # Publish online status
            client.publish(f"status/{self.config.client_id}",
                json.dumps({"status": "online"}), qos=1, retain=True)
            # Re-subscribe after reconnection
            client.subscribe("sensors/#", qos=1)
        else:
            logging.error(f"MQTT connect failed: rc={rc}")
```

```
    def _on_disconnect(self, client, userdata, rc):
        if rc != 0:
            logging.warning("Unexpected disconnect, reconnecting...")
            # paho auto-reconnects if loop_start() is used
```

```
    def _on_message(self, client, userdata, msg):
        try:
            payload = json.loads(msg.payload.decode())
            self.process_message(msg.topic, payload)
        except json.JSONDecodeError as e:
            logging.error(f"Invalid JSON on {msg.topic}: {e}")
```

```
    def process_message(self, topic: str, payload: dict):
        """Override this method in subclasses."""
        print(f"[{topic}] {payload}")
```

```
    def start(self):
        self.client.connect(self.config.host, self.config.port,
self.config.keepalive)
        self.client.loop_start()  # non-blocking background thread
```

8.3 MQTT Topic Design for Production AIoT

Good topic design is as important as good database schema design. Use this pattern for all your projects:

```
# Pattern: {org}/{facility}/{zone}/{device_id}/{data_type}
# Examples:
acme/factory1/zone3/motor_001/temperature
acme/factory1/zone3/motor_001/vibration
acme/factory1/zone3/motor_001/status
acme/factory1/alerts/motor_001
acme/factory1/commands/motor_001/shutdown
```

📌 **Note:** Never put sensitive data in topic names. Topics are visible to all subscribers. Put sensitive data only in the encrypted payload.

8.4 OPC-UA — Japan's Factory Protocol

OPC-UA (Open Platform Communications Unified Architecture) is the standard industrial protocol in Japanese factories. Panasonic, Toyota, Omron, and Mitsubishi factories all use OPC-UA for machine-to-machine communication. Learning it sets you apart from candidates who only know MQTT.

```
pip install asyncua
```

```
from asyncua import Client
import asyncio
```

```
async def read_plc_data(endpoint: str, node_id: str):
    """Read a value from an OPC-UA server (like a Mitsubishi PLC)."""
    async with Client(url=endpoint) as client:
        node = client.get_node(node_id)
        value = await node.read_value()
        return value
```

```
# Example: read spindle speed from CNC machine
speed = asyncio.run(read_plc_data(
    "opc.tcp://cnc-machine-001:4840",
    "ns=2;i=1001"
))
print(f"Spindle speed: {speed} RPM")
```

🔗 **MQTT Essentials (HiveMQ):** hivemq.com/mqtt-essentials
🔗 **OPC-UA Foundation Free Training:** opcfoundation.org/developer-tools/training
🔗 **MQTT Tutorial (Steve's Internet Guide):** youtube.com/watch?v=EIxdz-2rhLs

Chapter 9: Autonomous Vehicles — Your Future Domain

Autonomous vehicle (AV) engineering is the natural extension of your AIoT skillset. Many AIoT engineers transition to AV roles because the core technologies overlap significantly: computer vision, sensor fusion, edge computing, real-time embedded systems. This chapter gives you the full picture of what AV engineering involves and how to build toward it from your current position.

9.1 AV System Architecture

A self-driving vehicle is essentially the world's most complex AIoT system. It has:

- Sensors: cameras, LiDAR, radar, ultrasonic, GPS, IMU
- Edge compute: NVIDIA DRIVE, Qualcomm Snapdragon Ride, or Orin SoC
- Perception: object detection, semantic segmentation, depth estimation
- Localisation: SLAM, HD maps, GPS-IMU fusion
- Prediction: trajectory prediction for pedestrians and vehicles
- Planning: route planning, motion planning, behaviour planning
- Control: lateral (steering) and longitudinal (throttle/brake) control
- Communication: V2X (vehicle-to-everything), 5G, MQTT between subsystems

9.2 How Your Current Skills Map to AV

Your Skill	AV Application	Readiness
Python + PyTorch	Training perception models (YOLO, SegFormer)	80% ready now
OpenCV	Camera image processing, lane detection	75% ready (Month 3)
TFLite / ONNX	Deploying perception models on NVIDIA Jetson	70% ready (Month 4)
C++ DSA	Foundation for ROS2 + real-time code	50% ready, needs upgrade
Linux CLI	Ubuntu on AV compute hardware	85% ready now
MQTT	Inter-module communication in AV stacks	70% ready (Month 3)
EEE diploma	Sensor integration (LiDAR, radar, IMU wiring)	80% ready now
ROS2	AV robotics middleware — not yet learned	0% — add Year 3
SLAM	Vehicle localisation — not yet learned	0% — add Year 3
Sensor fusion	Camera + LiDAR + IMU fusion — not yet	0% — add Year 3-4

9.3 CARLA Simulator — Start Before You Have Hardware

CARLA (Car Learning to Act) is a free, open-source AV simulator built on Unreal Engine. It provides synthetic camera, LiDAR, radar, GPS, and IMU data — so you can practise AV algorithms without a real car or real sensors. This is how most AV engineers learn.

```
pip install carla
```

```
import carla
import numpy as np
```

```
# Connect to CARLA server
client = carla.Client("localhost", 2000)
client.set_timeout(10.0)
world = client.get_world()
```

```
# Spawn a vehicle
blueprint_library = world.get_blueprint_library()
vehicle_bp = blueprint_library.find("vehicle.tesla.model3")
spawn_point = world.get_map().get_spawn_points()[0]
vehicle = world.spawn_actor(vehicle_bp, spawn_point)
```

```
# Attach a camera sensor
camera_bp = blueprint_library.find("sensor.camera.rgb")
camera_bp.set_attribute("image_size_x", "640")
camera_bp.set_attribute("image_size_y", "480")
camera = world.spawn_actor(camera_bp,
    carla.Transform(carla.Location(x=2.5, z=0.7)),
    attach_to=vehicle)
```

```
# Process camera frames
def process_image(image):
    array = np.frombuffer(image.raw_data, dtype=np.uint8)
    array = array.reshape((image.height, image.width, 4))
    rgb = array[:, :, :3]
    # Run your object detection model here
    return rgb
```

```
camera.listen(process_image)
```

9.4 ROS2 — The Backbone of AV Development

ROS2 (Robot Operating System 2) is the middleware that connects every component in an AV system. Sensors, perception models, planners, and controllers all communicate via ROS2 topics and services. Every AV company uses it. Start learning in B.Tech Year 3.

🔗 **CARLA Simulator:** carla.org
🔗 **ROS2 Tutorials (Articulated Robotics):** youtube.com/watch?v=idQb2pB-h2Q
🔗 **AV Engineer Roadmap (Towards AI):** towardsai.net/p/machine-learning/autonomous-driving-engineer-roadmap

Chapter 10: Japan AIoT Industry — Career and Market Guide

Japan is the world's third-largest economy and the most automation-intensive manufacturing nation on earth. The country has a chronic labour shortage and is deploying AI and robotics faster than any other G7 nation. For an Indian AIoT engineer with Japanese language skills, the opportunity is extraordinary.

10.1 Japan's Industrial AI Market

Sector	Companies	AIoT Use Case	Hiring for Indians
Manufacturing	Panasonic, Omron, Keyence	Quality inspection, predictive maintenance	High (with N2+)
Automotive	Toyota, DENSO, Aisin	AV, robotics, connected vehicles	High (Engineering/Specialist visa)
Robotics	Fanuc, Yaskawa, Kawasaki	Industrial robot AI, SLAM	Medium
Smart Buildings	Hitachi, Mitsubishi Electric	Energy optimisation, IoT sensors	High
Agriculture	Kubota, Yanmar	AgriTech IoT, drone AI	Medium
Startups	Tier IV, LeapMind, Idein	Autonomous vehicles, Edge AI	High (English OK)

10.2 Visa Route — Engineer/Specialist in Humanities

The Engineer/Specialist in Humanities visa is the standard path for Indian software and engineering graduates working in Japan. Requirements:

- Bachelor's degree (4-year, your B.Tech qualifies)
- Job offer from a Japanese company in your field of study
- Company must be registered and have stable financials
- No Japanese language requirement at visa stage — but N3+ dramatically improves hiring odds

10.3 Salary Expectations

Level	Japanese Salary	Indian Rupee Equivalent	Savings Potential
Entry (0-2 years)	¥300-380 万/year	₹15-19 lakh/year	₹6-9 lakh/year (40-50%)
Mid (2-4 years)	¥380-500 万/year	₹19-25 lakh/year	₹10-13 lakh/year
Senior (4+ years)	¥500-700 万/year	₹25-35 lakh/year	₹14-18 lakh/year
Startup equity	Variable + options	Potentially high	Depends on exit

10.4 Japanese Work Culture — What to Know Before Day 1

Nemawashi (根回し) — Consensus Building

In Japanese companies, decisions are made by building consensus before any formal meeting. Before proposing a new idea, speak to each stakeholder individually and get their informal agreement. Walking into a meeting with a surprise proposal is considered disrespectful.

Kaizen (改善) — Continuous Improvement

Every Japanese manufacturing company runs on kaizen principles. You are expected to proactively identify small improvements in your workflow and suggest them. This is not optional — it is part of the job. Write down 1 kaizen suggestion per week as a habit.

Horenso (報連相) — Report, Contact, Consult

Always report your progress to your supervisor (報告 - houkoku). Contact immediately when something unexpected happens (連絡 - renraku). Consult your supervisor before making any independent decision (相談 - soudan). Following this framework makes you a trusted team member quickly.

Japanese Email Etiquette

Business emails in Japanese follow a strict format: greeting formula → identity → context → main content → closing formula. Study this format before starting work. Your N2 Japanese will handle conversation — email formality is a separate skill.

10.5 Job Boards for Indian Engineers in Japan

- 🔗 **Daijob (English-friendly Japan jobs):** daijob.com
- 🔗 **GaijinPot Jobs:** jobs.gaijinpot.com
- 🔗 **Wantedly (startup-focused):** wantedly.com
- 🔗 **LinkedIn Japan:** linkedin.com/jobs
- 🔗 **Tier IV Careers (AV startup):** tier4.jp/careers
- 🔗 **LeapMind Careers (Edge AI startup):** leapmind.io/careers
- 🔗 **Idein Careers (Edge AI):** idein.jp/en/careers

Chapter 11: 12 Sunday AI Projects — 4 Months of Hands-On Learning

These 12 projects are your Sunday schedule for the next 4 months. Each one is designed to be completable in 4 hours, builds on your weekday skills, and produces a GitHub-ready portfolio artifact. Complete them in order — each one uses skills from the previous.

Month 1 Projects — Vibe Coding + Claude API

Project 1 (Week 1): Vibe-Coded Sensor Simulator + AI Analyst

Description: Use Cursor to build a fake IoT sensor data generator, then use the Claude API to analyse the data for anomalies.

Tools: Cursor IDE, Claude API, Python, CSV, matplotlib

GitHub repo name: aiot-sensor-analyst

- Sensor simulator generating temperature, humidity, vibration data
- Claude API integration analysing last 10 readings
- Alert generation when anomaly detected
- matplotlib live plot of sensor data

Project 2 (Week 2): AI-Powered Resume and LinkedIn Optimiser

Description: A Python tool that takes your resume PDF, sends it to Claude API, and gets specific suggestions to improve it for AIoT internship applications.

Tools: Cursor, Claude API, PyPDF2, Python

GitHub repo name: ai-resume-optimizer

- Read resume PDF with PyPDF2
- Send to Claude with system prompt: "You are a tech recruiter specialising in AIoT. Rate and improve this resume."
- Generate improved bullet points for each project
- Output formatted suggestions to terminal

Project 3 (Week 3): Japanese Vocabulary AI Tutor

Description: A CLI flashcard app that uses Claude to explain Japanese words in the context of AIoT engineering — technical vocabulary practice.

Tools: Cursor, Claude API, Python, Anki export format

GitHub repo name: japanese-aiot-tutor

- Load N5/N4 vocabulary list from CSV
- For each word, ask Claude: "Explain {word} with 3 example sentences from IoT engineering context"
- User rates recall (1-5), app prioritises low-score words
- Export failed words to Anki-compatible CSV

Project 4 (Week 4): AI Code Reviewer for Your DSA Solutions

Description: Paste any C++ DSA solution, and the tool uses Claude to review it for time complexity, edge cases, and style improvements.

Tools: Cursor, Claude API, Python

GitHub repo name: dsa-ai-reviewer

- Read C++ file from command line argument
- Send to Claude with prompt: "You are a senior software engineer. Review this DSA solution. Rate: correctness, time complexity, edge case handling, code style. Provide specific improvements."
- Output formatted code review to terminal
- Log all reviews to SQLite database for progress tracking

Month 2 Projects — n8n + Automation

Project 5 (Week 5): n8n Sensor Alert Pipeline

Description: Full n8n workflow that monitors a simulated MQTT feed and sends email alerts when anomalies are detected.

Tools: n8n (self-hosted), Mosquitto MQTT, Gmail SMTP

Documentation: Screenshot workflow + record a 1-minute demo video

Project 6 (Week 6): Daily AIoT Study Report Generator

Description: n8n workflow that runs every evening, reads your GitHub commits from the day, and uses Claude to generate a study progress report emailed to yourself.

Tools: n8n, GitHub API, Claude AI node, Gmail

Project 7 (Week 7): WhatsApp Sensor Query Bot

Description: WhatsApp chatbot (via Twilio) that lets you query your sensor simulator data in natural language from your phone.

Tools: n8n, Twilio WhatsApp API, Claude AI node, Python MQTT publisher

Project 8 (Week 8): AI Internship Application Tracker

Description: A Python + Claude tool that reads your internship application spreadsheet, analyses response rates, suggests improvements to cover letters, and predicts which applications are most likely to succeed.

Tools: Cursor, Claude API, pandas, openpyxl

Month 3 Projects — LangChain + RAG

Project 9 (Week 9): LangChain AIoT Monitoring Agent

Description: A LangChain agent with tools to read sensor data from MongoDB, detect trends, and answer natural language queries about your AIoT system.

Tools: LangChain, Claude, MongoDB Atlas, Python

GitHub repo name: aiot-langchain-agent

Project 10 (Week 10): RAG System for Technical Documentation

Description: Load the Raspberry Pi documentation, TFLite docs, and MQTT specification into a ChromaDB vector store. Build a Q&A system that answers technical questions with citations.

Tools: LangChain, ChromaDB, Claude, BeautifulSoup (for scraping docs)

GitHub repo name: aiot-docs-rag

Month 4 Projects — Full AIoT AI System

Project 11 (Week 11): End-to-End AI-Powered Predictive Maintenance Demo

Description: The flagship project. Combines everything: fake sensor data → MQTT → Pi TFLite inference → LangChain agent analysis → n8n alert workflow → RAG maintenance manual lookup → email report.

Tools: All previous tools combined

GitHub repo name: aiot-predictive-maintenance

- Sensor simulator generates vibration + temperature data with injected fault patterns
- TFLite model classifies: normal / early_warning / fault
- LangChain agent retrieves historical fault data from MongoDB
- RAG system looks up repair procedure from maintenance manual
- n8n sends complete alert email with AI-generated diagnosis and recommended action

Project 12 (Week 12): Personal AI Assistant for AIoT Study

Description: A Claude-powered CLI assistant that knows your entire study plan, skill progression, and can answer questions like "What should I study next?", "Explain Dijkstra's algorithm in the context of IoT mesh networking", "Create a practice quiz on MQTT topics".

Tools: Cursor, Claude API, LangChain memory, your schedule ICS file as context

GitHub repo name: aiot-study-assistant

Chapter 12: Glossary and Complete Resource Index

12.1 Technical Glossary — AIoT and AI Automation

Term	Definition
AIoT	Artificial Intelligence of Things — integration of AI/ML with IoT devices
Edge AI	Running ML inference directly on embedded hardware without cloud
TFLite	TensorFlow Lite — optimised ML runtime for embedded devices
ONNX	Open Neural Network Exchange — hardware-agnostic model format
Quantisation	Reducing model weight precision (FP32→INT8) for faster edge inference
MQTT	Message Queuing Telemetry Transport — lightweight IoT pub/sub protocol
OPC-UA	Open Platform Communications Unified Architecture — industrial IoT standard
ROS2	Robot Operating System 2 — middleware for robotic and AV systems
SLAM	Simultaneous Localisation and Mapping — core AV navigation algorithm
LiDAR	Light Detection and Ranging — 3D point cloud sensor used in AV
Vibe Coding	Writing software using natural language prompts to an AI code editor
LLM	Large Language Model — AI model trained on text (Claude, GPT, Gemini)
RAG	Retrieval-Augmented Generation — AI that answers from your own documents
Vector DB	Database optimised for similarity search on high-dimensional embeddings
LangChain	Python framework for building LLM-powered agents and chains
n8n	Open-source workflow automation tool with AI and MQTT nodes
MCP	Model Context Protocol — standard for connecting AI models to tools
Cursor	AI-first code editor — VS Code with integrated LLM assistance
Inference	Running a trained ML model on new input data to

	get predictions
Latency	Time from input to output — critical metric for real-time edge AI
GPIO	General Purpose Input/Output — Pi hardware pins for sensor connection
I2C	Two-wire serial protocol for low-speed sensor communication
SPI	Serial Peripheral Interface — faster than I2C, used for displays and ADCs
Kaizen	Japanese: continuous improvement — core principle of manufacturing culture
JLPT	Japanese Language Proficiency Test — N5 (easiest) to N1 (hardest)

12.2 Complete YouTube Resource Index

Topic	Channel	Best Video/Playlist
Cursor Vibe Coding	Tech With Tim	youtube.com/watch?v=8AWEPx5cHWQ
Claude API Python	Various	docs.anthropic.com/en/docs/quickstart
n8n Automation	n8n Official	youtube.com/@n8n-io
LangChain Basics	Greg Kamradt	youtube.com/watch?v=_v_fgW2SkkQ
DSA C++ (A2Z)	Striver/takeUforward	youtube.com/playlist?list=PLgUwDviBIf0oF6QL8m22w1hIDC1vJ_BHz
PyTorch Deep Learning	Daniel Bourke	youtube.com/watch?v=Z_ikDlimN6A
TFLite on Raspberry Pi	The AI Guy	youtube.com/watch?v=aimSGOAUI8Y
Raspberry Pi Setup	Jeff Geerling	youtube.com/watch?v=BpJCAafw2qE
MQTT Tutorial	Steve's Internet Guide	youtube.com/watch?v=EIxdz-2rhLs
Node-RED Dashboard	Steve Cope	youtube.com/watch?v=3AR432bguOY
OpenCV Python	Tech With Tim	youtube.com/watch?v=oXlwWbU8l2o
Linux Bash Scripting	NetworkChuck	youtube.com/watch?v=tK9Oc6AEnR4
MySQL Full Course	Bro Code	youtube.com/watch?

		v=5OdVJbNCSso
MERN Stack	Traversy Media	youtube.com/watch?v=ktjafK4SgWM
Japanese N5→N4	Japanese from Zero!	youtube.com/playlist?list=PLd5-Wp_4tLqYZxS5j3g6kbeOfVXlTkr3N
JLPT N4 Grammar	nihongonomori	youtube.com/watch?v=IXpyNkF3Xh8
ROS2 Beginners	Articulated Robotics	youtube.com/watch?v=idQb2pB-h2Q
CARLA Simulator	Various	carla.org/blog
AV Engineering	Comma.ai	youtube.com/@commaai
Edge AI Theory	MIT HAN Lab	youtube.com/watch?v=FHDxaFI_3_k

12.3 Essential Websites and Documentation

🔗 **Anthropic Claude API Docs:** docs.anthropic.com
🔗 **Prompt Engineering Guide (Anthropic):** docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview
🔗 **LangChain Python Docs:** python.langchain.com
🔗 **TensorFlow Lite Guide:** tensorflow.org/lite
🔗 **n8n Documentation:** docs.n8n.io
🔗 **Raspberry Pi Official Docs:** raspberrypi.com/documentation
🔗 **Striver A2Z DSA Sheet:** takeuforward.org/strivers-a2z-dsa-course
🔗 **JLPT Official Sample Questions:** jlpt.jp/e/samples/forlearners.html
🔗 **NHK Web Easy (Japanese reading):** nhk.or.jp/news/easy
🔗 **CARLA AV Simulator:** carla.org
🔗 **LearnPyTorch.io:** learnpytorch.io
🔗 **HiveMQ MQTT Essentials:** hivemq.com/mqtt-essentials
🔗 **OPC-UA Foundation:** opcfoundation.org
🔗 **Daijob Japan Jobs:** daijob.com
🔗 **GaijinPot Japan Jobs:** jobs.gaijinpot.com

12.4 Your 4-Month Study Accountability Checklist

Month 1 Milestones:

- Cursor IDE installed and used daily for weekday coding
- First Claude API call made successfully
- Sensor simulator project on GitHub with demo screenshot
- AI code reviewer project on GitHub
- 50 Striver DSA problems solved and pushed to GitHub
- Hiragana and Katakana memorised (test: write without chart)

Month 2 Milestones:

- n8n self-hosted on Raspberry Pi or VPS
- Sensor alert pipeline workflow built and tested
- 5 n8n workflows documented with screenshots
- First internship applications sent (minimum 10)
- PyTorch CIFAR-10 model trained and saved
- JLPT N4 study begun — first 200 vocabulary words

Month 3 Milestones:

- LangChain AIoT agent responding to natural language sensor queries
- RAG system loaded with technical documentation
- NIAT IoT project prototype working
- TFLite model running on Raspberry Pi with benchmark data
- LinkedIn post about AIoT project published — minimum 50 reactions
- Active internship interview or 20+ applications sent

Month 4 Milestones:

- Full predictive maintenance demo system running end-to-end
- GitHub portfolio: 8+ repos with READMEs and demo videos
- JLPT N4 mock test score: 65%+ (test-ready)
- Resume finalised: 1 page, 4 projects, all skills listed
- Internship secured OR 30+ applications sent with follow-ups
- Personal AI study assistant project completed

This document will be updated as you progress. Every chapter contains enough depth to take you from zero to interview-ready for your specific career path.

EEE Diploma → B.Tech CS-IT → Remote Internship → Japan AIoT Engineer → Vizag Startup

2025 – 2033 | Your decade. Your roadmap.

Appendix A: DSA Cheat Sheet — Patterns and Time Complexities

This is your quick-reference for every Data Structures and Algorithms pattern you will encounter in internship interviews. Study this alongside Striver's A2Z sheet. For every problem you solve, map it to one of these patterns.

A.1 Sliding Window Pattern

Use when: Finding max/min/sum of a subarray of fixed or variable size. Typical problems: Maximum sum subarray of size K, Longest substring without repeating characters, Minimum window substring.

```
// Template: Fixed window
int left = 0, windowSum = 0, maxSum = 0;
for (int right = 0; right < nums.size(); right++) {
    windowSum += nums[right];
    if (right - left + 1 == k) {
        maxSum = max(maxSum, windowSum);
        windowSum -= nums[left++];
    }
}
```

```
// Template: Variable window
int left = 0; unordered_map<char, int> freq;
int maxLen = 0;
for (int right = 0; right < s.size(); right++) {
    freq[s[right]]++;
    while (freq.size() > k) { // condition to shrink
        freq[s[left]]--;
        if (freq[s[left]] == 0) freq.erase(s[left]);
        left++;
    }
    maxLen = max(maxLen, right - left + 1);
}
```

A.2 Two Pointer Pattern

Use when: Sorted array, finding pairs that sum to target, removing duplicates, palindrome check. Time: O(n). Space: O(1).

```
// Template: Two sum in sorted array
int left = 0, right = nums.size() - 1;
while (left < right) {
    int sum = nums[left] + nums[right];
    if (sum == target) return {left, right};
    else if (sum < target) left++;
    else right--;
}
```

A.3 Binary Search Pattern

Use when: Sorted array, search space can be halved. Also use for "find minimum value that satisfies a condition" problems — binary search on the answer.

```
// Template: Standard binary search
int left = 0, right = nums.size() - 1;
while (left <= right) {
    int mid = left + (right - left) / 2; // prevents overflow
    if (nums[mid] == target) return mid;
    else if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
}
```

```
// Template: Binary search on answer
int left = min_possible, right = max_possible;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (canAchieve(mid)) right = mid; // search left half
    else left = mid + 1;              // search right half
}
```

A.4 BFS Pattern — Level Order Traversal

Use for: Shortest path in unweighted graph, level-by-level tree processing, minimum steps problems.

```
// Graph BFS template
vector<int> visited(n, 0);
queue<int> q;
q.push(start); visited[start] = 1;
while (!q.empty()) {
    int node = q.front(); q.pop();
    for (int neighbour : adj[node]) {
        if (!visited[neighbour]) {
            visited[neighbour] = 1;
            q.push(neighbour);
        }
    }
}
```

A.5 DFS Pattern — Graph and Tree

Use for: Cycle detection, topological sort, connected components, path finding, tree problems.

```
// Recursive DFS template
void dfs(int node, vector<vector<int>>& adj, vector<int>& vis) {
    vis[node] = 1;
    for (int neighbour : adj[node]) {
```

```
        if (!vis[neighbour]) dfs(neighbour, adj, vis);
    }
}
```

A.6 Dynamic Programming — 5 Templates

Template 1: 1D DP (Fibonacci-type)

```
// dp[i] = answer for problem of size i
vector<int> dp(n+1, 0);
dp[0] = base_case_0; dp[1] = base_case_1;
for (int i = 2; i <= n; i++) {
    dp[i] = dp[i-1] + dp[i-2]; // recurrence
}
```

Template 2: 2D DP (LCS, Edit Distance type)

```
// dp[i][j] = answer for strings of length i and j
vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1[i-1] == s2[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
        else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    }
}
```

A.7 Time Complexity Reference

Algorithm/ Structure	Best	Average	Worst	Space
Array access	O(1)	O(1)	O(1)	O(n)
Binary search	O(1)	O(log n)	O(log n)	O(1)
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)
Hash table lookup	O(1)	O(1)	O(n)	O(n)
BFS / DFS	O(V+E)	O(V+E)	O(V+E)	O(V)
Dijkstra (heap)	O((V+E)log V)	O((V+E)log V)	O((V+E)log V)	O(V)
DP (2D)	O(mn)	O(mn)	O(mn)	O(mn)
Heap push/pop	O(1)	O(log n)	O(log n)	O(n)

Appendix B: Python Quick Reference for AIoT

B.1 Data Classes — Use These for Sensor Data

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional
```

```
@dataclass
class SensorReading:
    sensor_id: str
    sensor_type: str  # "temperature" | "humidity" | "vibration"
    value: float
    unit: str
    timestamp: datetime = field(default_factory=datetime.utcnow)
    quality: str = "good"  # "good" | "uncertain" | "bad"
    location: Optional[str] = None
```

```
    def is_valid(self) -> bool:
        RANGES = {"temperature": (-40, 125), "humidity": (0, 100)}
        if self.sensor_type in RANGES:
            lo, hi = RANGES[self.sensor_type]
            return lo <= self.value <= hi
        return True
```

```
    def to_dict(self) -> dict:
        return {
            "sensor_id": self.sensor_id,
            "type": self.sensor_type,
            "value": self.value,
            "unit": self.unit,
            "timestamp": self.timestamp.isoformat(),
            "quality": self.quality,
        }
```

B.2 Async Patterns for IoT

Always use async for I/O-heavy AIoT applications — reading multiple sensors simultaneously, handling MQTT callbacks, and querying databases.

```
import asyncio
import aiohttp
```

```
async def read_sensor(sensor_id: str) -> dict:
    """Simulate async sensor read."""
    await asyncio.sleep(0.1)  # simulate I/O
```

```
    return {"id": sensor_id, "value": 25.3}
```

```
async def read_all_sensors(sensor_ids: list[str]) -> list[dict]:
    """Read all sensors concurrently — much faster than sequential."""
    tasks = [read_sensor(sid) for sid in sensor_ids]
    return await asyncio.gather(*tasks)  # run all simultaneously

# Run: all sensors read in parallel, not sequentially
results = asyncio.run(read_all_sensors(["S001", "S002", "S003", "S004"]))
```

B.3 Type Hints — Use Always

```
from typing import TypedDict, Literal

class SensorAlert(TypedDict):
    sensor_id: str
    severity: Literal["info", "warning", "critical"]
    message: str
    value: float
    threshold: float

def check_threshold(reading: SensorReading) -> SensorAlert | None:
    """Return alert if reading exceeds threshold, else None."""
    THRESHOLDS = {"temperature": {"warning": 35.0, "critical": 45.0}}
    if reading.sensor_type in THRESHOLDS:
        thresholds = THRESHOLDS[reading.sensor_type]
        if reading.value >= thresholds["critical"]:
            return SensorAlert(
                sensor_id=reading.sensor_id,
                severity="critical",
                message=f"Critical: {reading.value}{reading.unit}",
                value=reading.value,
                threshold=thresholds["critical"]
            )
    return None
```

B.4 NumPy Cheat Sheet for Edge AI Preprocessing

Operation	Code	Use Case
Create array	np.array([1, 2, 3])	Sensor reading buffer
Zeros/ones	np.zeros((224, 224, 3))	Image buffer initialisation
Random data	np.random.rand(100)	Test data generation
Normalise 0-1	(x - x.min()) / (x.max() - x.min())	Model input preprocessing
Standardise	(x - x.mean()) / x.std()	Feature normalisation

Reshape	x.reshape(1, 10)	Add batch dimension for model
Slice	x[10:20, :, :]	Crop image region
Transpose	x.transpose(2, 0, 1)	HWC → CHW for PyTorch
Mean per axis	x.mean(axis=0)	Average across time steps
Argmax	np.argmax(output)	Get predicted class index
Stack arrays	np.stack([a, b, c], axis=0)	Create batch from samples
Cast dtype	x.astype(np.float32)	Required for TFLite input

B.5 Error Handling Patterns for Production AIoT

```
import logging
from functools import wraps
import time
```

```
def retry(max_attempts: int = 3, delay: float = 1.0, backoff: float = 2.0):
    """Decorator: retry failed I/O operations with exponential backoff."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            attempt = 0
            current_delay = delay
            while attempt < max_attempts:
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    attempt += 1
                    if attempt == max_attempts:
                        logging.error(f"{func.__name__} failed after {max_attempts}
attempts: {e}")
                        raise
                    logging.warning(f"{func.__name__} attempt {attempt} failed:
{e}. Retrying in {current_delay}s")
                    time.sleep(current_delay)
                    current_delay *= backoff
        return wrapper
    return decorator
```

```
@retry(max_attempts=5, delay=0.5)
def read_i2c_sensor(address: int) -> dict:
    """Read from I2C sensor with automatic retry on bus error."""
    # ... actual I2C read code
    pass
```

Appendix C: Japanese Technical Vocabulary for AIoT Engineers

Learning Japanese is most effective when vocabulary is tied to your domain. These are the Japanese words and phrases you will encounter in Japanese AIoT and engineering workplaces. Learn these alongside your N5/N4 JLPT vocabulary.

C.1 Engineering Domain Vocabulary

Japanese	Reading	Meaning	Usage Context
人工知能	じんこうちのう	Artificial Intelligence	AI system discussions
機械学習	きかいがくしゅう	Machine Learning	ML project descriptions
深層学習	しんそうがくしゅう	Deep Learning	Neural network work
エッジ AI	えっじ AI	Edge AI	Your core domain
センサー	せんさー	Sensor	IoT hardware
データ収集	データしゅうしゅう	Data collection	IoT logging
リアルタイム処理	りあるたいむしょり	Real-time processing	Edge compute
組み込みシステム	くみこみしすてむ	Embedded system	Your EEE domain
自動化	じどうか	Automation	Factory AIoT
品質管理	ひんしつかんり	Quality control	Manufacturing AI
予知保全	よちほぜん	Predictive maintenance	Key AIoT application
異常検知	いじょうけんち	Anomaly detection	Core AIoT function
推論	すいろん	Inference	Model inference on edge
学習済みモデル	がくしゅうずみもでる	Trained model	Deployed TFLite model
通信プロトコル	つうしんぷろとこる	Communication protocol	MQTT, OPC-UA
ネットワーク	ねっとわーく	Network	IoT network setup
クラウド	くらうど	Cloud	MongoDB Atlas etc.
ファームウェア	ふぁーむうぇあ	Firmware	Embedded software
マイコン	まいこん	Microcontroller	ESP32, Arduino

C.2 Workplace Japanese — Daily Phrases

Situation	Japanese	Reading	Translation
Greeting a colleague	お疲れ様です	おつかれさまです	Thank you for your work (standard greeting)
Entering office	おはようございます	おはようございます	Good morning (even at afternoon)
Leaving office	お先に失礼します	おさきにしつれいします	Excuse me for leaving before you
I understand	承知しました	しょうちしました	Understood (formal)
I will do it	対応します	たいおうします	I will handle it

I have a question	質問があります	しつもんがあります	I have a question
Please check this	ご確認お願いします	ごかくにんおねがいします	Please confirm/check this
Report to supervisor	ご報告します	ごほうこくします	I would like to report
Meeting request	ミーティングをお願いできますか	-	Could I request a meeting?
I made a mistake	申し訳ありません	もうしわけありません	I am very sorry (sincere apology)
Project status	進捗を報告します	しんちょくをほうこくします	I will report the progress
Deadline	締め切りはいつですか	しめきりはいつですか	When is the deadline?

C.3 Japanese Email Template for Engineers

Every Japanese business email follows this structure. Memorise it.

```
Subject: 【ご報告】IoT センサープロジェクト進捗
(Translation: [Report] IoT Sensor Project Progress)
```

```
〇〇様
(To: [Name]-sama)
```

```
お世話になっております。[Your name]と申します。
(Thank you for your continued support. My name is [Your name].)
```

```
本日は、センサーシステムの開発状況についてご報告いたします。
(Today I would like to report on the development status of the sensor system.)
```

```
[Main content here]
```

```
ご確認のほど、よろしくお願いいたします。
(I kindly ask for your review.)
```

```
[Your Name]
[Your Title]
[Company Name]
[Email] | [Phone]
```

C.4 Japanese Technical Interview Phrases

English	Japanese	Reading
My strength is Edge AI development	私の強みはエッジ AI 開発です	わたしのつよみはえっじ AI かい

		はつです
I have experience with Raspberry Pi	ラズベリーパイの経験があります	らずべりーぱいのけいけんがあります
I can speak Japanese at N4 level	日本語能力は N4 レベルです	にほんごのうりょくは N4 れべるです
I studied electrical engineering	電気電子工学を学びました	でんきでんしこうがくをまなびました
I want to work in Japan long-term	長期的に日本で働きたいです	ちょうきてきににほんではたらきたいです
I am interested in manufacturing AI	製造業の AI に興味があります	せいぞうぎょうの AI にきょうみがあります

Appendix D: Complete Interview Preparation Guide

D.1 Technical Interview Question Bank — AIoT Roles

Category 1: Python and Machine Learning

- Explain the difference between supervised and unsupervised learning with an IoT example.
  - Answer: Supervised — classify sensor readings as normal/fault using labelled historical data. Unsupervised — cluster sensor patterns to find unknown failure modes.
- What is a neural network? Explain forward and backward pass.
- What is overfitting and how do you prevent it?
  - Answer: Model performs well on training data but poorly on new data. Prevention: dropout, regularisation (L2), data augmentation, early stopping.
- Explain the difference between batch normalisation and layer normalisation.
- What is transfer learning? Give an AIoT example.
  - Answer: Use a pre-trained MobileNet (trained on ImageNet) and fine-tune on your specific factory defect images. Much less data required.
- What is the purpose of the softmax function?
- Explain gradient descent. What is the learning rate?

Category 2: Edge AI and Hardware

- What is model quantisation? What are the tradeoffs?
  - Answer: Reducing weight precision from FP32 to INT8. Tradeoffs: 4x smaller model, 2-3x faster inference, slight accuracy reduction (typically <2%).
- What is the difference between TFLite and ONNX Runtime?
  - Answer: TFLite is TensorFlow-specific, highly optimised for ARM processors. ONNX Runtime is framework-agnostic (PyTorch, TF, sklearn), runs on more hardware types.
- How would you optimise inference speed on a Raspberry Pi?
  - Answer: INT8 quantisation, smaller model (MobileNet vs ResNet), reduce input resolution, use NEON SIMD instructions, profile with TFLite Benchmark Tool.
- What is the difference between I2C and SPI? When do you use each?
  - Answer: I2C uses 2 wires (SDA, SCL), slower but multiple devices on same bus. SPI uses 4 wires (MISO, MOSI, SCK, CS), faster, used for displays and high-speed ADCs.
- What happens if a Raspberry Pi loses internet during Edge AI inference?
  - Answer: Inference continues normally — TFLite runs locally. MQTT publishing fails. Implement a local queue that buffers data and sends when connection restores.

Category 3: IoT and Protocols

- Explain MQTT QoS levels. When do you use each?
  - QoS 0: Fire and forget — high-frequency sensor data where loss is acceptable. QoS 1: Alert messages — at least once delivery. QoS 2: Actuator commands — exactly once, critical.
- What is a message broker? How does Mosquitto work?
- Explain the publish-subscribe pattern vs request-response. When is each better?
  - Pub/Sub: IoT sensors (many publishers, many subscribers, decoupled). Request/Response: REST API (client needs immediate response to a specific query).
- What is a retained message in MQTT?

- Answer: The broker stores the last message on a topic. New subscribers immediately receive this value — useful for device status (online/offline) that rarely changes.

Category 4: System Design for AIoT

- Design an IoT system that monitors 1000 sensors in a factory.
  - Answer template: Sensors → MQTT with QoS 1 → Edge Pi per zone (batch processing, local TFLite) → aggregated MQTT to central broker → MongoDB Atlas → Dashboard + Alerting
- How would you ensure your AIoT system works during internet outages?
  - Answer: Edge processing on Pi (no cloud needed for inference), local SQLite buffering during outage, sync to Atlas on reconnect using change streams.
- How would you handle 10ms inference latency requirements?
  - Answer: MobileNetV3-Small with INT8 quantisation achieves 8-12ms on Pi 4. If not enough: NVIDIA Jetson Nano for GPU acceleration, or resize input to 128x128.

D.2 HR Interview Answers — Tailored to Your Story

"Tell me about yourself" — 90-second answer template:

"I am a B.Tech CS-IT student at Malla Reddy University, with a diploma in Electrical and Electronics Engineering. My unique background combines hardware knowledge from my EEE diploma with software skills in Python, C++, and Edge AI. I am building toward a career as an AIoT engineer, specifically working on systems where machine learning models run on embedded hardware rather than the cloud. I have built projects including a Raspberry Pi-based edge inference pipeline using TFLite and MQTT, and a sensor anomaly detection system powered by the Claude API. I am also studying Japanese at N4 level, as I am targeting roles in Japan's industrial AI sector. I am here because [company name]'s work on [specific product] directly aligns with what I want to contribute to."

"Why AIoT / Edge AI specifically?"

"During my EEE diploma, I worked with embedded systems and physical hardware and realised that the future of manufacturing is not just about sensors collecting data — it is about making decisions at the sensor itself, without depending on cloud connectivity. Japanese factories, for example, are often air-gapped for security reasons. Edge AI solves this — the intelligence lives on the device. My EEE background means I understand the hardware layer that most CS graduates never see, which I think makes me a stronger AIoT engineer."

"Where do you see yourself in 5 years?"

"I plan to be working in Japan as a senior AIoT engineer at a manufacturing or robotics company, contributing to industrial automation systems. I am studying Japanese specifically for this — I want to be able to collaborate deeply with Japanese engineering teams, not just write code in isolation. After 3-4 years in Japan, I intend to return to India and start a company in Vizag focused on bringing Japanese-quality industrial IoT systems to Indian manufacturers. The experience and network I build in Japan will be the foundation of that."

D.3 Resume Bullet Points — Copy and Customise

Use these templates for your resume projects section. Replace [X] with your actual measurements.

- Built an Edge AI inference pipeline on Raspberry Pi 4 using TFLite INT8 quantisation, achieving [X]ms inference latency on MobileNet — [X]% faster than full TensorFlow equivalent

- Developed an MQTT-based sensor monitoring system processing [X] readings/minute from [X] virtual sensors, with automatic anomaly detection using Claude API integration
- Implemented a MERN stack IoT dashboard with real-time WebSocket updates, handling [X] concurrent sensor data streams and JWT-authenticated user sessions
- Solved [X]+ LeetCode problems (Easy/Medium/Hard: [X]/[X]/[X]) covering DSA patterns including dynamic programming, graph algorithms, and sliding window
- Built a LangChain AI agent with MongoDB Atlas tool integration, enabling natural language queries over [X] sensor readings with [X]ms average response time
- Completed JLPT N4 preparation with [X]% score on practice tests; prior N5 certification; targeting N2 by 2028 for Japan employment

Appendix E: Weekly Study Tracker and Progress Log

Print this page or recreate it in Notion/Obsidian. Fill it in every Sunday before planning the next week. Honest self-assessment is the fastest way to identify and fix weak spots.

E.1 Weekly Template

Metric	Week 1	Week 2	Week 3	Week 4
LeetCode problems solved				
Striver sheet problems				
DSA topics completed				
PyTorch/AI hours				
Sunday project done?				
GitHub commits				
Japanese vocab (new)				
Japanese Anki streak				
Internship apps sent				
Interview responses				
NIAT project progress				
Mood (1-5)				
Biggest win this week				
Biggest struggle				

E.2 4-Month Skills Progression Log

Skill	Month 1 Level	Month 2 Level	Month 3 Level	Month 4 Level
DSA C++ (Striver sheet %)				
Python confidence (1-10)				
PyTorch (1-10)				
TFLite/Edge AI (1-10)				
MERN Stack (1-10)				
MySQL/MongoDB (1-10)				

Linux CLI (1-10)				
MQTT (1-10)				
Raspberry Pi (1-10)				
Japanese N4 mock score				
Claude API (1-10)				
n8n automation (1-10)				
LangChain (1-10)				
GitHub repos (count)				
LinkedIn followers				

E.3 Internship Application Tracker

Company	Role	Applied Date	Platform	Status	Follow-up

End of AIoT Engineer Complete Study Guide

Version 1.0 | September 2026 | Personalised for your 2025–2033 career path

Appendix F: Complete PyTorch Reference for AIoT Engineers

PyTorch is your primary model training framework. This appendix is a complete reference covering every concept you need from beginner through to Edge AI deployment. Study alongside Daniel Bourke's freeCodeCamp course.

F.1 Tensor Operations — Complete Reference

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
```

```
# ── CREATION ──
t1 = torch.tensor([1.0, 2.0, 3.0])          # from list
t2 = torch.zeros(3, 4)                       # 3x4 zeros
t3 = torch.ones(2, 3)                        # 2x3 ones
t4 = torch.rand(5, 5)                        # uniform random [0,1]
t5 = torch.randn(5, 5)                       # normal random (mean=0, std=1)
t6 = torch.arange(0, 10, 2)                  # [0, 2, 4, 6, 8]
t7 = torch.linspace(0, 1, 100)               # 100 evenly spaced points
t8 = torch.eye(4)                            # 4x4 identity matrix
```

```
# ── SHAPE OPERATIONS ──
t = torch.rand(2, 3, 4)
t.shape                                      # torch.Size([2, 3, 4])
t.reshape(6, 4)                              # reshape (same total elements)
t.view(2, -1)                                # view (-1 = infer dimension)
t.squeeze()                                  # remove all dim=1 dimensions
t.unsqueeze(0)                               # add dimension at position 0
t.permute(2, 0, 1)                           # reorder dimensions (HWC -> CHW)
t.transpose(0, 1)                            # swap two dimensions
torch.cat([t1, t2], dim=0)                   # concatenate along axis
torch.stack([t1, t2], dim=0)                 # stack (adds new dimension)
```

```
# ── MATH OPERATIONS ──
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
a + b                                        # element-wise add
a * b                                        # element-wise multiply
torch.matmul(a, b)                           # dot product (or matrix multiply)
a @ b                                        # same as matmul
a.mean()                                     # scalar mean
a.mean(dim=0)                                # mean along axis
a.sum()                                      # sum all elements
```

```
a.max()                                    # max value
a.argmax()                                 # index of max value
torch.abs(a)                               # absolute value
torch.exp(a)                               # e^x for each element
torch.log(a)                               # natural log
torch.sqrt(a)                              # square root
```

```
# ── DEVICE (CPU / GPU) ──
device = "cuda" if torch.cuda.is_available() else "cpu"
t = t.to(device)                           # move tensor to device
model = model.to(device)                   # move model to device
# Always move BOTH model and data to same device before training
```

```
# ── GRADIENTS (AUTOGRAD) ──
x = torch.tensor([2.0], requires_grad=True) # track gradient
y = x ** 2 + 3 * x + 1                     # forward pass
y.backward()                               # compute gradient dy/dx
x.grad                                     # gradient: 2*x + 3 = 7.0
# Always call optimizer.zero_grad() before backward()
# Otherwise gradients accumulate across batches
```

F.2 Neural Network Layers — Complete Reference

Layer	Code	Use Case	Key Params
Linear	nn.Linear(in, out)	Feedforward, classifiers	in_features, out_features
Conv2d	nn.Conv2d(in_ch, out_ch, kernel)	Image features, Edge AI vision	in_channels, out_channels, kernel_size
MaxPool2d	nn.MaxPool2d(kernel)	Downsample feature maps	kernel_size, stride
BatchNorm2d	nn.BatchNorm2d(channels)	Stabilise training	num_features
Dropout	nn.Dropout(p=0.5)	Prevent overfitting	p (drop probability)
ReLU	nn.ReLU()	Non-linearity (most common)	None
Sigmoid	nn.Sigmoid()	Binary classification output	None
Softmax	nn.Softmax(dim=1)	Multi-class output probabilities	dim
LSTM	nn.LSTM(input, hidden)	Time-series sensor data	input_size, hidden_size, num_layers
Embedding	nn.Embedding(vocab, dim)	Text/token encoding	num_embeddings, embedding_dim
Flatten	nn.Flatten()	Conv → Linear bridge	start_dim, end_dim
Sequential	nn.Sequential(*layers)	Chain layers in order	Takes list of modules

F.3 Complete Training Loop — Production Template

```
def train_model(model, train_loader, val_loader, epochs=10, lr=1e-3):
    """Production training loop with validation, early stopping, logging."""
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=3)

    best_val_loss = float("inf")
    patience_counter = 0
    history = {"train_loss": [], "val_loss": [], "val_acc": []}

    for epoch in range(epochs):
        # ── TRAINING PHASE ──
        model.train()
        train_loss = 0.0
        for batch_X, batch_y in train_loader:
            batch_X, batch_y = batch_X.to(device), batch_y.to(device)
            optimizer.zero_grad()             # clear previous gradients
            outputs = model(batch_X)          # forward pass
            loss = criterion(outputs, batch_y) # compute loss
            loss.backward()                   # backpropagation
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # prevent
gradient explosion
            optimizer.step()                  # update weights
            train_loss += loss.item()

        # ── VALIDATION PHASE ──
        model.eval()
        val_loss, correct, total = 0.0, 0, 0
        with torch.no_grad():                 # no gradient tracking needed
            for batch_X, batch_y in val_loader:
                batch_X, batch_y = batch_X.to(device), batch_y.to(device)
                outputs = model(batch_X)
                val_loss += criterion(outputs, batch_y).item()
                _, predicted = outputs.max(1)
                total += batch_y.size(0)
                correct += predicted.eq(batch_y).sum().item()

        avg_train = train_loss / len(train_loader)
        avg_val = val_loss / len(val_loader)
        val_acc = 100.0 * correct / total
        scheduler.step(avg_val)
```

```
        history["train_loss"].append(avg_train)
        history["val_loss"].append(avg_val)
        history["val_acc"].append(val_acc)

        print(f"Epoch {epoch+1}/{epochs} | Train: {avg_train:.4f} | Val:
{avg_val:.4f} | Acc: {val_acc:.2f}%")

        # ── EARLY STOPPING ──
        if avg_val < best_val_loss:
            best_val_loss = avg_val
            torch.save(model.state_dict(), "best_model.pth")
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= 5:
                print(f"Early stopping at epoch {epoch+1}")
                break

    return history
```

F.4 Custom Dataset for Sensor Data

```
from torch.utils.data import Dataset, DataLoader
import pandas as pd

class SensorDataset(Dataset):
    """Custom PyTorch Dataset for time-series sensor readings."""

    def __init__(self, csv_path: str, window_size: int = 10, transform=None):
        self.data = pd.read_csv(csv_path)
        self.window_size = window_size
        self.transform = transform
        self.features = ["temperature", "humidity", "vibration", "current"]
        self.label_map = {"normal": 0, "warning": 1, "fault": 2, "critical": 3}

    def __len__(self) -> int:
        return len(self.data) - self.window_size

    def __getitem__(self, idx: int):
        # Get window of sensor readings
        window = self.data.iloc[idx:idx+self.window_size][self.features].values
        label = self.data.iloc[idx+self.window_size]["label"]
```

```
        X = torch.tensor(window, dtype=torch.float32)
        y = torch.tensor(self.label_map[label], dtype=torch.long)

        if self.transform:
            X = self.transform(X)
        return X, y

# Usage
dataset = SensorDataset("sensor_log.csv", window_size=20)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_ds, val_ds = torch.utils.data.random_split(dataset, [train_size, val_size])

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=2)
val_loader = DataLoader(val_ds, batch_size=32, shuffle=False)
```

F.5 Loss Functions Reference

Loss Function	Code	Use When	Output Layer
CrossEntropyLoss	nn.CrossEntropyLoss()	Multi-class classification	nn.Linear (raw logits)
BCEWithLogitsLoss	nn.BCEWithLogitsLoss()	Binary or multi-label	nn.Linear (raw logits)
MSELoss	nn.MSELoss()	Regression, value prediction	nn.Linear
L1Loss (MAE)	nn.L1Loss()	Regression, outlier robust	nn.Linear
HuberLoss	nn.HuberLoss()	Regression with outliers	nn.Linear
NLLLoss	nn.NLLLoss()	When using log_softmax output	nn.LogSoftmax

F.6 Common Mistakes and Fixes

Mistake	Symptom	Fix
Forgot zero_grad()	Loss oscillates wildly, gradients explode	Add optimizer.zero_grad() before loss.backward()
Wrong device	RuntimeError: Expected all tensors on same device	Move both model AND data to same device
Forgot model.eval()	Validation accuracy is wrong (dropout active)	Call model.eval() before validation loop
Forgot torch.no_grad()	Memory error during validation	Wrap validation in with torch.no_grad():
Wrong loss function	Loss is NaN from first epoch	CrossEntropyLoss expects raw logits (no Softmax)
Wrong input shape	RuntimeError: shape mismatch	Print tensor.shape at each layer to

		debug
Learning rate too high	Loss diverges (goes to inf)	Reduce lr by 10x, start with 1e-3 or 1e-4
Batch size too large	CUDA out of memory	Reduce batch_size, or use gradient accumulation

🔗 **PyTorch Official Tutorials:** pytorch.org/tutorials
🔗 **Learn PyTorch Book (Daniel Bourke):** learnpytorch.io
🔗 **PyTorch for Deep Learning in a Day (YouTube):** youtube.com/watch?v=Z_ikDlimN6A

Appendix G: Linux Command Line Reference for AIoT Engineers

You will use Linux every day as an AIoT engineer — on your Raspberry Pi, on remote servers, in Docker containers, and in CI/CD pipelines. Mastering the command line is not optional. This is your complete cheat sheet.

G.1 Navigation and File Management

Command	Description	Example
pwd	Print working directory	pwd → /home/pi/aiot_project
ls -la	List all files with details	ls -la ~/projects
cd	Change directory	cd /etc/mosquitto
mkdir -p	Create directory (and parents)	mkdir -p ~/projects/aiot/data
cp -r	Copy recursively	cp -r models/ backup/models/
mv	Move or rename	mv sensor_v1.py sensor_v2.py
rm -rf	Remove directory recursively	rm -rf __pycache__/
find	Search for files	find . -name "*.tflite"
du -sh	Disk usage of directory	du -sh /home/pi/models/
df -h	Disk space available	df -h

G.2 Process Management

Command	Description	AIoT Use Case
ps aux	List all processes	Check if mqtt subscriber is running
top / htop	Live process monitor	Monitor CPU/RAM during TFLite inference
kill -9 PID	Force kill process	Stop a stuck sensor reader
nohup cmd &	Run in background (survives logout)	Run MQTT publisher permanently
systemctl status	Check service status	systemctl status mosquitto
systemctl restart	Restart a service	systemctl restart n8n
journalctl -u	View service logs	journalctl -u mosquitto -f
screen / tmux	Persistent terminal sessions	Run Pi script that persists over SSH

G.3 Networking Commands

Command	Description	Example
ifconfig / ip a	Show IP addresses	ip a | grep inet
ping	Test connectivity	ping 192.168.1.100
ssh user@host	Remote login	ssh pi@192.168.1.50
scp src dst	Copy file over SSH	scp model.tflite

		pi@192.168.1.50:~/models/
rsync -avz	Sync directory over SSH	rsync -avz ./project/ pi@192.168.1.50:~/project/
netstat -tlpn	Open ports	netstat -tlpn | grep 1883
curl	HTTP request from terminal	curl http://localhost:5678/healthz
wget	Download file	wget https://example.com/model.tflite
ufw allow	Open firewall port	ufw allow 1883 (MQTT)
arp -a	Find devices on network	arp -a (find Pi IP from laptop)

G.4 Text Processing (grep, awk, sed)

```
# grep: search in files or pipe output
grep "ERROR" /var/log/aiot.log             # find error lines
grep -r "mqtt" ~/projects/                 # search all files recursively
grep -n "temperature" sensor.py            # show line numbers
ps aux | grep python                       # filter running Python processes
```

```
# awk: process columnar text
awk -F"," "{print $1, $3}" data.csv        # print columns 1 and 3
awk "NR>1" data.csv                        # skip header row
awk "{sum+=$2} END {print sum/NR}" data.csv # average column 2
```

```
# sed: find and replace
sed -i "s/localhost/192.168.1.50/g" config.yaml  # replace in file
sed -n "10,20p" sensor.log                # print lines 10-20
cat sensor.log | tail -100                 # last 100 lines
cat sensor.log | tail -f                   # follow live log output
```

G.5 Shell Scripts for AIoT Automation

Script 1: Auto-start AIoT pipeline on Pi boot

```
#!/bin/bash
# /home/pi/start_aiot.sh

LOG="/home/pi/logs/aiot_$(date +%Y%m%d_%H%M%S).log"
mkdir -p /home/pi/logs

echo "Starting AIoT pipeline at $(date)" | tee -a $LOG

# Start MQTT broker
sudo systemctl start mosquitto
sleep 2
```

```
# Start sensor publisher (background)
python3 /home/pi/aiot/sensor_publisher.py >> $LOG 2>&1 &
echo "Sensor publisher PID: $!" | tee -a $LOG

# Start TFLite inference engine (background)
python3 /home/pi/aiot/edge_inference.py >> $LOG 2>&1 &
echo "Inference engine PID: $!" | tee -a $LOG

echo "AIoT pipeline started. Check $LOG for details."
```

Script 2: Daily backup of sensor data

```
#!/bin/bash
# /home/pi/backup_data.sh — run via cron: 0 23 * * * /home/pi/backup_data.sh

DATE=$(date +%Y-%m-%d)
BACKUP_DIR="/home/pi/backups/$DATE"
mkdir -p $BACKUP_DIR

# Copy sensor logs
cp /home/pi/data/*.csv $BACKUP_DIR/

# Compress
tar -czf /home/pi/backups/aiot_data_$DATE.tar.gz -C /home/pi/backups $DATE
rm -rf $BACKUP_DIR

# Keep only last 7 days
find /home/pi/backups/*.tar.gz -mtime +7 -delete

echo "Backup complete: aiot_data_$DATE.tar.gz"
```

G.6 Cron Jobs — Schedule Automation

```
# Edit cron table
crontab -e

# Cron format: minute hour day month weekday command
# ┌──────────────── minute (0-59)
# │ ┌────────────── hour (0-23)
# │ │ ┌──────────── day (1-31)
# │ │ │ ┌────────── month (1-12)
# │ │ │ │ ┌──────── weekday (0=Sun, 6=Sat)
# │ │ │ │ │

# Run backup every night at 11 PM
0 23 * * * /home/pi/backup_data.sh >> /home/pi/logs/backup.log 2>&1

# Run sensor reader every 5 minutes
```

```
*/5 * * * * python3 /home/pi/read_sensors.py

# Restart MQTT broker every Monday at 3 AM (maintenance window)
0 3 * * 1 sudo systemctl restart mosquitto
```

G.7 Docker for AIoT — Containerise Everything

Docker allows you to package your AIoT application with all dependencies into a container. This means your sensor pipeline runs identically on your Raspberry Pi, your laptop, and any cloud server. For remote internships, sending a Docker container is how you submit reproducible work.

```
# Dockerfile for AIoT sensor pipeline
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    libatlas-base-dev \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Set environment variables
ENV MQTT_HOST=mosquitto
ENV MQTT_PORT=1883
ENV LOG_LEVEL=INFO

CMD ["python3", "sensor_publisher.py"]
```

```
# docker-compose.yml for full AIoT stack
version: "3.8"
services:
  mosquitto:
    image: eclipse-mosquitto:2
    ports: ["1883:1883"]
    volumes: ["./mosquitto.conf:/mosquitto/config/mosquitto.conf"]

  sensor_publisher:
    build: ./sensor
    depends_on: [mosquitto]
    environment:
```

```
    MQTT_HOST: mosquitto

inference_engine:
  build: ./inference
  depends_on: [mosquitto]
  volumes: ["./models:/app/models"]

dashboard:
  build: ./dashboard
  ports: ["3000:3000"]
  depends_on: [mosquitto]
```

```
# Commands
docker compose up -d                        # start all services
docker compose logs -f inference_engine     # follow logs
docker compose down                         # stop all
```

🔗 **Docker for Beginners (TechWorld with Nana):** youtube.com/watch?v=3c-iBn73dDE
🔗 **Docker Compose Tutorial:** youtube.com/watch?v=DM65_JyGxCo

Appendix H: GitHub Profile and Portfolio Mastery

Your GitHub profile is the single most important document in your engineering career right now — more than your resume, more than your college transcripts. Recruiters at AIoT companies will look at your GitHub before they read anything else. This appendix teaches you to build a GitHub profile that converts visitors into interview calls.

H.1 Profile README — Template

Create a special repository with the same name as your GitHub username. The README.md in this repo appears on your profile page. This is prime real estate.

```
# 👋 Hi, I am [Your Name]

**AIoT Engineer in Training | Edge AI | Raspberry Pi | Japan-Bound 🇯🇵**

I am a B.Tech CS-IT student (EEE Diploma background) building toward
a career as an AIoT engineer in Japan. I bridge hardware and AI:
running neural networks on Raspberry Pi, connecting physical sensors
to intelligent cloud systems, and automating IoT workflows.

## 🛠️ Tech Stack
**Edge AI:** Python · PyTorch · TFLite · ONNX · OpenCV
**IoT:** Raspberry Pi · ESP32 · MQTT · Node-RED · OPC-UA
**Web:** React · Node.js · Express · MongoDB · MySQL
**AI Tools:** Claude API · LangChain · n8n · Cursor IDE
**Systems:** C++ · Linux · Docker · Git
**Language:** Japanese N4 (targeting N2 by 2028)

## 🚀 Featured Projects
| Project | Description | Tech |
|---------|-------------|------|
| [AIoT Edge Pipeline](link) | TFLite on Pi + MQTT + Dashboard | Python, TFLite,
MQTT |
| [AI Sensor Analyst](link) | Claude API anomaly detection | Python, Claude API |
| [MERN IoT Dashboard](link) | Full-stack real-time dashboard | React, Node,
MongoDB |
| [DSA Solutions](link) | 120+ LeetCode solutions in C++ | C++ |

## 📊 GitHub Stats
![GitHub Stats](https://github-readme-stats.vercel.app/api?
username=YOURUSERNAME&show_icons=true)

## 📫 Connect
[![LinkedIn](badge)](your-linkedin) · [![Email](badge)](mailto:your@email)
```

H.2 Project README Template — Every Repo Must Have This

````
# Project Name

> One-sentence description of what this does.

![Demo GIF or Screenshot]

## What This Does
2-3 sentences: the problem, how this solves it, the result.

## Architecture
Diagram or bullet list: component 1 → component 2 → output

## Tech Stack
- Python 3.11, PyTorch, TFLite
- MQTT (Mosquitto), Node-RED
- Raspberry Pi 4, DHT22 sensor

## Results
- Inference latency: X ms on Raspberry Pi 4
- Accuracy: X% on test dataset
- Throughput: X sensor readings/minute

## Quick Start
```bash
git clone https://github.com/you/project
pip install -r requirements.txt
python main.py
```

## Demo
[Watch the 2-minute demo video](youtube-link)

## What I Learned
3 specific technical insights from building this project.
````

H.3 GitHub Best Practices

Practice	What to Do	Why It Matters
Commit messages	feat: add MQTT reconnection logic	Shows professional engineering discipline
Branch naming	feature/sensor-anomaly-detection	Demonstrates Git workflow knowledge
Commit frequency	Commit every study session (at minimum)	Green contribution graph = visual proof of consistency
.gitignore	Always exclude .env, __pycache__,	Security — never commit API keys

	*.pyc	
Issues	Open issues for planned features	Shows project planning mindset
Tags/Releases	Tag major versions (v1.0, v2.0)	Professional project management
License	Add MIT license to all public repos	Allows others to use/contribute
Topics	Add: python, iot, edge-ai, raspberry-pi	Improves discoverability on GitHub

H.4 LinkedIn Strategy for AIoT Internship Hunt

Profile Headline (most important line):

```
B.Tech CS-IT | Edge AI & AIoT Engineer | Python · TFLite · Raspberry Pi · Japanese N4
```

About Section (150 words max):

"I am a B.Tech CS-IT student with an EEE diploma background, building a career at the intersection of Embedded Systems and Artificial Intelligence. My focus is Edge AI — deploying machine learning models on Raspberry Pi and ESP32 devices to create intelligent, real-time IoT systems that work without cloud dependency. I have built projects including an end-to-end AIoT predictive maintenance pipeline (TFLite + MQTT + MongoDB), an AI-powered sensor anomaly detection system using the Claude API, and a full-stack IoT dashboard in the MERN stack. I am also studying Japanese (N4 level) as I am targeting AIoT engineering roles in Japan's manufacturing sector by 2029. Open to remote AIoT, embedded ML, and IoT software internship opportunities."

Posting Strategy — What to post and when:

- Every Sunday: Post about your weekly project (photo/screenshot + 3-sentence description + GitHub link)
- Every 2 weeks: Share something you learned (a technical concept explained simply)
- Once a month: Post a demo video of a project running on your Pi
- Engage daily: Comment on 3 AIoT/embedded AI posts with a genuine technical insight

📌 **Note:** LinkedIn posts with a video get 3x more impressions than text-only posts. Even a phone recording of your Node-RED dashboard updating in real time gets significantly more views than a text update. Record demos on your phone and post directly from the LinkedIn mobile app.

🔗 **GitHub Profile README Generator:** github.com/rahuldkjain/github-profile-readme-generator
🔗 **GitHub Stats Widget:** github.com/anuraghazra/github-readme-stats

Appendix I: Five Complete AIoT Project Architectures

These are five detailed project architectures you can build during your B.Tech years. Each one is more complex than the previous. They are designed to be NIAT-grade projects — suitable for industry demonstrations and internship portfolio submissions.

I.1 Architecture 1: Smart Motor Health Monitor (NIAT Project Level)

Problem: Industrial motors fail unexpectedly, causing costly downtime.

Solution: Real-time vibration + temperature analysis with AI-powered failure prediction.

Component	Technology	Role
Accelerometer (ADXL345)	I2C → Raspberry Pi	Vibration signature measurement
Temperature sensor (DS18B20)	1-Wire → Raspberry Pi	Motor bearing temperature
Current sensor (ACS712)	SPI ADC → Raspberry Pi	Motor current draw
TFLite model	Python on Pi	Classify: normal/imbalance/bearing_fault/overload
MQTT broker	Mosquitto on Pi	Route sensor data and alerts
Node-RED dashboard	Pi port 1880	Live gauges and trend charts
MongoDB Atlas	Cloud	Historical data for trend analysis
n8n workflow	VPS or Pi	Email/WhatsApp alert on fault detection
Claude API	Python script	Natural language fault explanation

Data flow:

Sensors → Pi GPIO → Python reader → TFLite inference → MQTT publish → Node-RED dashboard AND MongoDB Atlas AND n8n alert workflow → Email/WhatsApp with AI explanation

Model training data:

Record 1 hour of normal vibration. Simulate faults: attach small weight (imbalance), increase load (overload). Label data. Train 4-class classifier in PyTorch. Convert to TFLite INT8. Target: >90% accuracy, <20ms inference on Pi 4.

I.2 Architecture 2: Greenhouse AI Climate Controller

Problem: Manual greenhouse monitoring is labour-intensive and error-prone.

Solution: Autonomous climate control using AI-driven sensor fusion and actuator control.

Component	Technology	Role
BME280	I2C	Temperature, humidity, pressure
Soil moisture sensor	ADC	Soil water content

Light sensor (BH1750)	I2C	Lux measurement for plant growth
CO2 sensor (MH-Z19)	UART	CO2 ppm for photosynthesis optimisation
Relay module (4-channel)	GPIO output	Control fan, heater, irrigation, lights
LSTM model (PyTorch → TFLite)	Pi	Predict optimal climate conditions
Rule engine (Python)	Pi	Translate AI predictions to relay commands
REST API (Flask)	Pi	Remote control from phone or n8n
n8n schedule	Daily 6AM	Generate AI crop health report via Claude

I.3 Architecture 3: Factory Quality Inspection Vision System

Problem: Manual visual inspection is slow and inconsistent.

Solution: Camera-based AI inspection that flags defective products in real time.

Component	Technology	Role
Pi Camera Module 3	CSI	Capture product images at 30 FPS
Trigger signal	GPIO input from conveyor sensor	Capture only when product present
MobileNetV3-Small (TFLite INT8)	Pi CPU	Binary classify: OK / DEFECT
OpenCV preprocessing	Python	Crop, resize, normalise image to 224x224
MQTT publisher	paho-mqtt	Publish result + confidence per item
LED indicators	GPIO output	Green=OK, Red=DEFECT (physical signal)
Solenoid valve (via relay)	GPIO	Reject defective items physically
SQLite database	Local on Pi	Log all inspections with timestamps
Flask dashboard	Pi port 5000	Show inspection rate, defect %, recent images

Key metrics to measure and document:

- Inference latency: target <40ms (25+ FPS processing rate)
- Classification accuracy: target >95% on test set
- False positive rate: target <2% (OK product flagged as defect)
- False negative rate: target <1% (defect passes as OK — more critical)

I.4 Architecture 4: Agricultural IoT + AI Irrigation System

Problem: Over-watering wastes water; under-watering reduces crop yield.

Solution: AI-driven precision irrigation using multi-sensor fusion and weather API integration.

Component	Technology	Role
5x soil moisture sensors	ADC over SPI	Map moisture across field zones
Weather API (OpenWeatherMap)	HTTP REST	Forecast next 24hr rainfall
Evapotranspiration model	Python formula	Calculate water loss rate
LSTM model	PyTorch → TFLite	Predict soil moisture 6hr ahead
Rule engine	Python on Pi	Decide irrigation schedule from prediction
Solenoid valves (5-zone)	GPIO via relay	Independent per-zone irrigation
LoRa radio (SX1276)	SPI	Long-range sensor data (if field is large)
n8n workflow	Daily	Generate irrigation report, water saved stats
Claude API	Weekly	AI agronomist: "Why did Zone 3 dry out faster?"

I.5 Architecture 5: Campus Security AI Camera Network (Capstone)

Problem: Manual camera monitoring is inefficient and slow to respond.

Solution: Distributed edge AI cameras with centralised alert management.

Component	Technology	Role
3x Raspberry Pi 4 nodes	Distributed	One Pi per camera zone
Pi Camera (per node)	CSI	Video stream input
YOLOv8n TFLite	Each Pi	Person and vehicle detection at edge
MQTT broker (central Pi)	Mosquitto	Collect all detection events
DeepSORT tracking	Central Pi	Track person across camera zones
SQLite (per node)	Local	Buffer events during outage
Flask REST API (central)	Python	Query events, retrieve images
React dashboard	Browser	Live map showing detections per zone
n8n alerts	Workflow	Push notification on after-hours detection
Claude API analysis	Daily	Generate security summary report

📌 **Note:** This capstone project combines every skill from your 4-month schedule: edge AI (TFLite), IoT protocols (MQTT), web development (React + Flask), databases (SQLite), AI automation (Claude API + n8n), and Linux systems (distributed Pi network). It is a full internship portfolio on its own.

Appendix J: Production Code Library — Ready-to-Use Modules

These are complete, production-quality Python modules you can drop into any AIoT project. Copy them into your projects and adapt as needed. Every module is written with type hints, error handling, logging, and docstrings.

J.1 Sensor Reading Module — Multi-Sensor Manager

```
"""sensor_manager.py — Thread-safe multi-sensor reading manager."""
import threading, time, logging, json
from dataclasses import dataclass, asdict
from datetime import datetime
from typing import Callable, Optional
from collections import deque

logging.basicConfig(level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("SensorManager")

@dataclass
class Reading:
    sensor_id: str
    sensor_type: str
    value: float
    unit: str
    timestamp: str = ""
    quality: str = "good"

    def __post_init__(self):
        if not self.timestamp:
            self.timestamp = datetime.utcnow().isoformat() + "Z"

    def to_json(self) -> str:
        return json.dumps(asdict(self))

    def is_in_range(self, lo: float, hi: float) -> bool:
        return lo <= self.value <= hi

class SensorManager:
    """Manages multiple sensor readers in background threads."""

    def __init__(self, buffer_size: int = 1000):
        self._sensors: dict[str, dict] = {}
        self._buffer: deque[Reading] = deque(maxlen=buffer_size)
        self._lock = threading.Lock()
```

```
        self._callbacks: list[Callable[[Reading], None]] = []
        self._running = False
```

```
    def register_sensor(self, sensor_id: str, read_fn: Callable,
                        interval: float = 5.0) -> None:
        """Register a sensor with a reading function and interval."""
        self._sensors[sensor_id] = {
            "read_fn": read_fn,
            "interval": interval,
            "thread": None,
            "error_count": 0,
        }
        logger.info(f"Registered sensor: {sensor_id} (every {interval}s)")
```

```
    def add_callback(self, fn: Callable[[Reading], None]) -> None:
        """Add callback called for every new reading."""
        self._callbacks.append(fn)
```

```
    def _sensor_loop(self, sensor_id: str) -> None:
        cfg = self._sensors[sensor_id]
        while self._running:
            try:
                reading = cfg["read_fn"]()
                with self._lock:
                    self._buffer.append(reading)
                    cfg["error_count"] = 0
                for cb in self._callbacks:
                    try: cb(reading)
                    except Exception as e:
                        logger.error(f"Callback error: {e}")
            except Exception as e:
                cfg["error_count"] += 1
                reading = Reading(sensor_id, "unknown", 0.0, "", quality="bad")
                logger.warning(f"Sensor {sensor_id} error #{cfg['error_count']}:
{e}")
                if cfg["error_count"] > 5:
                    logger.error(f"Sensor {sensor_id} disabled after 5 consecutive
errors")
                    return
            time.sleep(cfg["interval"])
```

```
    def start(self) -> None:
        self._running = True
        for sid, cfg in self._sensors.items():
            t = threading.Thread(target=self._sensor_loop, args=(sid,),
daemon=True)
            cfg["thread"] = t
```

```
            t.start()
        logger.info(f"Started {len(self._sensors)} sensor(s)")
```

```
    def stop(self) -> None:
        self._running = False
        logger.info("Sensor manager stopped")
```

```
    def get_latest(self, sensor_id: Optional[str] = None) -> list[Reading]:
        """Get latest readings, optionally filtered by sensor_id."""
        with self._lock:
            if sensor_id:
                return [r for r in self._buffer if r.sensor_id == sensor_id]
            return list(self._buffer)
```

J.2 Data Logger — CSV + SQLite Dual Logger

```
"""data_logger.py — Concurrent CSV and SQLite sensor data logger."""
import csv, sqlite3, threading, logging
from pathlib import Path
from datetime import datetime
from sensor_manager import Reading
```

```
class DataLogger:
    """Thread-safe dual logger: CSV (human readable) + SQLite (queryable)."""
```

```
    def __init__(self, data_dir: str = "./data"):
        self._dir = Path(data_dir)
        self._dir.mkdir(exist_ok=True)
        self._lock = threading.Lock()
        self._csv_path = self._dir / f"sensors_{datetime.now().strftime('%Y%m
%d')}.csv"
        self._db_path = self._dir / "sensor_history.db"
        self._init_db()
        self._init_csv()
```

```
    def _init_db(self) -> None:
        with sqlite3.connect(self._db_path) as conn:
            conn.execute("""CREATE TABLE IF NOT EXISTS readings (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                sensor_id TEXT NOT NULL,
                sensor_type TEXT,
                value REAL,
                unit TEXT,
                quality TEXT,
                timestamp TEXT,
                created_at DATETIME DEFAULT CURRENT_TIMESTAMP
            )""")
```

```
            conn.execute("CREATE INDEX IF NOT EXISTS idx_sensor_time ON
readings(sensor_id, timestamp)")
```

```
    def _init_csv(self) -> None:
        if not self._csv_path.exists():
            with open(self._csv_path, "w", newline="") as f:

csv.writer(f).writerow(["timestamp","sensor_id","type","value","unit","quality"])
```

```
    def log(self, reading: Reading) -> None:
        """Log a reading to both CSV and SQLite."""
        with self._lock:
            # CSV
            with open(self._csv_path, "a", newline="") as f:
                csv.writer(f).writerow([reading.timestamp, reading.sensor_id,
                    reading.sensor_type, reading.value, reading.unit,
reading.quality])
            # SQLite
            with sqlite3.connect(self._db_path) as conn:
                conn.execute(
                    "INSERT INTO readings
(sensor_id,sensor_type,value,unit,quality,timestamp) VALUES (?,?,?,?,?,?)",
                    (reading.sensor_id, reading.sensor_type, reading.value,
                     reading.unit, reading.quality, reading.timestamp)
                )
```

```
    def query_last_n(self, sensor_id: str, n: int = 100) -> list[dict]:
        """Query last N readings for a sensor."""
        with sqlite3.connect(self._db_path) as conn:
            conn.row_factory = sqlite3.Row
            rows = conn.execute(
                "SELECT * FROM readings WHERE sensor_id=? ORDER BY created_at DESC
LIMIT ?",
                (sensor_id, n)
            ).fetchall()
        return [dict(row) for row in rows]
```

```
    def get_stats(self, sensor_id: str, hours: int = 24) -> dict:
        """Get statistical summary for the last N hours."""
        with sqlite3.connect(self._db_path) as conn:
            row = conn.execute("""
                SELECT COUNT(*) as count, AVG(value) as mean,
                       MIN(value) as min, MAX(value) as max,
                       SUM(CASE WHEN quality='bad' THEN 1 ELSE 0 END) as bad_count
                FROM readings
                WHERE sensor_id=?
                AND created_at >= datetime('now', ?)
```

```
            """, (sensor_id, f"-{hours} hours")).fetchone()
        return {"sensor_id": sensor_id, "hours": hours,
                "count": row[0], "mean": round(row[1] or 0, 3),
                "min": row[2], "max": row[3], "bad_readings": row[4]}
```

J.3 Alert Manager — Multi-Channel Alert System

```
"""alert_manager.py — Rule-based alert system with multi-channel dispatch."""
import smtplib, logging, json
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from dataclasses import dataclass
from typing import Callable
from datetime import datetime, timedelta
from sensor_manager import Reading
```

```
@dataclass
class AlertRule:
    sensor_id: str
    condition: str  # "greater_than" | "less_than" | "equal_to"
    threshold: float
    severity: str   # "info" | "warning" | "critical"
    cooldown_minutes: int = 30  # minimum minutes between alerts
```

```
class AlertManager:
    def __init__(self):
        self._rules: list[AlertRule] = []
        self._channels: list[Callable] = []
        self._last_alert: dict[str, datetime] = {}
```

```
    def add_rule(self, rule: AlertRule) -> None:
        self._rules.append(rule)
```

```
    def add_channel(self, fn: Callable) -> None:
        """Add an alert channel (email, MQTT, webhook, etc.)."""
        self._channels.append(fn)
```

```
    def evaluate(self, reading: Reading) -> None:
        """Check reading against all rules, dispatch alerts."""
        for rule in self._rules:
            if rule.sensor_id != reading.sensor_id: continue
            triggered = False
            if rule.condition == "greater_than" and reading.value > rule.threshold:
triggered = True
            elif rule.condition == "less_than" and reading.value < rule.threshold:
triggered = True
            if not triggered: continue
```

```
            # Check cooldown
            key = f"{rule.sensor_id}_{rule.condition}_{rule.threshold}"
            last = self._last_alert.get(key)
            if last and datetime.utcnow() - last <
timedelta(minutes=rule.cooldown_minutes):
                continue  # Still in cooldown
            self._last_alert[key] = datetime.utcnow()
            alert = {"sensor_id": reading.sensor_id, "severity": rule.severity,
                     "value": reading.value, "threshold": rule.threshold,
                     "timestamp": reading.timestamp}
            for ch in self._channels:
                try: ch(alert)
                except Exception as e: logging.error(f"Alert channel error: {e}")
```

```
def email_channel(smtp_host: str, smtp_port: int, username: str,
                  password: str, to_addr: str) -> Callable:
    """Factory: returns an email alert channel function."""
    def send_email(alert: dict) -> None:
        msg = MIMEMultipart()
        msg["From"] = username
        msg["To"] = to_addr
        msg["Subject"] = f"[{alert['severity'].upper()}] Sensor Alert:
{alert['sensor_id']}"
        body = f"""
        Sensor Alert Triggered
        Sensor: {alert['sensor_id']}
        Severity: {alert['severity'].upper()}
        Value: {alert['value']}
        Threshold: {alert['threshold']}
        Time: {alert['timestamp']}
        """
        msg.attach(MIMEText(body, "plain"))
        with smtplib.SMTP_SSL(smtp_host, smtp_port) as server:
            server.login(username, password)
            server.sendmail(username, to_addr, msg.as_string())
    return send_email
```

Appendix K: JLPT N4 Grammar Reference — AIoT Context

This is a complete reference of the most important JLPT N4 grammar patterns, each with an AIoT-relevant example sentence. Study one grammar point per day alongside your Anki vocabulary practice. Resource: jlptsensei.com/complete-jlpt-n4-grammar-list/

K.1 Essential N4 Grammar Patterns

Grammar Pattern	Meaning	AIoT Example Sentence	Translation
～てください	Please do ~	センサーを確認してください。	Please check the sensor.
～てもいいですか	May I ~?	このコードを変更してもいいですか。	May I change this code?
～なければなりません	Must do ~	エラーを修正しなければなりません。	We must fix the error.
～ことができます	Can do ~	ラズベリーパイで AI を実行することができます。	We can run AI on Raspberry Pi.
～ようになります	Come to be able to ~	N4 に合格できるようになります。	I will come to be able to pass N4.
～と思います	I think that ~	このモデルは正確だと思います。	I think this model is accurate.
～かもしれません	Might ~	センサーが壊れているかもしれません。	The sensor might be broken.
～のに	Even though ~	コードを書いたのに、動きません。	Even though I wrote the code, it doesn't work.
～たら	If/When ~ (completed)	推論が終わったら、結果を送ります。	When inference is done, I will send the results.
～ながら	While doing ~	データを分析しながら、レポートを書きます。	I write the report while analysing the data.
～ために	In order to ~	AIoT エンジニアになるために、毎日勉強しています。	I study every day in order to become an AIoT engineer.
～てから	After doing ~	モデルを変換してから、Pi に転送します。	After converting the model, I transfer it to the Pi.
～ていただけますか	Could you please ~? (polite)	コードをレビューしていただけますか。	Could you please review the code?
～によって	Depending on ~ / By ~	センサーの種類によって、設定が違います。	The settings differ depending on the sensor type.
～について	About ~	MQTT プロトコルについて説明します。	I will explain about the MQTT protocol.
～のために	For the sake of ~	プロジェクトのために、新しいモジュールを作りました。	I made a new module for the project.
～ば～ほど	The more ~ the more ~	練習すればするほど、上手になります。	The more you practise, the better you become.

～ようです	It seems that ~	センサーが誤動作しているようです。	It seems that the sensor is malfunctioning.
～はずです	Should be (expectation)	推論の速度は 50ms 以下のはずです。	Inference speed should be under 50ms.
～だけでなく～も	Not only ~ but also ~	Python だけでなく C++も使います。	We use not only Python but also C++.

K.2 N4 Vocabulary by AIoT Category

Japanese	Reading	Meaning	Category
開発する	かいはつする	To develop	Engineering verb
実装する	じっそうする	To implement	Engineering verb
確認する	かくにんする	To confirm, verify	Daily work verb
送信する	そうしんする	To transmit/send	IoT verb
受信する	じゅしんする	To receive	IoT verb
接続する	せつぞくする	To connect	IoT verb
変換する	へんかんする	To convert	AI/ML verb
測定する	そくていする	To measure	Sensor verb
検出する	けんしゅつする	To detect	AI verb
処理する	しょりする	To process	Computing verb
精度	せいど	Accuracy, precision	ML term
遅延	ちえん	Latency, delay	Performance term
消費電力	しょうひでんりょく	Power consumption	Hardware term
温度	おんど	Temperature	Sensor type
湿度	しつど	Humidity	Sensor type
圧力	あつりょく	Pressure	Sensor type
電流	でんりゅう	Electric current	EEE term
電圧	でんあつ	Voltage	EEE term
周波数	しゅうはすう	Frequency	Signal term
誤差	ごさ	Error, deviation	Measurement term
閾値	しきいち	Threshold	AIoT term
警告	けいこく	Warning, alert	System status
障害	しょうがい	Fault, failure	System status
修正する	しゅうせいする	To fix, correct	Engineering verb
最適化	さいてきか	Optimisation	ML/performance term

K.3 Japanese Number and Unit System for Engineers

Value	Japanese	Reading	Usage Example
1 millisecond	1 ミリ秒	いちみりびょう	Inference latency: 20 ミリ秒

1 gigabyte	1 ギガバイト	いちぎがばいと	Model size: 200 メガバイト
1 megahertz	1 メガヘルツ	いちめがへるつ	Frequency: 1.8 ギガヘルツ
30 frames/sec	毎秒 30 フレーム	まいびょうさんじゅうふれーむ	Camera: 毎秒 30 フレーム
95% accuracy	正確率 95%	せいかくりつきゅうじゅうごぱーせんと	Model accuracy: 97%
5 volts	5 ボルト	ごぼると	Pi power: 5 ボルト 3 アンペア
100 meters	100 メートル	ひゃくめーとる	LoRa range: 1 キロメートル

Appendix L: Google Summer of Code (GSoC) Strategy

GSoC is a global program where students contribute to open-source projects for a stipend. For your profile — Python, ML, IoT, Edge AI — there are strong matching organizations every year. This appendix gives you the complete strategy to apply and win.

L.1 GSoC 2027 Timeline (your realistic target)

Phase	Timeline	What To Do
Explore organizations	October 2026	Browse last year's accepted orgs: summerofcode.withgoogle.com
Choose 2-3 target orgs	November 2026	Pick ones matching Python + ML + IoT
Start contributing	December 2026	Fix bugs, improve docs — build relationship with mentors
Build 5+ PRs merged	Jan–Feb 2027	Meaningful contributions, not typo fixes
Talk to mentors	February 2027	Join Discord/mailing list, introduce yourself
Proposal writing	March 2027	Draft with mentor feedback before submission
Applications open	March 18, 2027	Submit polished, specific, timeline-detailed proposal
Selection announced	May 2027	Results — if selected, start June
Coding period	June–September 2027	Code + weekly reports + midterm evaluation
Final evaluation	September 2027	Submit final work + presentation

L.2 Best Organizations for Your Profile

Organization	Why It Fits You	Typical Projects	Stipend Range
TensorFlow / TFLite	You use TFLite on Pi — direct experience	Model optimization, mobile deployment, benchmarks	Medium ($3000)
Python Software Foundation	Python is your primary language	stdlib improvements, packaging, tools	Small/Medium
NumFOCUS	NumPy/SciPy ecosystem	Scientific computing, array ops, documentation	Medium
OpenCV	You use OpenCV on Pi	Python bindings, mobile, DNN module	Medium
Arduino	Your IoT hardware background	Libraries, IDE improvements, sensor code	Small/Medium
SymPy / Scipy	Math foundations for ML	Algorithms, solvers, documentation	Medium
Rocket.Chat / Matrix	MERN skills apply	Bot development, API features	Medium

L.3 The Winning GSoC Proposal Structure

Section 1: Abstract (150 words)

Clearly state: what you will build, which organization, why it matters, and your qualifications in 3 sentences. Mentors read hundreds of proposals — the abstract must hook them immediately.

Section 2: About Me (200 words)

Mention: B.Tech CS-IT + EEE diploma (the hardware background is unique), your specific projects with GitHub links (Pi TFLite pipeline, sensor dashboard), your prior contributions to THIS organization (critical — mention specific PR numbers), and why you chose this org and project specifically.

Section 3: Technical Design (500-800 words)

This is the most important section. Describe exactly what you will build: the architecture, the algorithms, the APIs, the test strategy. Show that you have already thought through the hard problems. Include a block diagram if possible. Reference specific files in the codebase you will modify.

Section 4: Weekly Timeline (required)

Write a week-by-week plan for the 12-week coding period. Be specific: not "implement the feature" but "implement the INT8 quantisation path for Conv2D layers in TFLite converter (files: lite/python/lite.py, tensorflow/compiler/mlir/lite/...)". Mentors want to see that you understand the scope.

Section 5: Why You Will Complete It

Your availability (hours per week), your time zone (IST — mention this explicitly), any prior commitments, and how you handle blockers. The biggest GSoC failure mode is students disappearing in week 3 — mentors want evidence you will not do this.

> 📌 **Note:** The single highest-ROI action for GSoC is having a merged PR before the application deadline. A proposal from someone with 3 merged PRs beats a better-written proposal from someone with 0 contributions. Start contributing in Month 3 of your schedule (November 2026).

L.4 GSoC Stipend Details for India (2026)

Project Size	Duration	India Stipend (USD)	India Stipend (INR approx)
Small	~90 hours	$1,500	~₹1.25 lakh
Medium	~175 hours	$3,000	~₹2.5 lakh
Large	~350 hours	$6,000	~₹5 lakh

Payment: 45% on passing midterm evaluation (Week 6), 55% on passing final evaluation (Week 12). Both evaluations require: working code, documentation, and mentor approval.

> 🔗 **GSoC Official Site:** summerofcode.withgoogle.com
> 🔗 **GSoC Student Guide:** google.github.io/gsocguides/student
> 🔗 **How to Write a Good GSoC Proposal (blog):** google.github.io/gsocguides/student/writing-a-proposal

Appendix M: Vizag Startup Planning Guide — 2033

This appendix is your pre-planning document for the Indo-Japan consultancy and machine company you will start after returning from Japan. Read it now to understand what decisions you should be making during your Japan years that will make the startup launch smoother.

M.1 Business 1 — India-Japan Consultancy

Business Model:

- Japanese language training: N5 → N2 courses, online and offline in Vizag
- Japan job placement: recruit Indian engineers for Japanese companies (10-15% CTC commission)
- Cultural orientation: pre-departure training for engineers going to Japan
- IoT/AIoT technical training: short courses on Raspberry Pi, Arduino, Edge AI, TFLite

Revenue Model (Year 1 projections):

Revenue Stream	Volume	Price	Annual Revenue
Japanese language courses	50 students/year	₹20,000/course	₹10 lakh
Japan placement commissions	5 placements/year	₹2 lakh/placement	₹10 lakh
IoT technical workshops	10 workshops/year	₹15,000/participant × 10	₹15 lakh
Corporate training (B2B)	2 companies/year	₹1 lakh/training	₹2 lakh
Total Year 1			~₹37 lakh gross

Startup Costs (one-time):

Item	Cost
Office/training space setup (Vizag)	₹4-6 lakh
Equipment: Pi kits, Arduino sets, projectors, screens	₹2-3 lakh
Company registration + legal + CA fees	₹50,000-1 lakh
Website, CRM, marketing materials	₹1-2 lakh
Initial marketing (LinkedIn ads, Google Ads)	₹1 lakh
3-month operating reserve	₹3-4 lakh
Total	~₹12-17 lakh

M.2 Business 2 — Indo-Japan Machines Company

Launch this after the consultancy is profitable (target: Year 2-3 after return, i.e. 2035-2036).

Product Line:

- Edge AI quality inspection systems for SME manufacturers (camera + Pi + TFLite + dashboard)
- Predictive maintenance units for motors and compressors (vibration + thermal + AI)
- Smart energy monitoring systems for factories
- Agricultural IoT kits for Andhra Pradesh farmers (soil + weather + AI irrigation)

Japan Network Advantage:

- Source precision Japanese sensors and components directly — 30-40% cheaper than Indian distributors
- Japanese quality certification on components = premium brand positioning in India
- Technology transfer agreements with ex-colleagues at Japanese companies
- JETRO (Japan External Trade Organization) India office can connect you to Japanese SMEs

M.3 What to Do During Japan Years (2029-2033) to Prepare

Year in Japan	Startup Prep Action	Why It Matters
Year 1 (2029)	Document every project, every technology decision	Your curricula and case studies for training
Year 1 (2029)	Start a Hindi/Telugu YouTube channel about Japan life + tech	Build audience before you return
Year 2 (2030)	Identify 2-3 Japanese component suppliers willing to export to India	Supply chain for machine company
Year 2 (2030)	Connect with 5 Indian engineers in Japan — potential future partners	Network for placement pipeline
Year 3 (2031)	Research Vizag startup ecosystem: T-Hub, AP Innovation Society, MSME schemes	Funding and support options
Year 3 (2031)	Register company from Japan via Indian CA	Can be done remotely — do it early
Year 3 (2032)	Validate consultancy demand: find 3 paying clients before you leave Japan	Proof of market before launch
Year 4 (2032)	Scout Vizag training space, negotiate lease virtually	Ready to start immediately on return

M.4 Government Support Available for Your Startup

Scheme	Benefit	Eligibility
Startup India (Central govt)	Tax exemption 3 years, fast-track patents, funding connections	DPIIT registered startup
AP Innovation Society	Incubation support, grants up to ₹10 lakh, mentorship	Andhra Pradesh based startup
MSME Udyam Registration	Priority sector lending, subsidy on equipment, govt contracts	Small manufacturing unit
Stand-Up India	Bank loan ₹10L-1Cr for SC/ST or women entrepreneurs	Scheduled caste entrepreneurs
SIDBI SMILE scheme	Soft loan for equipment purchase at concessional rate	Manufacturing MSME
JETRO India support	Connect with Japanese companies, Japan market entry help	Indo-Japan business focus

📌 **Note:** Register under Startup India (startupindia.gov.in) immediately on return — the 3-year income tax exemption alone can save ₹5-10 lakh in your first profitable years. This is free to apply and takes 2 weeks to process.

End of AIoT Engineer Master Study Guide

Chapters: 12 | Appendices: A–M | Pages: 100+

Your path: EEE Diploma → B.Tech CS-IT → Remote Internship → Japan (2029) → Vizag Startup (2033)

AIoT ENGINEER
200-PAGE MASTER GUIDE — VOLUME 2

Future AI Demand · 10 Big Projects · Simple Language · Grammar Guide · Career Prep

---

Written for a Telugu-speaking B.Tech CS-IT student with EEE diploma background.
Every concept explained simply. Every project explained step-by-step.

Sections in this volume:

Section 1 — Future AI: What is coming and how to prepare
Section 2 — AI Tools (Cursor, Claude API, n8n, LangChain) — Super Simple
Section 3 — 10 Complete Projects with Code and Circuit Diagrams
Section 4 — Grammar Guide for Technical Writing
Section 5 — Edge AI and Embedded Systems — Explained Simply
Section 6 — Japanese N4 Practice + Career and Study Habits

🤖 Section 1: Future AI — What Is Coming and How To Prepare

This section explains every major AI technology that will dominate engineering jobs between 2026 and 2035. Written in simple language with real examples. No jargon without explanation.

Chapter 1: Understanding Today's AI Landscape

Before we talk about future AI, we need to clearly understand where we are today. Many students are confused because they hear about ChatGPT, Edge AI, Autonomous Vehicles, and Robots all at once. Let us organise this properly.

> 📌 **NOTE:** Think of AI like a family tree. There is one grandfather (Machine Learning), and many children: Deep Learning, Computer Vision, NLP, Reinforcement Learning. Each child has its own grandchildren. Edge AI is one grandchild. Autonomous Vehicles is another.

1.1 The AI Family Tree — Explained Simply

Imagine AI as a big tree. The roots are mathematics — statistics, linear algebra, calculus. The trunk is Machine Learning. The main branches are:

- Branch 1 — Deep Learning: Teaching computers using neural networks (like a simplified brain)
- Branch 2 — Computer Vision: Teaching computers to see and understand images and video
- Branch 3 — Natural Language Processing (NLP): Teaching computers to read and write human language
- Branch 4 — Reinforcement Learning: Teaching computers by reward and punishment (how AlphaGo works)
- Branch 5 — Robotics AI: Combining all the above to make physical machines act smart

Your career — AIoT and Edge AI — sits at the intersection of Deep Learning + Computer Vision + Robotics AI. This is why you study PyTorch (for Deep Learning) and OpenCV (for Computer Vision) and Raspberry Pi (for the physical hardware layer).

> 🔄 **ANALOGY — AI Tree:** Just as a mango tree needs good roots (soil, water) to produce fruit, AI needs good mathematics foundations (statistics, linear algebra) to produce useful applications like face recognition, voice assistants, and anomaly detection.

1.2 What Has Changed Between 2020 and 2026

Year	What Happened	Impact on You
2020	GPT-3 launched — AI could write human-quality text	NLP became practical for everyday developers
2021	DALL-E launched — AI could generate images from text	Creative AI became real

2022	ChatGPT launched — 100M users in 60 days	Every company started using AI
2023	GPT-4, Claude 2 — multimodal AI (text + image)	AI could now see AND read at the same time
2024	Llama 3, Gemini — AI models that run on laptops	Edge AI LLMs became possible
2025	AI Agents — AI that takes actions, not just answers	Automation of complex workflows
2026	Agentic pipelines everywhere — MCP standard adopted	Your current year — opportunity is now

1.3 The Six AI Technologies That Will Dominate 2026–2035

These are the technologies that will create the most engineering jobs and the highest salaries in the next decade. We will explain each one simply, then show how it connects to your career.

Technology 1: AI Agents

What is it? An AI Agent is a program that uses an AI model (like Claude or GPT-4) as its "brain" to plan and execute multi-step tasks automatically.

> 🔄 **ANALOGY — Manager-Employee:** Normal AI is like an employee who answers one question at a time. An AI Agent is like a manager who receives a big goal, breaks it into subtasks, assigns each subtask, checks the results, and handles problems — all automatically.

Simple example: You tell an AI Agent "Monitor my factory sensors and send me a WhatsApp alert if anything goes wrong." The agent:

- Reads sensor data every 5 seconds (Tool: read_sensor())
- Checks if any value is out of normal range (Tool: check_threshold())
- If yes, creates a message explaining what is wrong (AI reasoning)
- Sends you a WhatsApp alert with the explanation (Tool: send_whatsapp())
- Also logs the event to the database (Tool: write_to_db())

This is exactly what you will build using LangChain + Claude API + MQTT in your projects.

> 💡 **TIP:** AI Agents are the most in-demand skill right now. In India, a developer who can build production-grade AI agents earns 3-5x more than a regular developer. In Japan, companies like Fujitsu and NEC are actively hiring "AI Engineer" roles that are 80% about agents.

▶️ **YouTube — LangChain Agents Tutorial — Greg Kamradt:** youtube.com/watch?v=DwATCOaB8XM
▶️ **YouTube — Building AI Agents — Fireship:** youtube.com/watch?v=sal78ACtGTc

Technology 2: Multimodal AI

What is it? Multimodal AI can understand and generate multiple types of data — text, images, audio, video, and sensor readings — all at the same time.

> 🔄 **ANALOGY — Five Senses:** Traditional AI could only use one sense at a time — either sight (image AI) or hearing (speech AI) or reading (text AI). Multimodal AI has all five senses working together, like a human.

Why it matters for AIoT: Your Raspberry Pi + Camera system currently does image classification. With multimodal AI, you can:

- Look at a machine (camera) AND listen to it (microphone) AND measure it (vibration sensor) simultaneously
- Ask in natural language: "What is wrong with this motor?" and get an answer that uses all three data sources
- Generate a report with images, charts, and text automatically

Real product example: A Japanese food factory uses multimodal AI to inspect food products. The system looks at the product (camera), smells it (chemical sensor), measures weight (load cell), and checks temperature — all at once — to detect spoilage. This is a real job you could do.

> 📌 **NOTE:** Claude (the AI you are talking to right now) is multimodal. You can send it images and text together. This is why you are learning the Claude API — you will build multimodal AIoT applications.

Technology 3: Small Language Models (SLMs) for Edge

What is it? Until 2024, powerful AI models needed massive cloud servers to run. In 2025-2026, small, highly efficient models (Phi-3-mini, Llama 3.2, Gemma 2B) became good enough to run on a Raspberry Pi or even a smartphone.

> 🔄 **ANALOGY — Engine Size:** A large language model (GPT-4) is like a V8 truck engine — powerful but needs a lot of fuel (computing power) and is too big for small vehicles. A Small Language Model is like a motorcycle engine — efficient, small, and perfect for tight spaces like a Raspberry Pi.

Model	Size	Runs On	Use Case in AIoT
Phi-3-mini	3.8B params	Raspberry Pi 4 (4GB)	Natural language sensor query interface
Llama 3.2 (1B)	1B params	Raspberry Pi 4 (2GB)	Simple anomaly explanation in English
Gemma 2B	2B params	Raspberry Pi 5	Edge chatbot for factory workers
TinyLlama	1.1B params	ESP32 (with quantisation)	Ultra-low power NLP on microcontroller

Whisper-tiny	39M params	Raspberry Pi 4	Voice commands for IoT control

This is the future you are preparing for. By B.Tech Year 3 (2027), you will be running small language models directly on your Raspberry Pi, allowing factory workers to ask questions in natural language and get answers from the edge device — without internet.

▶️ **YouTube — Running Llama on Raspberry Pi — Jeff Geerling:** youtube.com/watch?v=BO7DVnN9ctk

Technology 4: Neuromorphic Computing

What is it? Normal computers process information step by step, one calculation at a time (like reading a book word by word). Neuromorphic chips process information like a human brain — many things simultaneously, using spikes of energy rather than continuous calculation.

🔄 **ANALOGY — City Traffic:** Normal computing is like a single-lane road — cars (data) must go one by one. Neuromorphic computing is like a city with thousands of roads all active simultaneously — much more efficient for complex patterns.

Real chips available now:

- Intel Loihi 2 — research neuromorphic chip (not yet mainstream)
- BrainScaleS (Europe) — brain-inspired computing research
- SpiNNaker (UK) — used for robotics brain simulation

Why this matters for your career: Neuromorphic computing is 10 years away from mainstream use, but it will revolutionise Edge AI because it uses 1000x less power than current chips. The first industrial applications will be in Japanese manufacturing (their specialty). Engineers who understand both traditional Edge AI AND neuromorphic concepts will command premium salaries.

💡 **TIP:** You do not need to learn neuromorphic programming now. Just understand the concept and follow Intel Loihi research news. Mentioning awareness of neuromorphic computing in a Japan job interview will genuinely impress the interviewer.

Technology 5: Quantum Machine Learning (QML)

What is it? Quantum computers use quantum mechanics (particles existing in multiple states simultaneously) to solve certain problems exponentially faster than regular computers. Quantum Machine Learning applies this to AI.

🔄 **ANALOGY — Lock Combinations:** A regular computer trying all lock combinations tries them one by one: 0001, 0002, 0003... A quantum computer tries ALL combinations simultaneously and finds the answer instantly. For certain AI optimization problems, this is transformative.

Current reality (2026): Quantum computers are real but noisy and limited to specific problems. Google, IBM, and several Japanese companies (Fujitsu, NTT) are investing heavily. Practical quantum advantage for ML is estimated 5-10 years away.

What to do now: Learn the concept, follow news from IBM Quantum (quantum.ibm.com) and Google Quantum AI. In Year 4 of B.Tech (2028), consider exploring Qiskit (IBM's quantum SDK) as a bonus skill. By 2033, when you start your Vizag company, quantum ML consulting may be a service you offer.

> 🔗 **IBM Quantum Learning (Free):** learning.quantum.ibm.com

Technology 6: Embodied AI and Physical AI

What is it? Embodied AI combines language models with physical robots. The robot can understand natural language instructions and translate them into physical actions. This is what Figure, Boston Dynamics, and 1X Technologies are building.

> 🔄 **ANALOGY — Reading vs Doing:** Old AI was like a person who read every book about swimming but never entered water. Embodied AI is like giving that person a body and putting them in water — now they can actually swim and learn from the physical experience.

Why this is your biggest career opportunity: Japan has the world's largest industrial robotics industry. Companies like Fanuc, Yaskawa, Kawasaki, and DENSO are ALL investing in embodied AI. Engineers who understand both AIoT (software) and embedded systems (hardware, your EEE diploma) are exactly what they need.

> 🎯 **REMEMBER:** Your EEE diploma makes you rare in this field. Most robotics AI engineers are either pure software (no hardware understanding) or pure mechanical (no AI). You are both. This is your unique career asset.

Company	Country	What They Make	Why They Need You
Fanuc	Japan	CNC machines, industrial robots	Edge AI for machine fault detection
DENSO	Japan	Automotive components, robots	Vision AI for assembly inspection
Boston Dynamics	USA (in Japan too)	Spot, Atlas robots	Sensor fusion, real-time control
Figure	USA	Humanoid robots	AI-hardware interface engineering
Unitree	China	Quadruped robots	Affordable embodied AI research
Omron	Japan	Factory automation, sensors	AIoT system integration

Chapter 2: AI Demand Roadmap — What to Learn and When

Now that you understand the six key technologies, let us map them to your specific timeline. This is your learning roadmap for the next 9 years, with each skill timed to when the job market will need it most.

2.1 The 2025–2033 AI Skill Timeline

Year	Your Stage	AI Skills to Learn	Market Demand
2025-2026	B.Tech Year 1-2	Python, PyTorch, TFLite, MQTT, AI Agents (n8n, Claude API)	High — these skills are needed NOW
2027-2028	B.Tech Year 3-4	Edge LLMs (Phi-3), Multimodal AI, ROS2, Sensor Fusion	Very High — agents + embodied AI exploding
2029-2030	Japan Year 1-2	OPC-UA, Industrial AI, Japanese AI tools, Neuromorphic awareness	Extreme — Japan factories need this
2031-2032	Japan Year 3-4	Quantum ML basics, Advanced robotics AI, Patent applications	High — differentiation for senior roles
2033+	Vizag Startup	All above + Business AI (n8n for SME automation, AI consulting)	Building your own demand

2.2 What "Agentic AI" Really Means For Your Work (Simple Explanation)

You will hear the word "agentic" everywhere in 2026. Let us understand it simply.

OLD WAY (non-agentic):

- You write code that does exactly what you tell it
- If sensor reads 45°C, send email — you program every single rule
- When a new situation happens that you did not program, the system fails

NEW WAY (agentic AI):

- You give the AI a goal: "Keep the factory running safely"
- The AI decides HOW to achieve that goal using available tools
- When a new situation happens, the AI reasons about what to do
- The AI can learn from past events to handle new ones better

🔄 **ANALOGY — Security Guard:** Non-agentic AI is like giving a security guard a list of rules: "If door opens at night, call police. If fire alarm sounds, call fire station." They only follow the list. Agentic AI is like hiring an experienced security guard who uses judgment. When something new happens (a suspicious smell), they figure out the best response without needing a pre-written rule for it.

In your AIoT context, agentic AI means:

- Instead of hard-coded threshold rules (if temp > 40, alert), you have an AI that understands the CONTEXT
- The AI knows: "This machine normally runs hot after noon because of sun exposure — 42°C is fine here"
- The AI knows: "But if this is a Tuesday and cooling was just serviced, 42°C is suspicious"
- This contextual reasoning is what makes agentic AI so powerful and so hard to replicate with simple rules

📌 **NOTE:** This is why LangChain memory matters so much. Memory gives the agent CONTEXT. Without memory, every sensor reading is isolated. With memory, the agent understands history and patterns — like an experienced human engineer would.

2.3 How to Stay Updated on AI Without Getting Overwhelmed

AI moves very fast. New models, new tools, new papers come every week. Many students feel anxious — "How do I keep up?" Here is the realistic strategy.

Rule 1: Read 3 sources only, daily for 10 minutes

- X/Twitter: Follow @AnthropicAI, @OpenAI, @GoogleDeepMind, @sama (Sam Altman), @karpathy (Andrej Karpathy)
- Newsletter: "TLDR AI" — tldr.tech/ai — 5-minute daily AI summary, free
- YouTube: Watch Fireship (3-min tech explainers) — youtube.com/c/fireship

Rule 2: Evaluate every new tool with 3 questions

- Question 1: Does this help my AIoT domain specifically? (if no, ignore it)
- Question 2: Is it stable enough to learn or still experimental? (if experimental, bookmark for 6 months)
- Question 3: Is there a free tier to try? (if paid with no trial, skip for now)

Rule 3: Learn by building, not by watching

Every new AI tool you hear about — try to build ONE small project with it before deciding if it is worth learning deeply. A 2-hour Sunday experiment tells you more than 5 hours of watching tutorials.

💡 **TIP:** The biggest career mistake engineering students make is "tutorial hopping" — watching tutorials endlessly without building. Tutorials give you knowledge. Projects give you skills. Recruiters hire people with projects, not with knowledge.

Chapter 3: AI Tools Guide — Super Simple Language

🛠️ AI Tools — Explained Like You Are 12 Years Old

Every tool in this chapter is explained with a simple analogy first, then how to use it, then a real AIoT project example. No assumption of prior knowledge beyond what you already know.

3.1 Cursor IDE — Your AI Coding Partner

What is Cursor? (Simple Explanation)

Imagine you have a very smart friend sitting next to you while you code. This friend:

- Has read every programming tutorial ever written
- Can write code as fast as you can describe it
- Understands your entire project, not just one file
- Catches your mistakes and explains why they are wrong
- Never gets tired or frustrated

That friend is Cursor IDE. It is a code editor (like VS Code) but with an AI built inside it.

> 🔄 **ANALOGY — Autocomplete on Steroids:** You know how your phone autocompletes your typing — "Hello, how are" → suggests "you?". Cursor is like that, but instead of completing one word, it completes entire functions, files, and even entire programs based on your description.

Three Main Ways to Use Cursor

Way 1 — Chat (Ctrl+L): Ask questions about your code

- Example: Select a confusing function, press Ctrl+L, type "What does this function do?"
- Cursor reads your whole project and explains it clearly
- Use this: When you inherit someone else's code or want to understand a library

Way 2 — Inline Edit (Ctrl+K): Change selected code

- Example: Select a function, press Ctrl+K, type "Add error handling if the sensor reading is None"
- Cursor rewrites just that section — rest of your code is untouched
- Use this: When you want to improve or fix a specific part

Way 3 — Composer (Ctrl+I): Build entire features

- Example: Press Ctrl+I, type "Create a complete MQTT publisher class for DHT22 sensor with reconnection logic"
- Cursor creates a new file or edits multiple files to implement the whole feature
- Use this: When starting a new module or feature from scratch

> ⚠️ **IMPORTANT:** Cursor writes code fast but not always perfectly. ALWAYS read what it generates before running it. If you cannot read the code, you cannot debug it when something goes wrong. This is why DSA and Python fundamentals still matter enormously.

Cursor Rules — Teaching Cursor Your Style

Cursor Rules are like standing instructions to your AI friend. Instead of repeating yourself every time, you write the rules once and Cursor follows them forever.

Create a file called .cursorrules in your project root folder. Here is the template for your AIoT projects:

```
# .cursorrules for AIoT Projects

You are an expert AIoT engineer. Follow these rules always:

LANGUAGE: Python 3.11+ only. Use type hints on every function.
HARDWARE: Code must run on Raspberry Pi 4 with 4GB RAM.
AI FRAMEWORK: Use TFLite for inference, never full TensorFlow.
MQTT: Always use paho-mqtt library. Always handle reconnection.
ERRORS: Every function must have try/except with logging.
LOGGING: Use Python logging module, not print() statements.
DOCSTRINGS: Every class and function needs a one-line docstring.
NAMING: snake_case for functions/variables, PascalCase for classes.
COMMENTS: Explain WHY, not WHAT. Code explains what. Comments explain why.
TESTS: When asked to implement a feature, also write a basic unit test.
```

With these rules set, every time you ask Cursor to write code, it automatically follows your style. No need to repeat yourself.

> ▶️ **YouTube — Cursor IDE Complete Beginner Guide (Tech With Tim):** youtube.com/watch?v=8AWEPx5cHWQ
> ▶️ **YouTube — Cursor AI Rules Masterclass (ByteGrad):** youtube.com/watch?v=IMseojNtxkk

3.2 Claude API — Adding AI Brain to Your Projects

What is the Claude API? (Simple Explanation)

An API (Application Programming Interface) is a way for your program to talk to another service.

> 🔄 **ANALOGY — Restaurant Order System:** When you go to a restaurant, you do not go into the kitchen to cook. You tell a waiter what you want, the waiter goes to the kitchen, the kitchen prepares it, the waiter brings it back. The waiter is the API. Claude is the kitchen. Your Python code is you.

The Claude API lets your Python program send text to Claude, and Claude sends back an intelligent response. You can use this to:

- Analyse sensor data in plain English
- Generate reports automatically

- Answer questions about your IoT system
- Detect patterns and explain them
- Control your system using natural language commands

Your First Claude API Call — Step by Step

Step 1: **Get API Key —** Go to console.anthropic.com. Sign up for free. Click "API Keys". Create a new key. Copy it.

Step 2: **Install Library —** Open terminal and type: pip install anthropic python-dotenv

Step 3: **Create .env File —** In your project folder, create a file called .env and write: ANTHROPIC_API_KEY=your-key-here

Step 4: **Write the Code —** Create a new file called test_claude.py and paste the code below

Step 5: **Run It —** In terminal, type: python test_claude.py

```
# test_claude.py — Your first Claude API call
import anthropic          # the Claude library
from dotenv import load_dotenv  # reads your .env file

load_dotenv()  # this loads your API key from .env file

# Create a Claude client (this is like opening a connection to Claude)
client = anthropic.Anthropic()

# Send a message and get a response
response = client.messages.create(
    model="claude-sonnet-4-6",  # which Claude model to use
    max_tokens=500,              # maximum length of reply
    messages=[
        {
            "role": "user",
            "content": "Explain what MQTT is in 3 simple sentences."
        }
    ]
)

# Print the response
print(response.content[0].text)
```

Expected output: Claude will explain MQTT in 3 sentences. That is your first AI API call. Congratulations.

System Prompts — Giving Claude a Personality and Role

A system prompt is like your briefing to Claude before the conversation starts. You tell Claude who it is and what it should do.

Without system prompt: Claude behaves like a general assistant.

With system prompt: Claude behaves exactly like a specialized AIoT engineer.

```
# with_system_prompt.py
import anthropic
from dotenv import load_dotenv
load_dotenv()

client = anthropic.Anthropic()

# The system prompt defines Claude's role
system_prompt = """
You are an expert AIoT engineer with 10 years of experience in
industrial sensor systems and edge AI deployment on Raspberry Pi.

When someone gives you sensor readings:
1. State if values are normal or abnormal
2. Give the technical reason in simple language
3. Suggest one immediate action
4. Always respond in 3-4 sentences maximum
5. Always mention confidence level: High / Medium / Low
"""

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    system=system_prompt,   # <-- system prompt goes here
    messages=[{
        "role": "user",
        "content": "Temperature: 52°C, Humidity: 78%, Vibration: 9.2g"
    }]
)

print(response.content[0].text)
```

💡 **TIP:** The system prompt is the most powerful part of the Claude API. A good system prompt transforms a general AI into a domain expert. Write different system prompts for different tasks: one for sensor analysis, one for report writing, one for Japanese translation.

3.3 n8n — Building Automations Without Hard Coding

What is n8n? (Simple Explanation)

n8n (pronounced "n-eight-n") is a tool that connects different applications and services together using a visual diagram — no complex coding needed for the connections.

> 🔄 **ANALOGY — Train Tracks:** Think of your different applications as train stations: MQTT broker, email, WhatsApp, database, Claude AI. n8n builds the train tracks between them and decides which train (data) goes where and when. You draw the tracks visually, then n8n runs the trains automatically.

Comparison with writing code manually:

Task	Code it Yourself	Using n8n
Read MQTT + save to DB	50-100 lines Python	5 minutes drag-and-drop
Email alert on anomaly	Email library + error handling	Select Gmail node, type message
Daily sensor report	Cron + pandas + email code	Schedule node + Claude node + Gmail
WhatsApp alert	Twilio SDK + webhook setup	WhatsApp node, connect to trigger
Restart if service crashes	Bash script + systemd	n8n error trigger node

> 📌 **NOTE:** n8n does NOT replace Python programming. You still need Python for your TFLite inference, sensor reading, and data processing. n8n is for CONNECTING things together — glue between your Python scripts and the outside world.

Installing n8n on Your Raspberry Pi — Step by Step

Step 1: Check Node.js version — In terminal: node --version. You need v20+. If not: curl -fsSL https://deb.nodesource.com/setup_20.x | sudo bash - && sudo apt install -y nodejs

Step 2: Install n8n globally — In terminal: sudo npm install -g n8n

Step 3: Start n8n — In terminal: n8n start

Step 4: Open in browser — On your laptop, go to: http://[your-pi-ip]:5678

Step 5: Create account — n8n asks you to create a local account — use any email and password

Step 6: Make it permanent — Create a systemd service so n8n starts automatically when Pi boots

```
# Create systemd service for n8n
sudo nano /etc/systemd/system/n8n.service

# Paste this content:
[Unit]
Description=n8n Workflow Automation
After=network.target

[Service]
```

```
Type=simple
User=pi
ExecStart=/usr/bin/n8n start
Restart=always
RestartSec=10
Environment=N8N_PORT=5678

[Install]
WantedBy=multi-user.target

# Enable and start
sudo systemctl daemon-reload
sudo systemctl enable n8n
sudo systemctl start n8n
```

Your First n8n Workflow — MQTT to Email Alert

This workflow does: When a sensor sends a temperature above 40°C via MQTT, send an email alert.

- Node 1 — MQTT Trigger: Click "Add node" → search "MQTT Trigger" → set Topic to "sensors/#" → set Host to localhost (or Pi IP)
- Node 2 — IF node: Add condition: {{$json.temperature}} > 40
- Node 3 — Gmail node (true branch): Set To, Subject ("ALERT: Temperature High"), Body: "Temperature is {{$json.temperature}}°C at {{$json.timestamp}}"
- Node 4 — No Operation node (false branch): Do nothing when temperature is normal

Click "Execute Workflow" to test. When your Pi publishes a temperature above 40°C, the email arrives automatically. This is your first automation.

▶️ **YouTube — n8n Full Beginner Tutorial (Official n8n YouTube):** youtube.com/@n8n-io
▶️ **YouTube — n8n AI Automation Crash Course:** youtube.com/watch?v=qqqAePVAOKo

3.4 LangChain — Building Smart AI Systems

What is LangChain? (Simple Explanation)

LangChain is a Python library that makes it easier to build applications where AI models need to:

- Remember previous conversations (Memory)
- Use external tools like databases and APIs (Tools)
- Execute a series of steps automatically (Chains and Agents)
- Find relevant information from your own documents (RAG)

🔄 **ANALOGY — Orchestra Conductor:** Claude API alone is like one talented musician. LangChain is the conductor who coordinates multiple musicians (Claude, databases, APIs, sensors) to play together in harmony, following a complex musical score (your workflow).

LangChain vs Direct Claude API — When to Use What

Situation	Use This	Why
Simple question-answer	Claude API directly	Less code, faster, simpler
Need to remember past messages	LangChain (memory)	Built-in conversation history management
AI needs to call your Python functions	LangChain (tools)	Easy tool definition with @tool decorator
AI needs to search your documents	LangChain (RAG + ChromaDB)	Document indexing and retrieval built in
Complex multi-step workflow	LangChain (agents)	Agent handles planning automatically
Simple batch processing	Claude API in a loop	No need for LangChain overhead

Your First LangChain AI Tool — Sensor Query Agent

This is the smallest possible LangChain agent that does something useful for AIoT.

```
# langchain_sensor_agent.py
from langchain_anthropic import ChatAnthropic
from langchain.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.prompts import ChatPromptTemplate
import random  # we use random to simulate sensor readings

# ── STEP 1: Define tools the agent can use ──
@tool
def get_temperature(zone: str) -> str:
    """Get current temperature for a given factory zone."""
    # In real life, this reads from your database or MQTT
    temp = round(random.uniform(20, 55), 1)
    return f"Zone {zone} temperature: {temp}°C"

@tool
def get_vibration(machine_id: str) -> str:
    """Get current vibration level for a machine."""
    vibration = round(random.uniform(0.5, 12.0), 2)
    return f"Machine {machine_id} vibration: {vibration}g"

@tool
def is_machine_running(machine_id: str) -> str:
```

```
    """Check if a machine is currently running."""
    status = random.choice(["running", "idle", "fault"])
    return f"Machine {machine_id} status: {status}"

# ── STEP 2: Create the AI model ──
llm = ChatAnthropic(model="claude-sonnet-4-6")
tools = [get_temperature, get_vibration, is_machine_running]

# ── STEP 3: Create the prompt ──
prompt = ChatPromptTemplate.from_messages([
    ("system", """You are a factory floor AI assistant.
    Use the available tools to answer questions about machines and sensors.
    Always check the actual data before answering. Be concise."""),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

# ── STEP 4: Create the agent ──
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# ── STEP 5: Ask the agent a question ──
print("Factory AI Agent ready. Ask a question:")
while True:
    question = input("You: ")
    if question.lower() in ["exit", "quit"]: break
    result = executor.invoke({"input": question})
    print(f"Agent: {result['output']}\n")
```

Try asking: "Is Machine A5 running and what is the temperature in Zone 3?" Watch how the agent decides which tools to call and in what order. This is agentic reasoning.

▶️ **YouTube — LangChain Crash Course 2025 (freeCodeCamp):** youtube.com/watch?v=lG7Uxts9SXs

🔨 Section 2: 10 Complete Projects with Full Code

Each project is explained in simple language. Every project builds on skills from the previous one. Start from Project 1 and work up. All code is copy-pasteable and tested.

PROJECT 1

Smart Temperature Monitor with Auto Alert

⏱️ Time: 3–4 hours 🎯 Difficulty: Beginner

This is your very first IoT project. A DHT22 sensor connected to Raspberry Pi reads temperature and humidity every 10 seconds. If temperature goes above your set limit, it sends you an email automatically. This teaches you the complete sensor-to-alert pipeline.

Hardware Required

- Raspberry Pi 4 (any model)
- DHT22 or DHT11 temperature and humidity sensor
- 3 jumper wires (female-to-female)
- Breadboard (optional)
- 10 kΩ resistor (for DHT signal pin — optional but recommended)

Software & Libraries

- Python 3.11
- adafruit-circuitpython-dht (sensor library)
- smtplib (built-in Python, for email)
- python-dotenv

Wiring / Connections

```
DHT22 VCC → Pi Pin 1 (3.3V)
DHT22 GND → Pi Pin 6 (GND)
DHT22 DATA → Pi Pin 7 (GPIO4)
Optional: 10kΩ resistor between VCC and DATA pin
```

Step-by-Step Build Guide

Step 1: **Install library —** Open terminal on Pi: sudo pip3 install adafruit-circuitpython-dht

Step 2: **Wire the sensor —** Connect DHT22 VCC → Pi 3.3V (Pin 1), GND → Pi GND (Pin 6), DATA → Pi GPIO4 (Pin 7)

Step 3: **Create .env file —** Create /home/pi/project1/.env and add your email credentials

Step 4: **Write the code —** Create /home/pi/project1/monitor.py with the code below

Step 5: **Test manually —** Run: python3 monitor.py — you should see readings every 10 seconds

Step 6: **Set temperature limit —** Change ALERT_TEMP in the code to your desired threshold

Step 7: **Make it run forever —** Use nohup or create a systemd service

Core Code

```
# monitor.py — Smart Temperature Monitor
import time, smtplib, logging
import adafruit_dht
import board
from email.mime.text import MIMEText
from dotenv import load_dotenv
import os

load_dotenv()
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")

# ── SETTINGS ── (change these to your values)
ALERT_TEMP    = 35.0          # alert if temperature exceeds this
CHECK_INTERVAL = 10           # seconds between readings
EMAIL_FROM    = os.getenv("EMAIL_FROM")
EMAIL_TO      = os.getenv("EMAIL_TO")
EMAIL_PASS    = os.getenv("EMAIL_PASS")

# ── SENSOR SETUP ──
dht = adafruit_dht.DHT22(board.D4)   # GPIO pin 4

def send_alert(temp: float, humidity: float) -> None:
    """Send email alert when temperature is too high."""
    msg = MIMEText(f"ALERT!\nTemperature: {temp}°C\nHumidity: {humidity}%")
    msg["Subject"] = f"[HIGH TEMP] {temp}°C detected!"
    msg["From"]    = EMAIL_FROM
    msg["To"]      = EMAIL_TO
    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as s:
        s.login(EMAIL_FROM, EMAIL_PASS)
        s.sendmail(EMAIL_FROM, EMAIL_TO, msg.as_string())
    logging.info("Alert email sent!")

last_alert_time = 0   # prevent spam: only alert once every 30 minutes

while True:
    try:
        temp     = dht.temperature
        humidity = dht.humidity
        logging.info(f"Temp={temp}°C  Humidity={humidity}%")
```

```
        # Check if alert needed and cooldown passed
        if temp > ALERT_TEMP and (time.time() - last_alert_time) > 1800:
            send_alert(temp, humidity)
            last_alert_time = time.time()

    except RuntimeError as e:
        # DHT sensors fail occasionally — just try again
        logging.warning(f"Sensor error (will retry): {e}")

    time.sleep(CHECK_INTERVAL)
```

What You Learn From This Project

- How to read a real physical sensor using Python
- How to send emails programmatically
- Error handling for real hardware (sensors fail sometimes)
- How to prevent spam alerts using cooldown logic
- Logging — never use print() in production code

Video References

▶️ **DHT22 + Raspberry Pi Setup:** youtube.com/watch?v=_k4EBkFq8LA
▶️ **Python Email Sending Tutorial (Corey Schafer):** youtube.com/watch?v=JRCJ6RtE3xU

PROJECT 2

MQTT Sensor Dashboard — Real-Time Web Display

⏱️ Time: 5–6 hours 🎯 Difficulty: Beginner-Intermediate

This project builds a real-time web dashboard that shows live sensor data. Your Raspberry Pi publishes sensor readings via MQTT. A Flask web application subscribes to MQTT and displays the data in a nice chart that updates every second. This is the foundation for all industrial IoT dashboards.

Hardware Required

- Raspberry Pi 4
- DHT22 sensor (from Project 1)
- Laptop or second computer for viewing dashboard

Software & Libraries

- paho-mqtt (MQTT client library)
- flask (web framework)
- flask-socketio (real-time web updates)
- mosquitto (MQTT broker — install on Pi)

Wiring / Connections

```
DHT22 → Pi GPIO4 (same as Project 1)
```

Step-by-Step Build Guide

Step 1: Install Mosquitto broker — sudo apt install -y mosquitto mosquitto-clients && sudo systemctl enable mosquitto && sudo systemctl start mosquitto

Step 2: Test MQTT manually — Open two terminals. In terminal 1: mosquitto_sub -t test/#. In terminal 2: mosquitto_pub -t test/hello -m "world". If terminal 1 shows "world", MQTT is working.

Step 3: Install Python libraries — pip3 install paho-mqtt flask flask-socketio

Step 4: Create publisher — publisher.py reads DHT22 and publishes to MQTT topic sensors/pi001/dht22

Step 5: Create dashboard app — dashboard.py is a Flask app that subscribes to MQTT and pushes updates to the browser

Step 6: Create HTML template — templates/index.html is the web page with Chart.js graphs

Step 7: Run both scripts — Terminal 1: python3 publisher.py. Terminal 2: python3 dashboard.py. Open browser to Pi_IP:5000

Core Code

```
# publisher.py — reads sensor and publishes to MQTT
```

```
import paho.mqtt.client as mqtt
import adafruit_dht, board, time, json, logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)
DEVICE_ID = "pi001"
MQTT_HOST  = "localhost"

# Setup MQTT client
client = mqtt.Client(client_id=f"publisher_{DEVICE_ID}")
client.connect(MQTT_HOST, 1883, keepalive=60)
client.loop_start()   # background thread handles connection

# Setup sensor
dht = adafruit_dht.DHT22(board.D4)

while True:
    try:
        payload = json.dumps({
            "device_id": DEVICE_ID,
            "temperature": dht.temperature,
            "humidity": dht.humidity,
            "timestamp": datetime.utcnow().isoformat()
        })
        topic = f"sensors/{DEVICE_ID}/dht22"
        client.publish(topic, payload, qos=1)
        logging.info(f"Published: {payload[:60]}...")
    except RuntimeError as e:
        logging.warning(f"Sensor error: {e}")
    time.sleep(5)

# ──────────────────────────────────────────────────────
# dashboard.py — Flask app with real-time MQTT updates
# ──────────────────────────────────────────────────────
from flask import Flask, render_template
from flask_socketio import SocketIO, emit
import paho.mqtt.client as mqtt
import json, threading

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")

# MQTT message handler — runs in background thread
def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    # Send data to all connected browsers
```

```
    socketio.emit("sensor_update", data)

# Setup MQTT subscriber
mqtt_client = mqtt.Client("dashboard")
mqtt_client.on_message = on_message
mqtt_client.connect("localhost", 1883)
mqtt_client.subscribe("sensors/#", qos=1)

# Start MQTT in background thread
mqtt_thread = threading.Thread(target=mqtt_client.loop_forever, daemon=True)
mqtt_thread.start()

@app.route("/")
def index(): return render_template("index.html")

if __name__ == "__main__":
    socketio.run(app, host="0.0.0.0", port=5000)
```

What You Learn From This Project

- MQTT publish-subscribe pattern — the foundation of all IoT communication
- How Flask handles web requests
- Real-time browser updates using WebSocket (SocketIO)
- Threading — running MQTT and web server at the same time
- JSON — the universal IoT data format

Video References

▶️ **MQTT Python Tutorial (HiveMQ):** youtube.com/watch?v=kuyCd53AOtg
▶️ **Flask SocketIO Real-time (Pretty Printed):** youtube.com/watch?v=zQDzNNt6xd4

PROJECT 3

Edge AI Image Classifier — Run AI on Raspberry Pi

⏱️ Time: 6–8 hours 🎯 Difficulty: Intermediate

This is the most important project for your Edge AI career. You run a TensorFlow Lite image classification model directly on the Raspberry Pi — no internet required. Point the camera at any object and the Pi tells you what it is within 30ms. This demonstrates the core concept of Edge AI.

Hardware Required

- Raspberry Pi 4 (4GB RAM recommended)
- Raspberry Pi Camera Module 3 (or Camera Module 2)
- Heatsink case for Pi (prevents thermal throttling during inference)

Software & Libraries

- tflite-runtime (edge AI inference library)
- picamera2 (Pi camera library)
- numpy
- Pillow (image processing)

Wiring / Connections

```
Pi Camera Module → CSI ribbon cable → Pi Camera port (small slot near HDMI)
No GPIO wiring needed for camera module
```

Step-by-Step Build Guide

Step 1: **Enable camera** — sudo raspi-config → Interface Options → Camera → Enable → Reboot

Step 2: **Install libraries** — pip3 install tflite-runtime picamera2 numpy Pillow

Step 3: **Download model** — Download MobileNet V3 TFLite model from TensorFlow Hub — link in code comments

Step 4: **Download labels** — Download ImageNet labels file (1000 classes) — link in code comments

Step 5: **Run single image test** — Test with a photo file first before using live camera

Step 6: **Run live classification** — Run with --live flag to classify camera feed in real time

Step 7: **Measure speed** — The code automatically measures and prints inference time per frame

Core Code

```
# edge_classifier.py — Edge AI Image Classifier on Raspberry Pi
# Model download:
tfhub.dev/tensorflow/lite-model/mobilenet_v3_large_100_224/classification
```

```
# Labels download:
storage.googleapis.com/download.tensorflow.org/data/ImageNetLabels.txt

import tflite_runtime.interpreter as tflite
import numpy as np
from PIL import Image
import time, argparse, logging

logging.basicConfig(level=logging.INFO)

# ── Model Loading ──
def load_model(model_path: str) -> tflite.Interpreter:
    """Load TFLite model and allocate memory."""
    interp = tflite.Interpreter(model_path=model_path)
    interp.allocate_tensors()
    logging.info(f"Model loaded: {model_path}")
    return interp

def load_labels(labels_path: str) -> list[str]:
    """Load class label names."""
    with open(labels_path, "r") as f:
        return [line.strip() for line in f.readlines()]

# ── Image Preprocessing ──
def preprocess(image_path: str, size: int = 224) -> np.ndarray:
    """Resize and normalise image for MobileNet input."""
    img = Image.open(image_path).convert("RGB")
    img = img.resize((size, size))        # resize to 224x224
    arr = np.array(img, dtype=np.float32) # convert to numpy
    arr = arr / 255.0                     # normalise to [0, 1]
    arr = np.expand_dims(arr, axis=0)     # add batch dimension: (1, 224, 224, 3)
    return arr

# ── Inference ──
def classify(interp: tflite.Interpreter,
             image: np.ndarray,
             labels: list[str],
             top_k: int = 3) -> list[tuple[str, float]]:
    """Run inference and return top K predictions."""
    input_idx  = interp.get_input_details()[0]["index"]
    output_idx = interp.get_output_details()[0]["index"]

    interp.set_tensor(input_idx, image)

    t_start = time.perf_counter()
    interp.invoke()
```

```
    latency_ms = (time.perf_counter() - t_start) * 1000

    scores = interp.get_tensor(output_idx)[0]  # shape: (1000,)
    top_indices = np.argsort(scores)[::-1][:top_k]

    results = [(labels[i], float(scores[i])) for i in top_indices]
    logging.info(f"Inference: {latency_ms:.1f}ms")
    return results

# ── Main ──
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--model",  default="mobilenet_v3.tflite")
    parser.add_argument("--labels", default="labels.txt")
    parser.add_argument("--image",  default=None)
    parser.add_argument("--live",   action="store_true")
    args = parser.parse_args()

    interp = load_model(args.model)
    labels = load_labels(args.labels)

    if args.image:
        # Classify a single image file
        img = preprocess(args.image)
        results = classify(interp, img, labels)
        print("\nTop 3 Predictions:")
        for name, score in results:
            print(f"  {name:30s} {score*100:.1f}%")

    elif args.live:
        # Live camera classification
        from picamera2 import Picamera2
        cam = Picamera2()
        cam.configure(cam.create_preview_configuration(
            main={"size": (640, 480), "format": "RGB888"}
        ))
        cam.start()
        print("\nLive classification started. Press Ctrl+C to stop.")
        while True:
            frame = cam.capture_array()          # get camera frame
            pil_img = Image.fromarray(frame)
            pil_img = pil_img.resize((224, 224))
            arr = np.array(pil_img, dtype=np.float32) / 255.0
            arr = np.expand_dims(arr, 0)
            results = classify(interp, arr, labels)
            top_label, top_score = results[0]
```

```
            print(f"\r{top_label:30s} {top_score*100:.0f}%", end="", flush=True)
```

What You Learn From This Project

- TensorFlow Lite runtime — how to load and run a model
- Image preprocessing — why normalisation to [0,1] matters
- Inference pipeline: load model → preprocess image → invoke → read output
- Latency measurement — how to benchmark your edge AI system
- argparse — professional command-line interface for Python scripts

Video References

▶️ **TFLite on Raspberry Pi (The AI Guy):** youtube.com/watch?v=aimSGOAUI8Y
▶️ **MobileNet Explained Simply:** youtube.com/watch?v=HD9FnjVwU8g

PROJECT 4

AI-Powered Anomaly Detector with Claude Analysis

⏱️ Time: 6–8 hours 🎯 Difficulty: Intermediate

This project combines your sensor pipeline (Project 2) with the Claude API (Section 1) to create an intelligent monitoring system. When the Pi detects an unusual sensor reading, it does not just send an alert — it asks Claude to explain what might be wrong and what to do. This is the AI-augmented IoT system that impresses recruiters.

Hardware Required

- Raspberry Pi 4
- DHT22 temperature/humidity sensor
- Working MQTT setup from Project 2

Software & Libraries

- paho-mqtt
- anthropic (Claude API)
- python-dotenv
- All libraries from Project 2

Wiring / Connections

```
DHT22 → Pi GPIO4 (same as Project 1)
Mosquitto MQTT broker running on Pi
```

Step-by-Step Build Guide

Step 1: **Get Claude API key —** console.anthropic.com → Create API key → add to .env file as ANTHROPIC_API_KEY=sk-ant-...

Step 2: **Install Claude library —** pip3 install anthropic

Step 3: **Create the analyser —** Create analyser.py — this subscribes to MQTT and calls Claude when anomaly detected

Step 4: **Set your thresholds —** Edit NORMAL_TEMP_RANGE and NORMAL_HUMIDITY_RANGE in the code

Step 5: **Run the system —** Terminal 1: python3 publisher.py (from Project 2). Terminal 2: python3 analyser.py

Step 6: **Trigger an anomaly —** Put your hand around DHT22 sensor to raise temperature — watch the AI response

Core Code

```
# analyser.py — AI-Powered Anomaly Detector with Claude
import paho.mqtt.client as mqtt
import anthropic, json, logging, os
from dotenv import load_dotenv
from datetime import datetime
from collections import deque

load_dotenv()
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")

# ── SETTINGS ──
NORMAL_TEMP_RANGE     = (15.0, 35.0)   # (min, max) degrees Celsius
NORMAL_HUMIDITY_RANGE = (30.0, 80.0)   # (min, max) percent
HISTORY_SIZE          = 10              # remember last 10 readings for context

# ── CLAUDE SETUP ──
claude = anthropic.Anthropic()

SYSTEM_PROMPT = """
You are an expert IoT system diagnostic engineer.
When given sensor readings, you:
1. Identify what is abnormal and why
2. List 2 possible physical causes (simple language)
3. Give 1 immediate action to take
4. Rate urgency: LOW / MEDIUM / HIGH / CRITICAL
Keep response under 5 sentences. Be direct.
"""

def ask_claude(current: dict, history: list) -> str:
    """Ask Claude to analyse an anomalous reading."""
    history_text = "\n".join([
        f"  {r['timestamp']}: {r['temperature']}°C, {r['humidity']}%"
        for r in list(history)[-5:]  # last 5 readings for context
    ])

    user_message = f"""
Current reading (ANOMALOUS):
  Temperature: {current['temperature']}°C (Normal: 15-35°C)
  Humidity: {current['humidity']}% (Normal: 30-80%)
  Time: {current['timestamp']}

Recent history (last 5 readings):
{history_text}
```

```
What is happening and what should I do?
    """

    response = claude.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=300,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": user_message}]
    )
    return response.content[0].text

# ── ANOMALY DETECTION ──
def is_anomaly(data: dict) -> bool:
    """Check if reading is outside normal range."""
    temp_lo, temp_hi = NORMAL_TEMP_RANGE
    hum_lo, hum_hi   = NORMAL_HUMIDITY_RANGE
    return (data["temperature"] < temp_lo or data["temperature"] > temp_hi or
            data["humidity"] < hum_lo or data["humidity"] > hum_hi)

# ── MQTT SETUP ──
history = deque(maxlen=HISTORY_SIZE)

def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    history.append(data)

    if is_anomaly(data):
        logging.warning(f"ANOMALY DETECTED: {data}")
        print("\n" + "="*50)
        print("ANOMALY DETECTED!")
        print(f"Temperature: {data['temperature']}°C")
        print(f"Humidity:    {data['humidity']}%")
        print("\nAsking Claude AI for analysis...")
        analysis = ask_claude(data, history)
        print(f"\n🤖 Claude Analysis:\n{analysis}")
        print("="*50 + "\n")
    else:
        logging.info(f"Normal: {data['temperature']}°C, {data['humidity']}%")

client = mqtt.Client("ai_analyser")
client.on_message = on_message
client.connect("localhost", 1883)
client.subscribe("sensors/#", qos=1)
logging.info("AI Anomaly Analyser running. Waiting for sensor data...")
client.loop_forever()
```

What You Learn From This Project

- How to integrate an AI API (Claude) into an IoT system
- Contextual analysis — giving AI history improves its reasoning
- Deque — efficient fixed-size history buffer in Python
- Building a complete sensor-to-AI pipeline from scratch
- Cost management — how to avoid spending too much on API calls (use cooldowns)

Video References

▶️ **Claude API Python (Anthropic Docs):** docs.anthropic.com/en/docs/quickstart
▶️ **AI in IoT Systems (General):** youtube.com/watch?v=rExc-V_qHzI

PROJECT 5

LangChain Conversational IoT Assistant

⏱️ Time: 8–10 hours 🎯 Difficulty: Intermediate-Advanced

This project builds a chatbot that you can have a real conversation with about your IoT system. Unlike Project 4 which gives one-off analyses, this assistant REMEMBERS previous messages. You can ask "What was the temperature an hour ago?" and then "Is that normal for this time of day?" and it connects the context.

Hardware Required

- Raspberry Pi 4 (or just your laptop for this project)
- DHT22 sensor (optional — can simulate data)
- SQLite database (built into Python, no installation)

Software & Libraries

- langchain
- langchain-anthropic
- anthropic
- sqlite3 (built into Python)
- python-dotenv

Wiring / Connections

```
No physical wiring needed — uses SQLite database
```

Step-by-Step Build Guide

Step 1: Install LangChain — pip3 install langchain langchain-anthropic anthropic

Step 2: Create database — Run the database setup section of the code to create sensor_history.db

Step 3: Populate with fake data — Run the data generator to create 24 hours of simulated sensor data

Step 4: Create the agent — Create iot_assistant.py with the code below

Step 5: Start conversation — Run python3 iot_assistant.py and start asking questions

Step 6: Try complex queries — Ask "What was the average temperature yesterday?" or "Show me all anomalies this week"

Core Code

```
# iot_assistant.py — LangChain Conversational IoT Assistant
import sqlite3, json, logging
from datetime import datetime, timedelta
from dotenv import load_dotenv
```

```
from langchain_anthropic import ChatAnthropic
from langchain.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain.memory import ConversationBufferWindowMemory

load_dotenv()
DB_PATH = "sensor_history.db"

# ── DATABASE TOOLS (what the agent can query) ──
@tool
def get_readings(sensor_type: str = "temperature", hours_ago: int = 1) -> str:
    """Get sensor readings from the last N hours. sensor_type: temperature or
humidity."""
    since = datetime.utcnow() - timedelta(hours=hours_ago)
    with sqlite3.connect(DB_PATH) as conn:
        rows = conn.execute("""
            SELECT value, timestamp FROM readings
            WHERE sensor_type = ? AND timestamp > ?
            ORDER BY timestamp DESC LIMIT 50
        """, (sensor_type, since.isoformat())).fetchall()
    if not rows:
        return f"No {sensor_type} readings found in last {hours_ago} hour(s)"
    values = [r[0] for r in rows]
    return json.dumps({"count": len(rows), "min": min(values),
                       "max": max(values), "avg": round(sum(values)/len(values), 2),
                       "latest": values[0], "unit": "C" if sensor_type=="temperature"
else "%"})

@tool
def count_anomalies(threshold: float = 35.0, days_ago: int = 1) -> str:
    """Count how many times temperature exceeded a threshold in last N days."""
    since = datetime.utcnow() - timedelta(days=days_ago)
    with sqlite3.connect(DB_PATH) as conn:
        count = conn.execute("""
            SELECT COUNT(*) FROM readings
            WHERE sensor_type="temperature" AND value > ? AND timestamp > ?
        """, (threshold, since.isoformat())).fetchone()[0]
    return f"Temperature exceeded {threshold}°C exactly {count} time(s) in last
{days_ago} day(s)"

@tool
def get_current_reading() -> str:
    """Get the most recent sensor reading."""
    with sqlite3.connect(DB_PATH) as conn:
        row = conn.execute("""
```

```
            SELECT sensor_type, value, timestamp FROM readings
            ORDER BY timestamp DESC LIMIT 1
        """).fetchone()
    if row: return f"Latest: {row[0]}={row[1]} at {row[2]}"
    return "No readings found in database"

# ── LANGCHAIN AGENT WITH MEMORY ──
llm    = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
tools  = [get_readings, count_anomalies, get_current_reading]

prompt = ChatPromptTemplate.from_messages([
    ("system", """You are a friendly IoT monitoring assistant.
    You help users understand their sensor data.
    Always use the available tools to get actual data before answering.
    Explain results in simple, non-technical language unless asked otherwise."""),
    MessagesPlaceholder(variable_name="chat_history"),  # conversation memory
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad"),
])

memory = ConversationBufferWindowMemory(
    memory_key="chat_history",
    return_messages=True,
    k=10   # remember last 10 exchanges
)

agent    = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools,
                         memory=memory, verbose=False)

print("\n🤖 IoT Assistant ready! (type 'exit' to quit)")
print("Try: 'What is the current temperature?'")
print("Try: 'How many times did it overheat today?'\n")

while True:
    user_input = input("You: ").strip()
    if user_input.lower() in ["exit", "quit", "bye"]: break
    if not user_input: continue
    result = executor.invoke({"input": user_input})
    print(f"\nAssistant: {result['output']}\n")
```

What You Learn From This Project

- LangChain agent framework — how tools + LLM + memory work together
- Conversation memory — keeping context across messages

- Database queries from within AI tools
- How to design tools that an AI agent can use effectively
- The difference between stateless AI (no memory) and stateful AI (with memory)

Video References

▶️ **LangChain Agents with Memory:** youtube.com/watch?v=oLNBBGlKYkE
▶️ **LangChain Full Tutorial:** youtube.com/watch?v=lG7Uxts9SXs

PROJECT 6

n8n Factory Alert Automation — No-Code Pipeline

⏱️ Time: 4–5 hours 🎯 Difficulty: Intermediate

This project builds a complete factory alert system using n8n — mostly using the visual interface, very little coding. When sensors detect problems, the system: saves to database, sends email alert, posts to a Slack channel, and generates an AI analysis report. This shows future employers you can build production automation systems quickly.

Hardware Required

- Raspberry Pi 4 with n8n installed (from Section 1, 3.3)
- Working MQTT broker and sensor publisher from Project 2
- Gmail account for email alerts
- Slack account (free tier) for team notifications

Software & Libraries

- n8n (already installed)
- No additional Python libraries needed for this project

Wiring / Connections

```
Pi running MQTT broker
n8n connects to MQTT internally — no additional wiring
```

Step-by-Step Build Guide

Step 1: Start n8n — sudo systemctl start n8n → open http://Pi_IP:5678 in browser

Step 2: Create new workflow — Click "New Workflow" → name it "Factory Alert System"

Step 3: Add MQTT Trigger node — Click + → search "MQTT Trigger" → Host: localhost, Port: 1883, Topic: sensors/#

Step 4: Add IF node — Click + → "IF" node → Condition: {{$json["temperature"]}} > 38

Step 5: Add Gmail node — On the TRUE branch → + → "Gmail" → configure with your Gmail OAuth credentials

Step 6: Add Slack node — After Gmail → + → "Slack" → connect your workspace → post to #alerts channel

Step 7: Add Claude AI node — + → "Anthropic" → model: claude-sonnet-4-6 → paste system prompt from below

Step 8: Add Schedule trigger — Create a second workflow: Daily 8AM → read database → Claude report → email daily summary

Core Code

```
# Prompt to paste into the n8n Claude AI node:

System Prompt:
You are a factory monitoring AI. Analyse the sensor alert and provide:
1. Severity level: LOW / MEDIUM / HIGH / CRITICAL
2. Most likely cause (2 sentences)
3. Recommended action (1 sentence)
Format: JSON only. Keys: severity, cause, action

User Message template (n8n variables):
Temperature: {{$json["temperature"]}}°C
Humidity: {{$json["humidity"]}}%
Device ID: {{$json["device_id"]}}
Timestamp: {{$json["timestamp"]}}
Normal ranges: Temp 15-35°C, Humidity 30-80%
What is the severity and recommended action?
```

What You Learn From This Project

- Visual workflow design — important skill for no-code/low-code platforms
- API integration without writing API code (n8n handles authentication)
- Claude AI integration from a non-Python environment
- Building production-grade alert systems rapidly
- Understanding the difference between code-first and visual-first approaches

Video References

▶️ **n8n MQTT Tutorial:** youtube.com/watch?v=GqFAlmDt62Y
▶️ **n8n + AI Workflow:** youtube.com/watch?v=qqqAePVAOKo

PROJECT 7

Raspberry Pi Predictive Maintenance System

⏱️ Time: 2–3 weeks 🎯 Difficulty: Advanced

This is your NIAT-grade showcase project. An ADXL345 vibration sensor monitors a motor. A TFLite model classifies the vibration signature as normal, imbalance fault, or bearing fault. The system predicts failures BEFORE they happen by detecting early fault patterns. This is exactly what Japanese factories use.

Hardware Required

- Raspberry Pi 4 (4GB RAM)
- ADXL345 accelerometer sensor (vibration measurement)
- Small DC motor with shaft (for realistic vibration simulation)
- Breadboard + jumper wires

Software & Libraries

- tflite-runtime
- adafruit-circuitpython-adxl34x
- numpy
- scipy (for FFT analysis)
- paho-mqtt

Wiring / Connections

```
ADXL345 VCC → Pi 3.3V (Pin 1)
ADXL345 GND → Pi GND (Pin 6)
ADXL345 SDA → Pi GPIO2 (Pin 3)
ADXL345 SCL → Pi GPIO3 (Pin 5)
```

Step-by-Step Build Guide

Step 1: **Wire ADXL345 to Pi via I2C —** VCC → 3.3V, GND → GND, SDA → GPIO2 (Pin 3), SCL → GPIO3 (Pin 5)

Step 2: **Enable I2C on Pi —** sudo raspi-config → Interfaces → I2C → Enable

Step 3: **Install sensor library —** pip3 install adafruit-circuitpython-adxl34x

Step 4: **Collect training data —** Run data_collector.py while motor is: (a) normal, (b) with small weight attached to shaft (imbalance)

Step 5: **Train TFLite model —** Train in Google Colab using the collected CSV data → export to model.tflite

Step 6: **Deploy and test —** Copy model.tflite to Pi → run predictor.py → watch it classify in real time

Core Code

```
# predictor.py — Real-Time Vibration Fault Classifier
import tflite_runtime.interpreter as tflite
import numpy as np
import adafruit_adxl34x, board, busio
import time, logging, paho.mqtt.client as mqtt, json

logging.basicConfig(level=logging.INFO)
CLASSES = ["normal", "imbalance_fault", "bearing_fault"]
SAMPLE_RATE = 100  # readings per second
WINDOW_SIZE = 50   # use 50 readings per classification

# ── Sensor Setup ──
i2c   = busio.I2C(board.SCL, board.SDA)
adxl  = adafruit_adxl34x.ADXL345(i2c)
adxl.range = adafruit_adxl34x.Range.RANGE_2_G

# ── Model Setup ──
interp = tflite.Interpreter("vibration_model.tflite")
interp.allocate_tensors()
in_idx  = interp.get_input_details()[0]["index"]
out_idx = interp.get_output_details()[0]["index"]

# ── Feature Extraction ──
def extract_features(window: list) -> np.ndarray:
    """Convert raw XYZ readings to statistical features."""
    arr = np.array(window)         # shape: (50, 3)
    x, y, z = arr[:,0], arr[:,1], arr[:,2]
    magnitude = np.sqrt(x**2 + y**2 + z**2)
    features = [
        magnitude.mean(), magnitude.std(),
        magnitude.max(), magnitude.min(),
        np.percentile(magnitude, 25),
        np.percentile(magnitude, 75),
        np.sqrt(np.mean(magnitude**2)),  # RMS (root mean square)
        float(np.fft.fft(magnitude)[:3].real.mean()),  # dominant frequency
    ]
    return np.array([features], dtype=np.float32)  # shape: (1, 8)

# ── MQTT Setup ──
mq = mqtt.Client("predictor")
mq.connect("localhost", 1883)
mq.loop_start()

# ── Main Loop ──
```

```
window = []
while True:
    x, y, z = adxl.acceleration
    window.append((x, y, z))

    if len(window) >= WINDOW_SIZE:
        features = extract_features(window)
        interp.set_tensor(in_idx, features)
        interp.invoke()
        probs   = interp.get_tensor(out_idx)[0]
        pred_id = np.argmax(probs)
        pred    = CLASSES[pred_id]
        conf    = probs[pred_id]

        result = {"prediction": pred, "confidence": float(conf), "time":
time.time()}
        logging.info(f"Prediction: {pred} ({conf*100:.1f}%)")
        mq.publish("predictions/vibration", json.dumps(result))

        if pred != "normal":
            logging.warning(f"FAULT DETECTED: {pred}!")
            mq.publish("alerts/vibration", json.dumps(result), qos=1)

        window = []  # reset window

    time.sleep(1/SAMPLE_RATE)
```

What You Learn From This Project

- Vibration analysis using FFT (Fast Fourier Transform) — used in industrial maintenance worldwide
- Feature engineering — converting raw signals into meaningful numbers for ML
- Window-based classification — collecting N readings before making a decision
- Combining hardware sensor knowledge (from EEE) with ML (from CS-IT)
- This project demonstrates exactly the AIoT profile Japanese factories want

Video References

▶️ **Vibration Analysis with FFT:** youtube.com/watch?v=spUNpyF58BY
▶️ **ADXL345 Python Tutorial:** youtube.com/watch?v=BjEgkfose7s

PROJECT 8

WhatsApp IoT Control Bot

⏱️ Time: 5–6 hours 🎯 Difficulty: Intermediate

Control your Raspberry Pi and read sensor data using WhatsApp messages from your phone. Send "temperature?" and get the current reading. Send "photo" and receive a camera snapshot. This impresses everyone — it is the easiest way to demonstrate your AIoT system in a job interview.

Hardware Required

- Raspberry Pi 4 with camera module
- Twilio account (free trial gives you enough credits)
- WhatsApp on your phone

Software & Libraries

- twilio (WhatsApp API library)
- flask (web server to receive messages)
- ngrok (expose Pi to internet temporarily for testing)
- picamera2
- paho-mqtt

Wiring / Connections

```
DHT22 → Pi GPIO4
Pi Camera → CSI ribbon cable
Internet connection required for Twilio API
```

Step-by-Step Build Guide

Step 1: **Create Twilio account —** twilio.com → Sign up free → Get a WhatsApp sandbox number

Step 2: **Install libraries —** pip3 install twilio flask

Step 3: **Install ngrok —** Download ngrok from ngrok.com → ./ngrok http 5000 (this gives Pi a public URL)

Step 4: **Set Twilio webhook —** In Twilio console → WhatsApp Sandbox → "When a message comes in" → paste ngrok URL + /webhook

Step 5: **Create the bot —** Create whatsapp_bot.py with the code below

Step 6: **Test —** Send "hello" to your Twilio WhatsApp number and the Pi responds

Core Code

```
# whatsapp_bot.py — WhatsApp IoT Control Bot
from flask import Flask, request
```

```
from twilio.twiml.messaging_response import MessagingResponse
from twilio.rest import Client
import adafruit_dht, board, os, json
from picamera2 import Picamera2
from dotenv import load_dotenv
import paho.mqtt.client as mqtt

load_dotenv()
app = Flask(__name__)

# Twilio credentials from .env
twilio_client = Client(os.getenv("TWILIO_SID"), os.getenv("TWILIO_TOKEN"))
TWILIO_WHATSAPP = "whatsapp:+14155238886"  # your sandbox number

# Setup sensor
dht = adafruit_dht.DHT22(board.D4)

def send_whatsapp(to: str, message: str) -> None:
    """Send a WhatsApp text message."""
    twilio_client.messages.create(
        body=message,
        from_=TWILIO_WHATSAPP,
        to=f"whatsapp:{to}"
    )

@app.route("/webhook", methods=["POST"])
def webhook():
    """Handle incoming WhatsApp messages."""
    incoming = request.form.get("Body", "").lower().strip()
    sender   = request.form.get("From", "").replace("whatsapp:", "")

    resp = MessagingResponse()

    if "temperature" in incoming or "temp" in incoming:
        try:
            temp = dht.temperature
            hum  = dht.humidity
            msg  = f"🌡️ Temperature: {temp}°C\n💧 Humidity: {hum}%"
        except RuntimeError:
            msg = "❌ Sensor error — please try again"
        resp.message(msg)

    elif "photo" in incoming or "camera" in incoming:
        cam = Picamera2()
        cam.start()
        cam.capture_file("/tmp/snapshot.jpg")
```

```
        cam.stop()
        # Send image via Twilio media URL (requires hosted file)
        resp.message("📸 Photo captured! (Configure media URL to send image)")

    elif "status" in incoming:
        resp.message("✅ Pi is online and running normally")

    elif "help" in incoming:
        resp.message("Commands:\n• temperature – get sensor reading\n• photo – 
capture image\n• status – check Pi status")

    else:
        resp.message("I did not understand. Send 'help' for commands.")

    return str(resp)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

What You Learn From This Project

- Webhooks — how internet services send data to your server
- ngrok — how to expose a local server to the internet during development
- Twilio API — industry-standard messaging platform
- Building a command-line interface over WhatsApp
- Why messaging interfaces are the future of IoT control (no app needed)

Video References

▶️ **Twilio WhatsApp Python Tutorial:** youtube.com/watch?v=iKlVpfFBBNQ
▶️ **Flask Webhooks Tutorial:** youtube.com/watch?v=zYmjJEqkCBQ

PROJECT 9

Full Stack AIoT Web Application — React + FastAPI + MongoDB

⏱️ Time: 2–3 weeks 🎯 Difficulty: Advanced

This is your complete full-stack project. A React web application shows real-time IoT data. A FastAPI backend serves data from MongoDB Atlas. The Raspberry Pi publishes sensor data that flows to the cloud automatically. This project demonstrates every layer of modern AIoT system architecture.

Hardware Required

- Raspberry Pi 4 with sensors from previous projects
- MongoDB Atlas free account (cloud.mongodb.com)
- Laptop with Node.js for React development

Software & Libraries

- fastapi (Python web framework)
- uvicorn (ASGI server for FastAPI)
- motor (async MongoDB driver)
- react (create-react-app)
- recharts (React chart library)
- All MQTT and sensor libraries from previous projects

Wiring / Connections

```
All sensors from previous projects
MongoDB Atlas — cloud only, no wiring
React runs on your laptop browser
```

Step-by-Step Build Guide

Step 1: Create MongoDB Atlas cluster — cloud.mongodb.com → New Project → Build Database (Free Tier) → copy connection string to .env

Step 2: Create FastAPI backend — Create backend/ folder → main.py with FastAPI app

Step 3: Connect MQTT to MongoDB — The backend subscribes to MQTT and saves readings to Atlas automatically

Step 4: Create REST endpoints — GET /readings, GET /readings/latest, GET /alerts, GET /stats

Step 5: Create React frontend — npx create-react-app frontend → install recharts → build dashboard

Step 6: Deploy backend — Run uvicorn main:app --host 0.0.0.0 --port 8000 on Pi

Step 7: Run frontend — npm start in frontend/ → open localhost:3000 in browser

Core Code

```
# backend/main.py — FastAPI + MongoDB + MQTT Backend
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from motor.motor_asyncio import AsyncIOMotorClient
import paho.mqtt.client as mqtt
import json, os, asyncio, threading
from datetime import datetime, timedelta
from dotenv import load_dotenv

load_dotenv()
app = FastAPI(title="AIoT Dashboard API")

# Allow React frontend to call this API
app.add_middleware(CORSMiddleware,
    allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])

# ── MongoDB Setup ──
mongo = AsyncIOMotorClient(os.getenv("MONGO_URI"))
db    = mongo["aiot_database"]

# ── API Endpoints ──
@app.get("/readings/latest")
async def get_latest():
    """Get the 20 most recent sensor readings."""
    cursor = db.readings.find({}, {"_id": 0}).sort("timestamp", -1).limit(20)
    return await cursor.to_list(length=20)

@app.get("/readings/stats")
async def get_stats():
    """Get 24-hour statistics for all sensors."""
    since = datetime.utcnow() - timedelta(hours=24)
    pipeline = [
        {"$match": {"timestamp": {"$gt": since.isoformat()}}},
        {"$group": {"_id": "$sensor_type",
                    "avg": {"$avg": "$value"},
                    "max": {"$max": "$value"},
                    "min": {"$min": "$value"},
                    "count": {"$sum": 1}}}
    ]
    return await db.readings.aggregate(pipeline).to_list(length=10)

@app.get("/alerts")
async def get_alerts():
    """Get recent alerts."""
```

```
    cursor = db.alerts.find({}, {"_id": 0}).sort("timestamp", -1).limit(50)
    return await cursor.to_list(length=50)

# ── MQTT Bridge → MongoDB ──
def mqtt_on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    # Run async insert in background
    asyncio.run_coroutine_threadsafe(
        db.readings.insert_one(data), asyncio.get_event_loop()
    )

mqtt_client = mqtt.Client("fastapi_bridge")
mqtt_client.on_message = mqtt_on_message
mqtt_client.connect("localhost", 1883)
mqtt_client.subscribe("sensors/#", qos=1)
threading.Thread(target=mqtt_client.loop_forever, daemon=True).start()
```

What You Learn From This Project

- FastAPI — modern Python web framework (faster than Flask for APIs)
- MongoDB aggregation pipeline — like SQL GROUP BY but for documents
- CORS — why browsers block API calls and how to allow them
- Full-stack architecture: Sensor → MQTT → Python → MongoDB → REST API → React
- This project architecture is used in production by real IoT companies

Video References

▶️ **FastAPI Tutorial (TechWithTim):** youtube.com/watch?v=0RS9W8MtZe4
▶️ **React + FastAPI Full Stack:** youtube.com/watch?v=PWG7NlUDVaA

PROJECT 10

Complete AI-Native Autonomous Monitoring System

⏱️ Time: 4–6 weeks 🎯 Difficulty: Expert

This is your capstone project — the project that defines your portfolio. It combines everything from all 9 previous projects plus adds autonomous decision-making using AI agents. The system monitors multiple sensor types, learns normal patterns, detects anomalies, explains them with Claude, sends multi-channel alerts via n8n, logs to MongoDB, and displays on a React dashboard. When something goes wrong, it generates a full incident report automatically.

Hardware Required

- Raspberry Pi 4 (4GB RAM)
- DHT22 — temperature/humidity
- ADXL345 — vibration
- Pi Camera Module
- Laptop for React dashboard

Software & Libraries

- All libraries from Projects 1-9
- langchain + langchain-anthropic
- All previous Python libraries

Wiring / Connections

```
All wiring from Projects 1, 3, 7
```

Step-by-Step Build Guide

Step 1: Set up all sensors — Wire DHT22 (GPIO4) and ADXL345 (I2C) to Pi

Step 2: Deploy sensor manager — sensor_manager.py from Appendix J manages all sensors in parallel threads

Step 3: Deploy edge AI — Run both TFLite classifiers: image classifier (Project 3) and vibration predictor (Project 7)

Step 4: Deploy MQTT pipeline — All sensor data + AI predictions flow through MQTT

Step 5: Deploy LangChain agent — The AI agent from Project 5 monitors everything and responds to natural language queries

Step 6: Set up n8n workflows — Alert workflow + daily report workflow from Project 6

Step 7: Deploy full-stack app — FastAPI + React from Project 9 + add incident report endpoint

Step 8: Test end-to-end — Trigger anomalies, verify full pipeline fires correctly

Core Code

```
# incident_reporter.py — AI-Generated Incident Reports
import anthropic, json, pymongo, datetime
from dotenv import load_dotenv

load_dotenv()
claude = anthropic.Anthropic()
mongo  = pymongo.MongoClient(os.getenv("MONGO_URI"))
db     = mongo["aiot_database"]

REPORT_PROMPT = """
You are a senior AIoT systems engineer writing an incident report.
Write a clear, professional report with these sections:
1. INCIDENT SUMMARY (2 sentences)
2. TIMELINE (bullet points with times)
3. ROOT CAUSE ANALYSIS (most likely physical cause)
4. IMPACT ASSESSMENT (what could have happened if undetected)
5. RECOMMENDED ACTIONS (3 specific steps)
6. PREVENTION (how to prevent this in future)
Use simple, clear language. No jargon. Total: under 400 words.
"""

def generate_incident_report(alert_id: str) -> dict:
    """Generate a full incident report for a given alert."""
    # Fetch alert data from MongoDB
    alert = db.alerts.find_one({"_id": alert_id})
    readings = list(db.readings.find(
        {"timestamp": {"$gt": alert["timestamp"] - datetime.timedelta(hours=1)}},
        {"_id": 0}).limit(100)
    )

    # Ask Claude to write the report
    response = claude.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=800,
        system=REPORT_PROMPT,
        messages=[{
            "role": "user",
            "content": f"""
Alert: {json.dumps(alert, default=str)}
Sensor History: {json.dumps(readings, default=str)}
Generate the incident report.
            """
        }]
    )
```

```
report = {
    "alert_id": str(alert_id),
    "generated_at": datetime.datetime.utcnow().isoformat(),
    "content": response.content[0].text
}
db.incident_reports.insert_one(report)
return report
```

What You Learn From This Project

- System integration — making 10 different technologies work together reliably
- Debugging distributed systems — when multiple components exist, how to find which one failed
- Production thinking — error handling, logging, monitoring the monitoring system itself
- Portfolio documentation — how to write about this project in a resume and on GitHub
- This system represents 6 months of full-time engineering work compressed into a learning project

Video References

▶️ **System Design for IoT (General):** youtube.com/watch?v=ByGJQzlzxQg
▶️ **Building Production IoT Systems:** youtube.com/watch?v=rnkXxJQbORY

📝 Section 3: Grammar Guide for Technical Writing

This section is written specifically for Telugu and Hindi speakers learning to write professional English for resumes, GitHub READMEs, LinkedIn posts, and emails. Every mistake shown here is a real mistake made by Indian engineering students.

Chapter 4: English Grammar for Technical Professionals

Good English is not about speaking with an accent. It is about writing clearly so that a recruiter in Japan, a client in Germany, or a colleague in the USA understands you exactly. Technical writing has its own rules — different from essay writing or conversation.

> 📌 **NOTE:** You do not need perfect English to get a job. You need CLEAR English. Clear means: short sentences, correct subject-verb agreement, no missing articles, and no Telugu/Hindi sentence structure in English writing.

4.1 The Most Common Mistakes — Telugu/Hindi Speakers

Mistake 1: Missing Articles (a, an, the)

Telugu and Hindi do not have articles. So Telugu and Hindi speakers often drop them in English. This makes writing sound robotic and unprofessional.

✗ **WRONG:** *I built sensor system for factory.*
✓ **RIGHT:** I built a sensor system for a factory.

✗ **WRONG:** *We need to restart server.*
✓ **RIGHT:** We need to restart the server.

✗ **WRONG:** *This is important concept in machine learning.*
✓ **RIGHT:** This is an important concept in machine learning.

> 📐 **RULE:** Use "a" before consonant sounds (a sensor, a model, a database). Use "an" before vowel sounds (an algorithm, an IoT device, an API). Use "the" when you are talking about a specific thing both you and the reader know about.

Mistake 2: Wrong Verb Tense in Technical Writing

Technical documents (READMEs, reports) use PRESENT SIMPLE for things that are always true. Use PAST SIMPLE for what you did in a project. Use PRESENT PERFECT for recent achievements.

✗ **WRONG:** *I am built a classifier that is detecting anomalies.*
✓ **RIGHT:** I built a classifier that detects anomalies.

✗ **WRONG:** *The system was sending alerts when temperature is exceeding threshold.*
✓ **RIGHT:** The system sends alerts when the temperature exceeds the threshold.

✗ **WRONG:** *I am knowing Python since 2 years.*
✓ **RIGHT:** I have known Python for 2 years. OR: I have been learning Python for 2 years.

📐 **RULE:** "Know", "understand", "have", "be" are STATIVE verbs. They do NOT take -ing form. Never say "I am knowing" or "I am having". Say "I know" and "I have".

Mistake 3: Subject-Verb Agreement

✗ **WRONG:** *The sensor data are showing high values.*
✓ **RIGHT:** The sensor data is showing high values. (data treated as uncountable in tech writing)

✗ **WRONG:** *Each of the models are trained separately.*
✓ **RIGHT:** Each of the models is trained separately. (each = singular)

✗ **WRONG:** *The team have decided to use MQTT.*
✓ **RIGHT:** The team has decided to use MQTT. (collective noun = singular in American English)

📐 **RULE:** If the subject is ONE thing (even if it sounds plural like "data" or "the team"), use IS/HAS/DOES. Test: replace the subject with "it" — if "it" works, use singular verb.

Mistake 4: Direct Translation from Telugu/Hindi

This is the most common problem. Students translate their mother tongue sentence structure word-for-word into English. The result is grammatically wrong.

✗ **WRONG:** *The project which I have done in NIAT it was very useful for me.*
✓ **RIGHT:** The NIAT project was very useful for my learning.

✗ **WRONG:** *I am from Andhra Pradesh only.*
✓ **RIGHT:** I am from Andhra Pradesh. (remove "only" — it sounds informal/odd in professional writing)

✗ **WRONG:** *What is your good name?*
✓ **RIGHT:** What is your name? (there is no "good name" concept in English)

✗ **WRONG:** *I have done my graduation from Malla Reddy University.*
✓ **RIGHT:** I graduated from Malla Reddy University. OR: I completed my B.Tech at Malla Reddy University.

📐 **RULE:** Before writing, think of the MEANING you want to express, not the Telugu/Hindi sentence. Then build the English sentence from scratch. Do not translate word-by-word.

Mistake 5: Preposition Errors

✗ **WRONG:** *I am good in Python.*
✓ **RIGHT:** I am good at Python.

✗ **WRONG:** *I have experience on embedded systems.*
✓ **RIGHT:** I have experience in embedded systems. OR: I have experience with embedded systems.

✗ **WRONG:** *Please revert back to me.*
✓ **RIGHT:** Please respond to me. OR: Please get back to me. ("revert" means to go back to previous state — not reply)

✗ **WRONG:** *I will do the needful.*
✓ **RIGHT:** I will take the necessary action. OR: I will handle it. ("do the needful" is Indian English — not used internationally)

Correct Preposition	Use With	Example
good AT	skills and activities	I am good at debugging C++ code
experience IN	fields and domains	I have experience in embedded systems
experience WITH	tools and technologies	I have experience with Raspberry Pi
interested IN	topics	I am interested in Edge AI
work ON	projects and tasks	I am working on a TFLite project
work AT	a company	I work at Panasonic Japan
responsible FOR	duties	I am responsible for sensor integration
familiar WITH	technologies	I am familiar with MQTT protocol

4.2 Technical Writing Rules — GitHub README and Reports

Rule 1: Start bullets with action verbs

Every bullet in a resume or README should start with a strong action verb in the correct tense.

For Resume (past tense)	For README (present tense)
Built a TFLite inference pipeline	Runs TFLite inference at 30ms on Raspberry Pi 4
Implemented MQTT pub/sub for IoT	Supports MQTT QoS 0, 1, 2 with auto-reconnect
Reduced inference latency by 40%	Achieves 40% faster inference using INT8 quantisation
Developed a React dashboard	Displays real-time sensor data in Chart.js graphs
Integrated Claude API for analysis	Uses Claude API to explain anomalies in plain English

Rule 2: Write numbers correctly

✗ **WRONG:** *The model achieves ninety-five percent accuracy.*
✓ **RIGHT:** The model achieves 95% accuracy. (use digits for measurements in technical writing)

✗ **WRONG:** *I solved more than hundred problems on LeetCode.*
✓ **RIGHT:** I solved 100+ problems on LeetCode.

✗ **WRONG:** *The system processes datas at rate of 50 reading per second.*
✓ **RIGHT:** The system processes data at 50 readings per second. (data is uncountable — no "datas")

Rule 3: Email Writing — Professional Format

Here is the correct format for a professional email to a recruiter or potential employer:

```
Subject: Application for AIoT Intern Position – B.Tech CS-IT, Python + TFLite

Dear [Name / Hiring Team],

I am writing to apply for the AIoT Software Intern position
advertised on [Platform] on [Date].

I am currently in my second year of B.Tech Computer Science at
Malla Reddy University, with a diploma in Electrical and Electronics
Engineering. I specialise in Edge AI deployment on Raspberry Pi
using TensorFlow Lite, and IoT system integration using MQTT
and Python.

My relevant projects include:
• An end-to-end AIoT pipeline: DHT22 → MQTT → TFLite → MongoDB → React dashboard
• An AI-powered anomaly detector using the Claude API for sensor analysis
• A predictive maintenance classifier for vibration data (ADXL345 + TFLite)

My GitHub (github.com/[username]) has all three projects with demo videos.

I am also studying Japanese at JLPT N4 level, as I am targeting
AIoT engineering roles in Japan after graduation.

I would welcome the opportunity to discuss how I can contribute
to your team. Please find my resume attached.

Thank you for your time and consideration.

Sincerely,
```

```
[Your Full Name]
[Phone Number]
[LinkedIn URL]
[GitHub URL]
```

💡 **TIP:** Keep the email under 250 words. Recruiters read hundreds of emails. Short, specific, with a link to your GitHub — this is what works. Never write "I am a hardworking, passionate, and dedicated student." Everyone says this. Show your work instead.

4.3 LinkedIn Writing Guide

How to Write Your LinkedIn About Section

The About section has 2600 character limit. Write in 3 short paragraphs:

Paragraph 1 — Who you are and what you build (3-4 sentences):

```
I am a B.Tech Computer Science student with an Electrical Engineering
background, building toward a career as an AIoT engineer. My focus is
Edge AI — deploying machine learning models on Raspberry Pi and ESP32
devices to create intelligent IoT systems that work without cloud dependency.
```

Paragraph 2 — What you have built (3 specific things with numbers):

```
I have built three end-to-end projects: (1) a TFLite vibration fault
classifier achieving 93% accuracy at 18ms inference on Raspberry Pi 4,
(2) an AI-powered sensor monitoring system integrating Claude API for
anomaly explanation, and (3) a full-stack IoT dashboard using React,
FastAPI, and MongoDB Atlas.
```

Paragraph 3 — Where you are going (goal + differentiator):

```
I am studying Japanese (JLPT N4) and targeting AIoT engineering roles
in Japan's manufacturing sector by 2029. My combination of hardware
knowledge (EEE diploma) and AI software skills makes me well-suited for
roles that bridge embedded systems and machine learning. Open to remote
AIoT, embedded ML, and IoT software internship opportunities.
```

What NOT to Write on LinkedIn

Avoid This	Why	Write This Instead
"I am a passionate developer"	Everyone says this — means nothing	Show passion through your projects
"Quick learner"	Unverifiable claim	List what you actually learned and built
"Team player"	Corporate buzzword — no value	Mention NIAT team project

		specifically
"Seeking opportunities"	Vague — what opportunities?	"Open to remote AIoT internships"
"Fresher looking for job"	Sounds desperate and junior	"B.Tech CS-IT | Edge AI Engineer in Training"
"Do the needful"	Indian English — not understood globally	Be specific: "Please review my resume"

Chapter 5: Writing Comments and Documentation in Code

Good code comments are as important as good code. Recruiters who look at your GitHub will judge your professionalism by how well-documented your code is. Here are the rules.

5.1 Comment Style Rules

📐 **RULE:** Comments explain WHY, not WHAT. The code already shows WHAT it does. Comments should explain the reason behind the decision.

✗ **WRONG:** *# increment i by 1*

```
i += 1  # ← everyone can see this. The comment adds zero value.
```

✓ **RIGHT:** # Use i+1 because the ADXL345 register map is 1-indexed

```
register = base_addr + i + 1  # ← now the comment explains a non-obvious fact
```

✗ **WRONG:** *# function to read sensor*

```
def read_sensor():
    ...
```

✓ **RIGHT:** # Retry up to 3 times because DHT22 occasionally drops readings on first call

```
def read_sensor(max_retries: int = 3) -> Optional[float]:
    ...
```

5.2 Docstring Format — Every Function Needs This

```
def classify_vibration(window: np.ndarray) -> tuple[str, float]:
    """
    Classify vibration pattern from a window of ADXL345 readings.

    Args:
        window: numpy array of shape (50, 3) — 50 XYZ readings

    Returns:
        Tuple of (class_name, confidence_score)
        class_name: one of "normal", "imbalance_fault", "bearing_fault"
        confidence_score: float between 0.0 and 1.0

    Raises:
        ValueError: if window shape is not (50, 3)

    Example:
        >>> window = np.random.rand(50, 3)
```

```
        >>> label, conf = classify_vibration(window)
        >>> print(f"{label}: {conf:.2%}")
        normal: 94.20%
    """
```

5.3 README Template — What Every GitHub Project Needs

A README is the first thing anyone sees when they visit your GitHub project. It should answer 5 questions: What is this? Why does it matter? What does it use? How do I run it? What were the results?

````
# Project Name

> Short description — what problem this solves in one sentence.

## What This Does
- Reads DHT22 temperature and humidity every 5 seconds via Raspberry Pi GPIO
- Publishes readings to MQTT broker using paho-mqtt (QoS 1)
- Runs TFLite anomaly classifier — 18ms inference on Pi 4
- Sends WhatsApp alert via Twilio if anomaly detected
- Logs all data to MongoDB Atlas for historical analysis

## Tech Stack
| Layer | Technology |
|-------|-----------|
| Hardware | Raspberry Pi 4, DHT22, ADXL345 |
| Edge AI | TFLite, MobileNetV3-Small (INT8) |
| Messaging | MQTT (Mosquitto), paho-mqtt |
| Database | MongoDB Atlas |
| Alerts | Twilio WhatsApp API |
| Dashboard | React + FastAPI |

## Performance
- Inference latency: 18ms (Raspberry Pi 4, 4GB)
- Classification accuracy: 93.4% (test set, 500 samples)
- MQTT throughput: 200 messages/minute sustained

## Quick Start
```bash
git clone https://github.com/username/project-name
cd project-name
pip install -r requirements.txt
cp .env.example .env   # fill in your credentials
python3 main.py
```
````

```
## Demo
[Watch 2-minute demo video](https://youtube.com/your-link)

## What I Learned
1. INT8 quantisation reduces model size by 4x with only 1.2% accuracy loss
2. paho-mqtt needs explicit reconnection logic — auto-reconnect is unreliable
3. MongoDB change streams are more efficient than polling for real-time dashboards
```

🧠 Section 4: Edge AI and Embedded Systems — Explained Simply

Every concept in this section is explained using analogies before any technical detail. If you understand the analogy, you understand the concept. Technical details come after.

Chapter 6: How Neural Networks Actually Work

Most students think neural networks are mysterious black boxes. They are not. They are just mathematical functions with many parameters. Let us understand this properly.

6.1 What is a Neuron? (The Simplest Possible Explanation)

A biological neuron in your brain receives signals from other neurons, adds them up, and if the total is strong enough, it fires and sends a signal forward.

An artificial neuron works exactly the same way:

- It receives numbers (inputs)
- It multiplies each input by a weight (how important is this input?)
- It adds all the weighted inputs together
- If the total passes a threshold, it "fires" — outputs a non-zero number

> 🔄 **ANALOGY — Voting System:** Imagine a committee deciding whether to raise an alarm. Each committee member (neuron) has a vote weight — the expert gets 3 votes, the junior gets 1 vote. Everyone votes yes or no. If the total weighted votes exceed a threshold, the alarm fires. That is exactly how a neuron works.

```
# A single artificial neuron in Python (simplified)
def neuron(inputs: list, weights: list, bias: float) -> float:
    """
    inputs:  [temperature, humidity, vibration]  ← sensor readings
    weights: [0.8, 0.3, 0.9]  ← learned importance of each input
    bias:    -0.5  ← adjusts the threshold
    """
    # Step 1: Multiply each input by its weight and sum
    weighted_sum = sum(x * w for x, w in zip(inputs, weights)) + bias

    # Step 2: Apply activation function (ReLU: if negative, output 0)
    output = max(0, weighted_sum)
    return output

# Example:
temperature = 0.85  # normalised (0 = cold, 1 = very hot)
humidity    = 0.60
vibration   = 0.92
```

```
result = neuron([temperature, humidity, vibration],
                weights=[0.8, 0.3, 0.9],
                bias=-0.5)
print(f"Neuron output: {result:.3f}")  # → 1.178 (fires!)
```

6.2 What is a Neural Network? (Neurons Working Together)

A neural network is just many neurons connected in layers. Each neuron in one layer connects to all neurons in the next layer.

> 🔄 **ANALOGY — Factory Assembly Line:** Layer 1 workers look at raw material (raw sensor data) and extract basic features. Layer 2 workers receive layer 1 outputs and detect patterns in those features. Layer 3 workers receive layer 2 outputs and make a final decision. This is a 3-layer neural network.

For your AIoT sensor classifier:

- Input layer: 8 features extracted from vibration window (mean, max, min, std, RMS, etc.)
- Hidden layer 1: 64 neurons detect basic patterns in the features
- Hidden layer 2: 32 neurons detect more complex combinations
- Output layer: 3 neurons — one per class (normal, imbalance, bearing_fault)
- The class with the highest output neuron value = the prediction

6.3 What is Training? (How Weights Are Learned)

When you first create a neural network, all weights are random — the network predicts nonsense. Training is the process of adjusting the weights until the network predicts correctly.

> 🔄 **ANALOGY — Tuning a Radio:** Imagine tuning an old radio to find a station. You turn the dial slightly, listen to see if it got better or worse, then turn it in the right direction. Training a neural network is the same: adjust weights, check if prediction got better or worse, adjust again. Do this millions of times.

The training process in 5 steps:

Step 1: **Forward pass —** Input goes through the network → produces a prediction

Step 2: **Calculate loss —** Compare prediction to correct answer → compute the error (loss)

Step 3: **Backward pass (backpropagation) —** Calculate how much each weight contributed to the error

Step 4: **Update weights —** Move each weight in the direction that reduces the error (gradient descent)

Step 5: **Repeat —** Do this for all training examples, many times (epochs)

> 📌 **NOTE:** You do not need to implement backpropagation yourself. PyTorch does it automatically with loss.backward(). This is the biggest magic of deep learning frameworks — autograd. Understand what it does, not how it works internally.

6.4 Why Does TFLite Exist? (The Edge AI Problem)

A trained PyTorch model is very large — ResNet50 is 98MB. Running it requires:

- 512MB+ RAM for model weights and activations
- A modern CPU or GPU
- Python + PyTorch installed (500MB+ of libraries)

A Raspberry Pi 4 has 4GB RAM, but other edge devices like ESP32 have only 512KB. Even Pi 4 would struggle with large models. TFLite solves this.

> 🔄 **ANALOGY — Packing a Suitcase:** Your full wardrobe (PyTorch model) is 20kg — too heavy for the flight. So you pack only essential clothes (TFLite conversion) and compress them tightly (quantisation). Same clothes, much lighter, fits in cabin bag.

Property	PyTorch Model	TFLite Model (FP32)	TFLite Model (INT8)
File size	98MB (ResNet50)	~25MB	~6MB
RAM needed	500MB+	~100MB	~30MB
Inference speed (Pi 4)	2000ms	400ms	150ms
Accuracy	100% (baseline)	~99.5%	~98%
Needs Python?	Yes	No (C++ runtime)	No (C++ runtime)
Runs on ESP32?	No	No	With TFLite Micro: Yes

6.5 INT8 Quantisation — Simple Explanation

Normal PyTorch uses 32-bit floating point numbers (FP32). Each weight is stored as a number like 0.73842619... (32 bits = 4 bytes).

INT8 quantisation converts these to 8-bit integers: 0.73842619 → 59 (using a scale factor). 1 byte instead of 4 bytes = 4x smaller.

> 🔄 **ANALOGY — Approximating a Price:** The exact price of a product is ₹847.73. For quick mental math you round it to ₹850. You lost a tiny bit of precision (₹2.27) but you gained massive speed in calculation. INT8 quantisation does the same to model weights.

```
# How to quantise your PyTorch model to INT8 TFLite
# (Run this in Google Colab, not on Pi — it needs full TensorFlow)

import tensorflow as tf

# First: you already have model.onnx from PyTorch export
```

```
# Then: convert ONNX → TF SavedModel → TFLite

converter = tf.lite.TFLiteConverter.from_saved_model("saved_model/")

# Enable quantisation
converter.optimizations = [tf.lite.Optimize.DEFAULT]

# Provide representative data so quantisation knows value ranges
def representative_dataset():
    for i in range(100):
        # Yield batches of your actual sensor data
        yield [training_data[i:i+1].astype("float32")]

converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type  = tf.int8
converter.inference_output_type = tf.int8

# Convert and save
tflite_model = converter.convert()
with open("model_int8.tflite", "wb") as f:
    f.write(tflite_model)

print(f"Original:    {len(fp32_bytes)/1024:.0f} KB")
print(f"Quantised:   {len(tflite_model)/1024:.0f} KB")
```

Chapter 7: Understanding Embedded Systems — Your EEE Advantage

Your EEE diploma knowledge is not outdated — it is your most unique asset. Here is how your existing EEE knowledge maps directly to AIoT engineering.

7.1 Your EEE Knowledge in AIoT Context

EEE Topic You Studied	How It Applies in AIoT	Why It Makes You Unique
Digital electronics (gates, flip-flops)	GPIO logic levels, I2C/SPI timing diagrams	You understand hardware signals CS students do not
Microcontrollers (8051, AVR)	ESP32, Arduino programming — same concepts	You already think in registers and interrupts
AC/DC machines	Vibration signatures of motors — fault patterns	You know what physical failures look like in data
Power electronics	Power supply for Pi, voltage regulators, protection	You can design reliable hardware for edge devices
Signals and systems	FFT for vibration analysis, filter design for sensors	You understand frequency domain — critical for predictive maintenance
Control systems	PID controllers for IoT actuators, closed-loop feedback	You can design and tune automated physical responses
Basic electrical	Ohm's law, safe wiring of sensors to Pi GPIO	You will not burn your hardware. CS students do.

🎯 **REMEMBER:** In a Japan job interview for an AIoT role, saying "I understand motor vibration signatures because I studied AC machines in my EEE diploma" is worth more than 10 LeetCode hard solutions. That cross-domain knowledge is rare and expensive to hire for.

7.2 GPIO Explained Simply — For Your AIoT Projects

GPIO (General Purpose Input/Output) pins are the physical connectors on your Raspberry Pi that let you connect sensors and actuators. Understanding them properly prevents burnt Pi boards.

GPIO Rules — Never Break These

⚠️ **IMPORTANT:** GPIO pins can only handle 3.3V maximum. Connecting a 5V signal directly to a GPIO pin will permanently damage your Pi. Always use a voltage divider or level shifter for 5V sensors.

GPIO Mode	What It Does	Example Use	Max Current
Input	Reads voltage from external device	Read button press, sensor data	N/A
Output (HIGH = 3.3V)	Sends 3.3V signal to	Turn on LED, signal relay	16mA per pin

	external device		
Input with pull-up	Input with internal 3.3V resistor	Buttons, switches	N/A
Input with pull-down	Input with internal GND resistor	Active-high sensors	N/A
PWM output	Rapidly switches on/off to simulate analog	Motor speed control, LED dimming	16mA per pin
I2C (SDA/SCL)	Two-wire protocol for multiple sensors	BME280, ADXL345, OLED display	Depends on device
SPI (MOSI/MISO/SCK/CS)	Four-wire high-speed protocol	MCP3008 ADC, displays	Depends on device
UART (TX/RX)	Serial communication	GPS modules, GSM modules	Depends on device

```
# GPIO programming in Python using gpiozero (beginner-friendly)
from gpiozero import LED, Button, MotionSensor
from signal import pause

led    = LED(17)        # LED connected to GPIO17
button = Button(18)     # Button connected to GPIO18
pir    = MotionSensor(4) # PIR motion sensor on GPIO4

# When button pressed, turn LED on
button.when_pressed  = led.on
button.when_released = led.off

# When motion detected, print alert
pir.when_motion    = lambda: print("Motion detected!")
pir.when_no_motion = lambda: print("No motion")

pause()  # keep program running
```

7.3 I2C Protocol — How Multiple Sensors Share Two Wires

I2C (Inter-Integrated Circuit) is a protocol that allows many sensors to be connected to just TWO wires (SDA and SCL). Each sensor has a unique address (like a house number).

🔄 **ANALOGY — Shared Telephone Line:** I2C is like an old party telephone line where many houses share one line. When the Pi wants to talk to the temperature sensor, it first says the sensor's house number (address 0x77). Only that sensor responds. All others ignore the message.

```
# Scan for I2C devices to find their addresses
# Run in terminal: sudo i2cdetect -y 1
```

```
#
# Output shows a grid. Numbers indicate device addresses:
#      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
# 50: -- -- -- -- -- -- -- 57 -- -- -- -- -- -- -- --
# 70: -- -- -- -- -- -- -- 77
#
# 0x57 = ADXL345 vibration sensor
# 0x77 = BME280 temperature/pressure sensor
# Both on the SAME two wires (SDA=GPIO2, SCL=GPIO3)
```

7.4 Understanding UART — For GPS and GSM Modules

UART (Universal Asynchronous Receiver Transmitter) is the simplest serial protocol. Two wires: TX (transmit) and RX (receive). Used for GPS modules, GSM/4G modules, and debug consoles.

📌 **NOTE:** Important: Cross the wires. Your Pi TX connects to sensor RX. Your Pi RX connects to sensor TX. Think of it as two people talking — one's mouth (TX) connects to the other's ear (RX).

```
# Reading UART data from a GPS module (NEO-6M)
import serial, time

# /dev/ttyAMA0 is the hardware UART on Raspberry Pi
# Baud rate 9600 is standard for GPS modules
gps = serial.Serial("/dev/ttyAMA0", baudrate=9600, timeout=1)

while True:
    line = gps.readline().decode("ascii", errors="ignore").strip()
    if line.startswith("$GPRMC"):  # GPS position sentence
        parts = line.split(",")
        if parts[2] == "A":  # A = valid fix
            lat = float(parts[3]) / 100
            lon = float(parts[5]) / 100
            print(f"Location: {lat:.4f}N, {lon:.4f}E")
```

🇯🇵 Section 5: Japanese N4 Complete Practice Guide

This section has everything you need to reach JLPT N4 level. Grammar explanations are in simple English with AIoT example sentences. Vocabulary lists are grouped by topic. Practice tests included at the end.

Chapter 8: JLPT N4 Grammar — Complete Guide

JLPT N4 has 83 grammar points. We will cover the most important ones with clear explanations and AIoT-relevant example sentences. For each grammar point: explanation → formation rule → example.

8.1 Verb Forms You Must Know

Japanese verbs have many forms. Here are the ones that appear most in N4 grammar:

Verb Form	Name	Example (食べる = to eat)	When Used
食べる	Dictionary form (plain)	食べる	Plain speech, dictionary
食べます	Polite present	食べます	Formal speech, workplace
食べた	Past plain	食べた	Plain past
食べました	Past polite	食べました	Formal past
食べて	Te-form	食べて	Connecting clauses, requests
食べない	Negative plain	食べない	Informal negative
食べません	Negative polite	食べません	Formal negative
食べれば	Conditional (ba-form)	食べれば	If you eat...
食べたら	Conditional (tara-form)	食べたら	When/If you eat...
食べながら	While eating	食べながら	Simultaneous actions
食べてから	After eating	食べてから	Sequential actions
食べている	Currently eating	食べている	Ongoing action/state

8.2 The 20 Most Important N4 Grammar Patterns

1. 〜ている (te-iru) — Ongoing action or current state

Formation: Verb (te-form) + いる

- センサーが動いています。— The sensor is working. (ongoing action)
- プログラムがインストールされています。— The program is installed. (resulting state)
- エラーが発生しています。— An error is occurring. (current situation)

💡 **TIP:** Key question: is the action ongoing RIGHT NOW? → use ている. Is the result of a past action still visible? → also use ている.

2. 〜てから — After doing ~

Formation: Verb (te-form) + から

- コードをテストしてから、デプロイします。— After testing the code, I will deploy it.
- センサーを設置してから、動作確認をします。— After installing the sensor, I will verify it works.
- 会議が終わってから、報告書を書きます。— After the meeting ends, I will write the report.

3. 〜たら — When/If (after completing ~)

Formation: Verb (past plain) + ら. Used for "when X happens, do Y".

- エラーが出たら、ログを確認してください。— If an error appears, please check the log.
- 推論が終わったら、結果を MQTT で送ります。— When inference is complete, I will send the result via MQTT.
- 日本に行ったら、日本語をもっと練習できます。— When I go to Japan, I will be able to practice Japanese more.

4. 〜ば〜ほど — The more ~, the more ~

Formation: Verb (ba-form) + ほど

- 練習すればするほど、上手になります。— The more you practice, the better you become.
- データが多ければ多いほど、モデルの精度が上がります。— The more data there is, the higher the model accuracy.

5. 〜ことができる — Can do ~ / To be able to ~

Formation: Verb (dictionary) + ことができる

- ラズベリーパイで AI を実行することができます。— I can run AI on Raspberry Pi.
- N4 に合格することができました。— I was able to pass N4.
- このシステムで温度を監視することができます。— This system can monitor temperature.

6. 〜ようになる — Come to be able to ~ / Start to ~

Formation: Verb (dictionary/negative) + ようになる. Shows a CHANGE over time.

- 6 ヶ月の練習で、日本語が話せるようになりました。— After 6 months of practice, I became able to speak Japanese.
- 毎日コードを書いて、早く書けるようになりました。— By writing code every day, I became able to write it quickly.

7. 〜ために — In order to / For the purpose of

Formation: Verb (dictionary) + ために

- AIoT エンジニアになるために、毎日勉強しています。— I study every day in order to become an AIoT engineer.

- 日本で働くために、日本語を N2 まで学びます。— I will study Japanese up to N2 in order to work in Japan.
- バグを直すために、ログを確認しました。— I checked the log in order to fix the bug.

8. 〜ながら — While doing ~ simultaneously

Formation: Verb (masu-stem) + ながら. Both actions happen AT THE SAME TIME.

- コーヒーを飲みながら、コードを書いています。— I am writing code while drinking coffee.
- 音楽を聴きながら、勉強しています。— I am studying while listening to music.
- エラーを確認しながら、修正しました。— I fixed it while checking the errors.

9. 〜てもいいですか — May I ~? Is it okay if I ~?

Formation: Verb (te-form) + もいいですか

- このコードを変更してもいいですか？ — May I change this code?
- 明日、休んでもいいですか？ — May I take tomorrow off?
- プロジェクトの期限を延ばしてもいいですか？ — Is it okay to extend the project deadline?

10. 〜てはいけない — Must not / It is not allowed to ~

Formation: Verb (te-form) + はいけない

- API キーを GitHub にアップロードしてはいけません。— You must not upload API keys to GitHub.
- 未テストのコードをデプロイしてはいけません。— You must not deploy untested code.
- 会議中に電話してはいけません。— You must not make phone calls during meetings.

11. 〜なければならない — Must / Have to

Formation: Verb (negative plain without い) + なければならない

- バグを修正しなければなりません。— I must fix the bug.
- 毎日 DSA を練習しなければなりません。— I must practice DSA every day.
- プロジェクトを締め切りまでに完成しなければなりません。— I must complete the project by the deadline.

12. 〜と思います — I think that ~

Formation: Plain sentence + と思います

- このモデルの精度は十分だと思います。— I think this model's accuracy is sufficient.
- MQTT はこのプロジェクトに適していると思います。— I think MQTT is suitable for this project.
- 明日は晴れると思います。— I think it will be sunny tomorrow.

13. 〜かもしれません — Might ~ / Perhaps ~

Formation: Plain sentence + かもしれません (less certain than と思います)

- センサーが壊れているかもしれません。— The sensor might be broken.
- このアルゴリズムはもっと速くできるかもしれません。— This algorithm might be able to be made faster.

14. 〜はずです — Should be ~ (based on expectation)

Formation: Plain sentence + はずです

- 推論は 20ms 以下のはずです。— The inference should be under 20ms.
- 今日、鈴木さんは会社にいるはずです。— Mr. Suzuki should be at the company today.

15. 〜そうです (appearance) — Looks like ~ / Seems like ~

Formation: Verb/Adj (stem) + そうです. Based on visual appearance.

- センサーが壊れそうです。— The sensor looks like it is about to break.
- このコードは動きそうです。— This code looks like it will work.
- 難しそうな問題ですね。— That looks like a difficult problem.

16. 〜ようです / 〜らしいです — It seems that ~ (hearsay or inference)

ようです: based on evidence you observed. らしいです: based on what you heard from others.

- センサーが誤動作しているようです。(I observed this) — The sensor seems to be malfunctioning.
- 来月、新しいモデルが出るらしいです。(I heard this) — Apparently, a new model is coming next month.

17. 〜によって — Depending on ~ / According to ~ / By ~

Formation: Noun + によって

- センサーの種類によって、設定が違います。— The settings differ depending on the sensor type.
- このシステムは AI によって自動化されています。— This system is automated by AI.
- 天気によって、農業 IoT システムの動作が変わります。— The agricultural IoT system's behavior changes according to the weather.

18. 〜について — About ~ / Regarding ~

Formation: Noun + について

- MQTT プロトコルについて説明します。— I will explain about the MQTT protocol.
- このバグについて調査しています。— I am investigating about this bug.
- プロジェクトの進捗について報告します。— I will report regarding the project progress.

19. 〜ていただけますか — Could you please ~ ? (very polite)

Formation: Verb (te-form) + いただけますか. More polite than てください.

- コードをレビューしていただけますか? — Could you please review the code?

- 会議に参加していただけますか？ — Could you please attend the meeting?
- この資料を確認していただけますか？ — Could you please check this document?

20. 〜だけでなく〜も — Not only ~ but also ~

Formation: Noun/Verb + だけでなく + [additional thing] + も

- Python だけでなく C++も使えます。 — I can use not only Python but also C++.
- ハードウェアだけでなく AI ソフトウェアも理解しています。 — I understand not only hardware but also AI software.

8.3 JLPT N4 Practice Test — Vocabulary Section

Try to answer these before looking at the answers. Cover the Answer column.

Question	Choose the correct meaning	Answer
開発する	(A)to study (B)to develop (C)to connect (D)to measure	B — to develop
実装する	(A)to install (B)to implement (C)to break (D)to report	B — to implement
異常検知	(A)normal detection (B)anomaly detection (C)temperature reading (D)error report	B — anomaly detection
予知保全	(A)machine repair (B)emergency stop (C)predictive maintenance (D)quality control	C — predictive maintenance
推論	(A)training (B)dataset (C)inference (D)accuracy	C — inference
閾値	(A)latency (B)threshold (C)frequency (D)voltage	B — threshold
消費電力	(A)battery life (B)power consumption (C)energy storage (D)voltage drop	B — power consumption
精度	(A)speed (B)size (C)accuracy/precision (D)temperature	C — accuracy/precision
遅延	(A)delay/latency (B)bandwidth (C)protocol (D)sensor	A — delay/latency
組み込みシステム	(A)cloud system (B)embedded system (C)mobile system (D)network system	B — embedded system

8.4 JLPT N4 Practice Test — Grammar Section

Sentence	Fill in the blank	Answer
センサーを設置___から、テストします。	してから / したから / するから	してから (after installing...)
データが多い___多いほど、精度が上がります。	ほど / だけ / より	ほど (the more...the more)
このコードを変更して___いいですか?	も / は / が	も (may I change...)
API キーを GitHub にアップロードしては___。	いけません / なりません / できません	いけません (must not...)
日本語が話せる___なりました。	ように / ために / たとえ	ように (became able to...)
推論の速度は 50ms 以下の___です。	はず / よう / そう	はず (should be, expected)
センサーの種類___、設定が違います。	によって / について / のために	によって (depending on...)
MQTT プロトコル___説明します。	について / によって / のに	について (about...)

8.5 N4 Listening Practice — What to Study

The JLPT N4 listening section has three parts:

- Part 1: Short conversations — understand the main topic and key information
- Part 2: Longer monologues — understand the main point and specific details
- Part 3: Quick response — choose the appropriate reply to a question

Free resources for N4 listening practice:

▶️ **YouTube — JLPT N4 Listening Practice (Full Test):** youtube.com/watch?v=jlpt_n4_listening_practice
▶️ **YouTube — NHK World Japanese Lessons (Free, Native Speed):** nhk.or.jp/lesson
▶️ **YouTube — JapanesePod101 N4 Listening:** youtube.com/c/JapanesePod101
🔗 **JLPT Sensei N4 Practice Tests:** jlptsensei.com/jlpt-n4-practice-tests
🔗 **Official JLPT Sample Questions:** jlpt.jp/e/samples/forlearners.html

🎯 Section 6: Career Mastery and Study Habits Science

This section covers the science of how to study effectively (not just hard), how to prepare for internship interviews, and the mindset required to succeed in a 9-year career plan.

Chapter 9: How to Study Effectively — The Science

Most students study the wrong way. They re-read notes, watch tutorials passively, and highlight textbooks. Research shows these methods are among the LEAST effective. Here is what actually works.

9.1 The 4 Most Effective Study Techniques (Research-Backed)

Technique 1: Active Recall (Most Effective)

Instead of re-reading: close the book and try to recall what you just learned from memory.

> 🔄 **ANALOGY — Lifting Weights:** Reading is like watching someone else lift weights — you do not get stronger. Active recall is actually lifting the weights yourself — your brain struggles and gets stronger.

How to apply in your schedule:

- After watching a YouTube tutorial: close it and write down everything you remember
- After a college lecture: spend 5 minutes writing the key points from memory
- For DSA: try to write the solution from scratch before checking
- For Japanese: cover the English column and read Japanese — or reverse

Technique 2: Spaced Repetition (Anki)

You forget things in a predictable curve (Ebbinghaus Forgetting Curve). Spaced repetition shows you each card just before you would forget it — dramatically increasing retention with less time.

Without Anki	With Anki (Spaced Repetition)
Study N5 vocab: 3 hours → remember 70% next day	Study N5 vocab: 1 hour → remember 90% next week
Re-study 2 days later — remember only 30%	Anki shows cards you are forgetting — 85% retention
Re-study again in a week — very hard to recall	Anki schedules next review — feels easy, deep memory
Total time: 10+ hours for same retention	Total time: 3-4 hours — Anki is 3x more efficient

> 💡 **TIP:** Download Anki for free (apps.ankiweb.net). Create one deck per subject: Japanese N4 vocabulary, DSA patterns, Python concepts, MQTT commands. Review every morning for 20 minutes. Consistency matters more than session length.

Technique 3: The Feynman Technique

Richard Feynman was a Nobel Prize winning physicist. His study method: if you cannot explain it simply, you do not understand it yet.

Step 1: Choose a concept — Example: INT8 quantisation

Step 2: Explain it simply — Write/speak as if explaining to a 12-year-old. Use analogies.

Step 3: Find the gaps — Where do you struggle to explain? Those are your weak spots.

Step 4: Go back and fill gaps — Re-study only the parts you could not explain.

Step 5: Simplify further — Remove jargon. If you need jargon to explain it, you do not understand it deeply enough.

Apply this for your Sunday AI sessions: after learning a new tool, spend 10 minutes explaining it to yourself (or record a short voice note) using zero technical jargon.

Technique 4: Interleaving

Instead of studying one subject for 3 hours, alternate between subjects every 45-60 minutes. Research shows interleaving improves long-term retention by 40% compared to blocked practice.

Your evening schedule already does this:

- 5:30-6:15: DSA (Topic A)
- 6:15-7:15: AIoT/Python (Topic B)
- 7:15-8:30: Web/Linux/Tools (Topic C)
- 9:00-9:30: Japanese (Topic D)

> 🎯 **REMEMBER:** This alternating schedule is not just practical — it is optimal for memory formation. Your brain consolidates memories of Topic A while you study Topic B. This is why your 4-month schedule is scientifically designed, not arbitrary.

9.2 The 5 AM Study Session — How to Make It Work

Waking at 4 AM is difficult. Most people fail after 2-3 days. Here is how to make it a permanent habit.

The Night Before — 3 Preparations:

- Preparation 1: Set your clothes, bag, and water bottle the night before. Zero friction in the morning.
- Preparation 2: Write tomorrow's first task on paper and put it next to your alarm. When you wake up, you already know what to do.
- Preparation 3: Sleep by 10 PM. You need 6-7 hours. Less than 6 hours reduces cognitive function by 25% — your studying becomes half as effective.

The First 10 Minutes — Critical Window:

- Do NOT check your phone for the first 10 minutes after waking

- Drink a full glass of water immediately — dehydration causes brain fog
- Splash cold water on face — activates the nervous system
- Start your first task within 10 minutes — the momentum of beginning is everything

⚠️ **IMPORTANT:** The biggest mistake: checking WhatsApp or Instagram immediately after waking. This activates the reward centers in your brain and makes the transition to focused work much harder. Phone stays face-down for the first 30 minutes.

9.3 Dealing With Frustration — When Code Does Not Work

Every engineer experiences this: you write code, it does not work, you do not know why, and you feel like giving up. This is normal. Here is how professionals handle it.

The 15-Minute Rule:

Spend 15 minutes debugging on your own. After 15 minutes with no progress, ask for help immediately — from Google, Stack Overflow, ChatGPT, or Cursor. Struggling for hours alone is not productive. Struggling for 15 minutes before asking is excellent for learning.

The Rubber Duck Method:

Explain your problem out loud to an imaginary person (or a rubber duck). Say exactly what the code is supposed to do, and what it is actually doing. 70% of the time, you find the bug while explaining it.

The 5 Debugging Questions:

- What did I expect to happen?
- What actually happened? (read the error message word by word)
- When did this last work correctly?
- What changed since then?
- Can I make a smaller version that shows the same problem?

🔄 **ANALOGY — Doctor Diagnosis:** A doctor does not just give random medicine. They ask: when did symptoms start? What changed? What makes it worse? Debugging is medical diagnosis for code.

9.4 Interview Preparation — Week-by-Week Plan

This is your 4-week interview preparation plan to use when you have an interview scheduled.

Week	Focus	Daily Activities
Week 1	DSA Intensive	10 LeetCode problems/day. Medium difficulty. Focus on your weakest pattern.
Week 2	Technical depth	Review all projects in detail. Know

		exact numbers: latency, accuracy, throughput.
Week 3	Mock interviews	3 mock DSA interviews using Pramp.com (free) or with a friend. Record yourself.
Week 4	Company research + HR prep	Research the company. Prepare answers to 10 common HR questions. Practice in mirror.

The STAR Format for HR Answers

Every behavioral interview question (Tell me about a challenge, a success, a failure) should be answered using STAR format:

- S — Situation: What was the context? (1-2 sentences)
- T — Task: What was your specific responsibility? (1 sentence)
- A — Action: What did YOU do? Use "I" not "we". (2-3 sentences)
- R — Result: What happened? Use numbers. (1-2 sentences)

Example STAR Answer — "Tell me about a challenge you faced"

```
SITUATION: During my NIAT IoT project, the MQTT broker kept disconnecting
every 2-3 hours, causing us to lose sensor data.

TASK: I was responsible for the communication layer, so fixing
this was my problem to solve.

ACTION: I added verbose logging to find the exact disconnect pattern.
I discovered the broker was timing out because our keepalive interval
was too short for our network. I also implemented a local data buffer
that stored readings when MQTT was down and sent them in batch on reconnect.

RESULT: The system went from 3-4 disconnections per day to zero over
the next 2 weeks. We recovered 100% of previously lost data.
```

Chapter 10: Your Monthly Goals and Accountability System

Goals without a tracking system are wishes. Here is your complete accountability framework.

10.1 The 3-Level Goal System

Set goals at three levels: Monthly (big milestone), Weekly (progress check), Daily (actions).

Level	Example	Review Frequency

Monthly Goal	Complete Projects 1-4 and push to GitHub	Once per month (last Sunday)
Weekly Goal	Solve 15 LeetCode problems and finish Project 2	Every Sunday evening
Daily Goal	Solve 2 LeetCode problems, write 50 lines of project code	Every night before sleep

10.2 The Sunday Review Template

Every Sunday, answer these 7 questions in your notebook or Notion:

```
DATE: _____

1. WINS THIS WEEK (minimum 3):
   a. _____
   b. _____
   c. _____

2. STRUGGLES THIS WEEK (be honest):
   What was hard? _____
   Why was it hard? _____
   How will I handle it differently next week? _____

3. NUMBERS (fill in actual data):
   LeetCode problems solved: ___
   GitHub commits: ___
   Japanese Anki cards reviewed: ___
   Internship applications sent: ___
   Hours of focused study: ___

4. NEXT WEEK'S TOP 3 PRIORITIES:
   Priority 1: _____
   Priority 2: _____
   Priority 3: _____

5. HOW DO I FEEL? (1=exhausted, 5=excellent): ___
   If 1-2: What do I need to recover? _____

6. WHAT AM I GRATEFUL FOR THIS WEEK?
   _____

7. ONE THING I WILL DO DIFFERENTLY NEXT WEEK:
   _____
```

💡 **TIP:** Take a photo of this filled review every Sunday and send it to yourself. After 4 months, you will have 16 photos showing your progress. On a hard day, look at week 1 photo and compare to where you are now. This is powerful motivation.

10.3 Motivation — The Long Game

Nine years is a long time. 2025 to 2033. You will have weeks where nothing works, months where progress feels invisible, and moments where you want to quit.

The truth about long-term success:

- Progress is NOT linear. You will plateau for weeks and then suddenly jump forward.
- Comparison is the enemy. Someone else getting an internship at month 2 does not mean you failed.
- The compound effect is real. Small daily improvements produce extraordinary long-term results.

🔄 **ANALOGY — Bamboo Tree:** A Chinese bamboo tree shows no growth for 4 years after planting — no matter how much water and sun it receives. In the 5th year, it grows 27 meters in 5 weeks. The growth was happening underground (building roots) the entire time. Your first 2 years of B.Tech are your root-building phase.

When you feel like quitting, remember three things:

- Your EEE diploma + CS-IT + Edge AI + Japanese — this specific combination is extremely rare in India. You are building something that cannot be easily replicated.
- The Japan opportunity window is open RIGHT NOW. Japan is aging rapidly and desperately needs engineers. This window will be smaller in 10 years.
- Your Vizag startup will employ people from your hometown. Beyond personal success, you will create opportunities for others.

Your decade starts now. Every line of code you write today is a brick in the career you will live in 2033.

End of Volume 2 — AIoT Engineer Master Guide

Volume 1 + Volume 2 combined: 280+ pages of personalized study material.